
Angular

-

fonctionnalités avancées

Table des matières

I - Evolution d'angular et architectures associées.....	5
1. Evolution d'angular.....	5
2. Eléments stables (angular).....	6
3. Mode standalone et tendance micro-frontend.....	7
II - Cli NX / monorepo.....	8
1. Nx CLI / monorepo.....	8
2. Pertinence de l'utilisation de nx.....	12

III - RxJs, signaux et combinaisons.....	15
1. introduction à RxJs.....	15
2. Fondamentaux de RxJs.....	16
3. Fonctionnalités avancés de RxJs.....	21
IV - ZoneLess et composants avancés.....	28
1. Ancien ZoneJs pour angular 2 à 20.....	28
2. ChangeDetectorRef.....	31
3. Stratégies de détection des changements.....	32
4. Mode ZoneLess (par défaut depuis angular 21).....	32
5. OnPush vs ZoneLess.....	33
6. Eléments avancés sur composants angular.....	34
7. Voir si nécessaire l'annexe de rappels sur les composants (input, output, model , projection).....	37
8. ng-template et TemplateRef.....	38
9. ViewChild / ContentChild.....	41
10. Animations (angular ≥ 20).....	48
V - LazyLoading.....	52
1. Ancien Lazy loading (avec modules).....	52
2. Lazy loading de composant.....	58
3. @defer (avec angular ≥ 17).....	61
VI - Architecture micro-frontend.....	63
1. Micro-frontend avec angular.....	63
VII - Formulaires avancés et dynamiques.....	71
1. Formulaires en mode reactive-form.....	71
2. SignalForms (preview , depuis Angular 21).....	75
VIII - Patterns , gestions d'états avancées (ngrx,...).....	80
1. Extension ngrx.....	80
2. Actions et Effets avec service (NgRx).....	88
IX - Navigations contrôlées (guards, resolvers).....	93
1. Rappels élémentaires sur routing angular.....	93
2. Route conditionnée par gardien.....	97
3. router-outlet annexe (complexe).....	100

4. Resolver et autres gardiens.....	101
X - Tests "angular" (unitaires & e2e).....	104
1. Différent types de tests autour de angular.....	104
2. Test "end-to-end / cypress".....	104
3. Tests "end-to-end / playwright".....	107
4. Attention à la cohérence des tests.....	109
5. Tests unitaires élémentaires.....	110
6. Lancement tests "Angular" avec ng test.....	113
7. Tests unitaires "angular"(composants, service, ...).....	114
XI - Déploiements avec ou sans SSR/SSG.....	124
1. Application isomorphe et optimisation SSR.....	124
XII - Aspects divers et optimisations.....	135
XIII - Annexe – Standalone, signaux et rappels.....	137
1. Structure d'une application angular 2+.....	137
2. Modules applicatifs (maintenant facultatifs).....	138
3. Signaux (depuis angular 16+).....	140
4. Code flow (depuis angular 17).....	149
XIV - Annexe – Rappels sur input/output , projection.....	152
1. input() , ouput() , model() , <ng-content>.....	152
XV - Annexe – Angular library & workspace.....	160
1. Notion de "angular library".....	160
2. Préparation publication de librairies.....	161
3. Programmation de librairie(s) angular.....	162
4. Utilisation d'une librairie angular.....	167
XVI - Annexe – OAuth2 avec angular.....	168
1. Sécurisation d'une application "angular".....	168
2. Token pour appels aux Web-services REST.....	169
3. Authentification via OAuth2/OIDC.....	171
XVII - Annexe – PWA (Progressive Web App).....	177
1. PWA (Progressive Web App) – aperçu général.....	177

2. Web App Manifest / add to home screen.....	180
3. Service-worker et pwa pour Angular.....	186

XVIII - Annexe – extension @angular/material.....195

1. Angular-Material (librairie de composants).....	195
2. Essentiel de "flex-layout" (en intégration angular).....	197
3. Quelques composants "angular-material".....	198

XIX - Annexe – internationalisation angular (i18n).....210

1. internationalisation (i18n).....	210
-------------------------------------	-----

XX - Annexe – Enoncés de TP (angular avancé).....217

1. Préparations et installations.....	217
2. TD "création d'un workspace Nx".....	217
3. TP "début application principale".....	218
4. TP "Service HTTP avec RxJs".....	218
5. TP "composant avec signaux".....	220
6. TP avec Observable et signal.....	221
7. TP "composants réutilisables" dans projet my-lib.....	221
8. TP "lazy loading de composant et sous-routes".....	222
9. TP facultatif/délicat "architecture micro-frontend".....	222
10. TP "formulaire avancé ou dynamique".....	223
11. TP libre "gestion avancée des états (ngrx ou ...).....	224
12. TP "resolver".....	224
13. TP "tests unitaires".....	225
14. TP "test e2e".....	225
15. TP "angular SSR".....	225
16. TP libre "selon intérêts (PWA ou Oauth2 ou i18n)".....	225

I - Evolution d'angular et architectures associées

1. Evolution d'angular

1.1. Époque pré-historique (avant 2015)

- Javascript de niveau es5 (sans Promise, sans module , ...)
- pas encore d'orientation webComponent
- tout début du "frontEnd" séparé du "backend"
- AngularJs (pas encore angular)
- Utilisation interne de jquery light

1.2. es2015+, orientation composant, ts

- Angular avec typescript et modules es2015+
- Orientation composant (@Component) et service (@Injectable)
- choix techniques forts: RxJs/Observable , ZoneJs , ...
- déjà disponibles : pipes, routing, gardiens , intercepteurs , directives, ...
- typescript rarement en version "strict" dans les années 2016-2019
- mode JIT en phase de dev puis AOT en production
- structuration à base de module (@NgModule, ...)
- utilisation interne de webpack pour le build des bundles
- injection par constructeur

1.3. vers 2020: rigueur accrue, meilleurs performances

- Dés angular 9 , restructuration du moteur interne (ivy) et petit à petit AOT partout (en phase de développement et de production)
- Typescript utilisé par défaut en mode "strict" (rigoureux)
- Performances améliorées et tailles des bundles bien optimisées
- Rxjs , Http , routing de plus en plus mature
- LazyLoading de niveau "module" (avec @NgModule)
- Plus de selenium/protractor pour les tests e2e mais préconisation "cypress"
- Cohabitation RxJs/Observable entre Promise/async-await .
- Extensions du moment "vers 2020" : "PrimeNg" , "NgxBootstrap" , "AngularMaterial" , "PWA" , "@angular/universal" , "ionic" , "scss" , "ngrx" , ...

1.4. après 2023: standalone , signaux , zoneless , ...

- Disparition progressive des modules angular (@NgModule de plus en plus obsolète) et potentiel remplacement par le concept émergent d'architecture micro-frontend .
- Composants de plus en plus autonomes (mode "standalone") et donc plus facilement réutilisables . Dépendances plus fines .

- Moins de directives (~~ngIf~~, ~~ngFor~~, ...) et remplacement par nouvelles syntaxes "codeFlow" de angular 17+ (@if, @for, ...)
- Nouveau concept de **signal** (*inspiré du concurrent VueJs*) et utilisation/intégration progressive pour gagner en simplicité et performance (signal(), computed(), ~~@Input~~ → input(), ...)
- Progressif abandon de ZoneJs : mode "ZoneLess"
- LazyLoading de niveau composant (avec éventuels sous composants et sous routes)
- Cohabitation entre RxJs/Observable, Promise/async-await et signaux avec une tendance "de plus en plus de signaux" (meilleure performance)
- Changement progressif de la technologie de build ("~~webpack~~" → "Vite")
- Changement progressif de la technologie de test unitaire ("~~karma, jasmine~~" → "Vitest")
- Tests e2e gérés au dehors d'angular (ex : soit avec cypress, soit avec playwright)
- Plus d'ancienne animation angular, tendance css : @layer, monté de taiwind, ...
- ancienne extension ~~@angular/universal~~ remplacée par @angular/ssr (avec hydratation progressive)
- injection par fonction inject(), de plus en plus d'éléments sous forme de fonctions légères (gardiens, intercepteurs, ...)

2. Éléments stables (angular)

2.1. Concepts clefs les plus stables

- Composant, services et pipes
- Routing et gardiens
- imbrications de sous composants (avec projection, ...)
- tests unitaires
- communications HTTP/REST via service Http et RxJs/Observable

2.2. Besoins récurrents sur la plupart des projets

- Maintenir des bibliothèques d'éléments réutilisables (fonctions utilitaires, composants génériques, templates applicatifs, services d'authentification, ...)
- Bien tester tout ça et faire évoluer cela sur la plupart des projets (sans trop de lourdeur ni de copier/coller) ⇒ pistes : "cli nx" ou "workspace/librairie angular" .
- Extension @angular/ssr
- ...

3. Mode standalone et tendance micro-frontend

Avant le mode standalone (par défaut depuis angular 17) :

- soit petit projet avec un seul module (@NgModule)
- soit gros projet avec plusieurs modules (core , shared , features-a , features-b, ...) et avec lazy loading de niveau module .

Avec le mode standalone (par défaut depuis angular 17) :

- soit petit projet sans module (utilisant si possible des bibliothèques réutilisables)
- soit gros projet idéalement basé sur l'architecture micro-frontend (application angular principale core/host chargeant dynamiquement des micro-frontends dits "remotes" : mfe-a , mfe-b)

En théorie , les différentes applications complémentaires "micro-frontend" sont complètement indépendantes les unes des autres et peuvent fonctionner (au moins partiellement) toute seules.

En pratique, la réutilisation d'éléments communs (composants et services génériques/réutilisables) permet de gagner un peu en productivité/sophistication/... avec quelques avantages et quelques inconvénients : pour bien partager certaines choses , il faut que toutes les applications s'accordent un peu sur quelques points (version minimum , charte graphique , ...) .

II - Cli NX / monorepo

1. Nx CLI / monorepo

1.1. Présentation de Nx CLI et monorepo

*Nx CLI est une **extension angular** permettant de développer et maintenir des **applications évoluées** (par exemple avec plusieurs sous-projets dans le cadre d'une architecture micro-frontend avec éventuelle librairie partagée).*

Nx CLI peut être vu comme une **version améliorée** de Ng (Angular CLI).

Nx reprend à peu près toutes les fonctionnalités de Ng et ajoute essentiellement les fonctionnalités suivantes :

- **monorepo** : gestion de référentiel unique comportant plusieurs projets
- plugins de type "**generators**" et "**executors**"
- intégration d'outils de tests modernes (Vitest , Playwright , Cypress , ...)
- gestion de caches , ...

Nx est en fait un **écosystème javascript/typescript complet** qui supporte plein de technologies via des **plugins** (angular, react, vue, node , ...). Nx est le principal produit de nrwl.io .

Installation & vérification :

```
npm add --global nx
nx --version
```

Pour éventuellement créer un nouveau référentiel nx vide la commande et `npx nx@latest init`

NB : un **repo/workspace "nx"** est constitué au minimum de package.json et **nx.json**

Plein de possibilités :

- créer un référentiel/workspace nx vide puis ajouter angular dedans
- créer d'un seul coup un projet simple nx+angular et si besoin le faire évoluer ultérieurement en mode "multi-projets/monorepo" .
- créer dès le départ un "monorepo" pour héberger plusieurs sous projets complémentaires

1.2. Nx et angular

Vis à vis d'angular , Nx peut être vu comme une petite surcouche fine qui :

- apporte un tout petit peu de complexité/spécificités supplémentaires
- qui est **surtout utile** pour des grosses **applications** (ou petites applications à bien structurer et maintenir sur le long terme) **qui utilisent des librairies partagées** (services et composants réutilisables).

Précisions structurelles :

- Tout projet nx est structuré comme un répertoire "**workspace nx**" (avec fichier package.json et **nx.json**)
- *Certains rares petits projets nx isolés ne sont pas décomposés en sous projet*

- La principale valeur ajoutée de Nx étant le partage de sous parties (et facilité de build/tests/déploiement CI_CD) , la plupart des workspaces nx sont en mode "monorepo" (au sens git et nx).
- Un "**workspace nx monorepo**" c'est avant tout une structure ou écosystème et ça peut comporter tout un tas de sous parties complémentaires :
 - **librairies angular**
 - **application(s) angular**
 - **éventuel workspace angular** comportant lui même des sous parties (librairies , app, ...)

Création d'un projet simple (sans sous parties) :

```
npx create-nx-workspace myngapp --preset=angular-standalone
```

Let's create a new workspace [https://nx.dev/getting-started/intro]

√ Which bundler would you like to use? · **esbuild**

√ Default stylesheet format · **css**

√ Do you want to enable Server-Side Rendering (SSR) and Static Site Generation (SSG/Prerendering)? · **No**

√ Which unit test runner would you like to use? · **vitest-angular**

√ Test runner to use for end to end (E2E) tests · **playwright**

→ structure angular habituelle mais avec **project.json** à la place de ~~angular.json~~

→ lancement (en mode dev) via **npx nx serve**

1.3. nx monorepo et angular

1) création d'un workspace nx en mode "angular monorepo" :

```
npx create-nx-workspace@latest my-nx-workspace --preset=angular-monorepo
```

2a)création (si nécessaire) d'un sous projet de type application angular :

```
nx g @nx/angular:app apps/my-app
```

2b)création d'un sous projet de type librairie angular :

```
nx g @nx/angular:lib libs/my-lib
```

avec éventuelle option importante:

--buildable

ou encore mieux :

--publishable --importPath myorg-mylib

--publishable --importPath @myorg/mylib (pour futur npm publish avec ou pas --access public)

3)Visualisation des projets créés :

nx show projects

nx show project my-app

Attention : un mauvais réglage de .gitignore peut perturber la visibilité des projets

Si besoin : renommer temporairement .gitignore (puis ultérieure restauration)

4) lancement ou test :

nx serve my-app

nx test my-app
nx test my-lib

5) ajouter composant:

```
nx g @nx/angular:component apps/my-app/src/app/cxy/cxy --type=component
```

ou bien

```
set NX_DEFAULT_PROJECT=my-app  

cd apps/my-app/src/app  

nx g @nx/angular:component cxy/cxy --type=component
```

6) ajouter un service :

```
set NX_DEFAULT_PROJECT=my-lib ou my-app  

nx g @nx/angular:service common/service/context --type=service
```

1.4. partage de librairie

Le partage de sous projets "librairies" entre applications d'un workspace nx constitue l'une des principales valeurs ajoutées de "Nx" (c'est plus ou moins sa raison d'être) .

Mode opératoire :

1) au sein d'un projet de type librairie (ex : libs/my-lib) , il faut si besoin ajouter le fichier **package.json** (s'il existe pas encore) avec un contenu de de genre :

```
{  
  "name": "@my-nx-workspace/my-lib" ou "xx-lib-yy" , ...  
}
```

2) au sein d'un projet de type application ayant besoin de cette librairie (ex : apps/my-app) , il faut si besoin ajouter le fichier **package.json** (s'il existe pas encore) avec un contenu de de genre :

```
{  
  "name": "@my-nx-workspace/my-app",  
  "dependencies": {  
    "@my-nx-workspace/my-lib": "workspace:*"  
  }  
}
```

3) au sein du sous projet "librairie" (ex : libs/my-lib) , il faudra penser à mettre à jour le fichier **src/index.ts** avec un contenu de ce genre :

```
//export * from './lib/my-lib/my-lib';  

export * from './lib/service/xxx.service';  

export * from './lib/service/yyy.service';
```

4) au sein d'un projet applicatif utilisant la librairie (dans ce même workspace nx), on pourra enfin (et simplement) effectuer des *imports typescripts* de cette façon :

```
...  

import {XxxService} from '@my-nx-workspace/my-lib';
```

```
@Component({....})
export class WelcomeComponent {
  public xxxService = inject(XxxService);
  ...
}
```

1.5. Lancement des tests via nx

Lancement d'un test unitaire angular :

```
nx test my-app
nx test my-app --include=**/financial.service.spec.ts
nx test my-app --include=**/financial.component.spec.ts
```

Lancement d'un test e2e avec playwright :

Remarque importante :

nx créer à coté du sous projet applicatif **xxx-app** un sous projet de test e2e de nom **xxx-app-e2e** .

npx playwright install (au préalable si besoin)

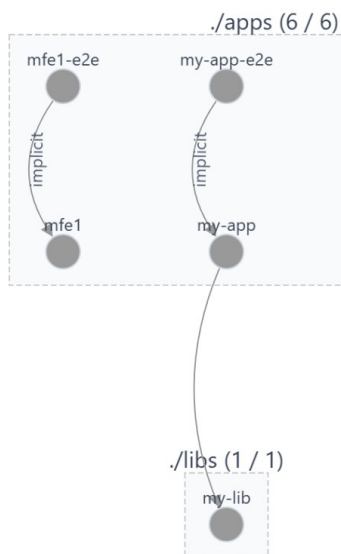
nx e2e my-app-e2e

et pas absolument besoin de lancer préalablement **nx serve my-app**

1.6. Affichage du graphe des dépendances

npx nx graph

génère un graphe de dépendance que l'on consulte dans un navigateur :



2. Pertinence de l'utilisation de nx

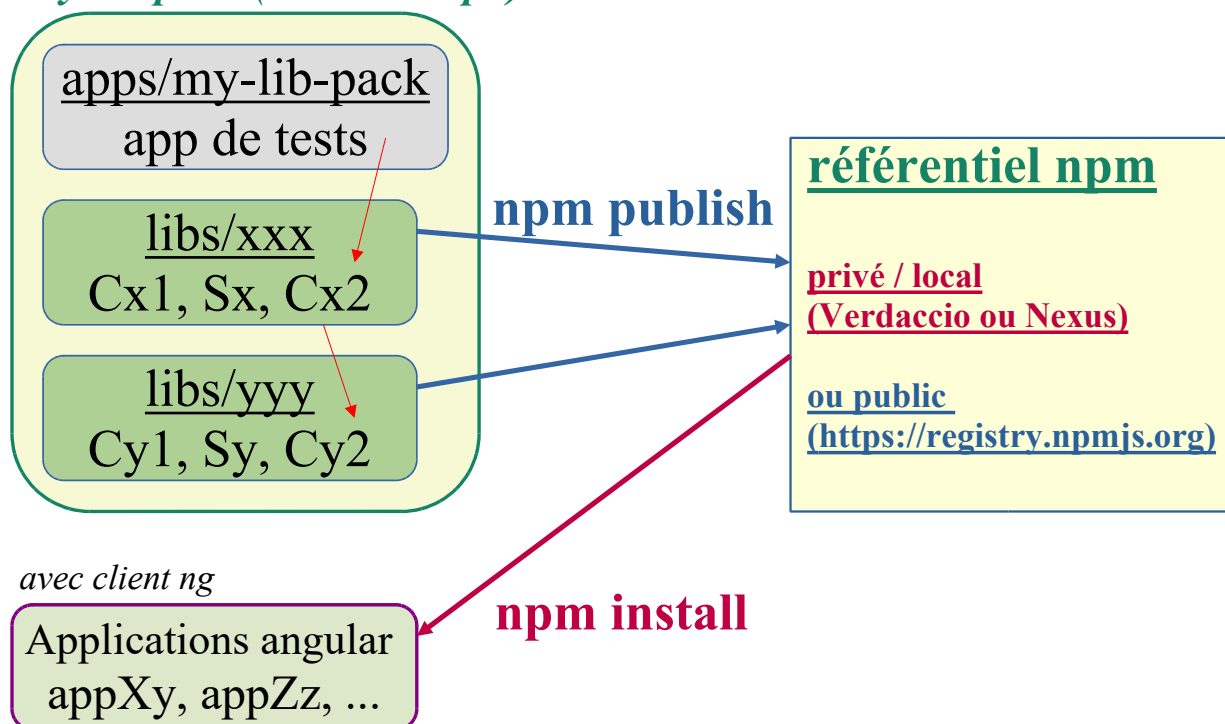
Points forts de Nx	Dépendances inter-projets , graphes des dépendances , programmation et tests de bibliothèques de composants/services réutilisables.
Points faibles de Nx	Ce n'est pas un projet officiellement maintenu par google/angular (comme ng) mais un extension tierce partie (développée par nrwl.io) avec une moins grande pérennité . Un peu plus de bugs que "ng" , documentation pas parfaitement à jour .

Idee (à débattre) :

- Toute application importante devrait idéalement s'appuyer sur le client "ng" habituel pour des raisons de stabilité et pérennité .
- *Un paquet de bibliothèques partagées (souvent complémentaires avec des inter-dépendances) plus l'indispensable application de test qui va avec peut avantageusement être codé via le client nx (avec le mode --publishable pour chaque bibliothèque).*

Nx monorepo pour développer un paquet de bibliothèques inter-dépendantes et une application de test

my-lib-pack (nx monorepo)



nx monorepo angular Workspace my-pack created by

```
npx create-nx-workspace@latest my-pack --preset=angular-monorepo
```

avec app/my-pack for testing

```
nx g @nx/angular:lib libs/xxx --publishable --importPath my-pack-xxx
nx g @nx/angular:lib libs/yyy --publishable --importPath my-pack-yyy
```

NB: la valeur de `--importPath` correspond au nom officiel de la librairie qui sera stockée dans un référentiel npm et qui sera ultérieurement importée dans les futures applications angular qui en auront besoin.

Construction et publication des librairies :

nx build xxx

nx build yyy

cd dist/libs/xxx

npm login

npm publish (or `npm publish --access public` if scoped)

=====

NB: for publishing to public npm set registry <https://registry.npmjs.org> need 2FA key (my-pack-npm Registered in bitwarden) during npm login phase (in browser) , just before npm publish

start_verdaccio	REM npm install -g verdaccio verdaccio
set_verdaccio_registry	REM npm install -g verdaccio REM verdaccio REM npm adduser --registry http://localhost:4873 REM (user1/pwd1 user1@xyz.com or ...) call npm set registry http://localhost:4873/ npm login REM (user1/pwd1 or ...) REM npm whoami
restore_npmjs_registry	npm set registry https://registry.npmjs.org
show_current_registry	call npm config get registry pause

NB:

- publier vers le référentiel public (par défaut) <https://registry.npmjs.org> nécessite la création d'un compte sur le site npmjs .
- On ne peut publier (vers verdaccio local ou autre) que des packages dont la version est au moins supérieure ou égale au dernier numéro de version enregistré .

Exemple de fichier **libs/yyy/package.json** :

```
{
  "name": "my-pack-yyy",
  "version": "0.0.4",
  "peerDependencies": {
    "@angular/common": "^21.2.0",
    "@angular/core": "^21.2.0",
    "my-pack-xxx": "^0.0.4"
  },
}
```

```
"sideEffects": false,  
"dependencies": { }  
}
```

III - RxJs, signaux et combinaisons

1. introduction à RxJs

1.1. Principes de la programmation réactive

La programmation réactive consiste essentiellement à programmer un enchaînement de traitements asynchrones pour réagir à un flux de données entrantes .

Un des intérêts de la programmation réactive réside dans la souplesse et la flexibilité des traitements fonctionnels mis en place :

- En entrée, on pourra faire librement varier la source des données (jeux de données statiques , réponses http , input "web-socket" , événement DOM/js ,)
- En sortie, on pourra éventuellement enregistrer plusieurs observateurs/consommateurs (si besoin de traitement en parallèles . par exemple : affichages multiples synchronisés) .

1.2. RxJs (présentation / évolution)

RxJs est une bibliothèque **javascript** (assimilable à un mini framework) spécialisée dans la programmation réactive .

Il existe des variantes dans d'autres langages de programmation (ex : RxJava , ...) .

1.3. Principales fonctionnalités d'un "Observable"

Observable est la structure de données principale de l'api "RxJs" .

- Par certains cotés , un "Observable" ressemble beaucoup à un objet "Promise" et permet d'enregistrer élégamment une suite de traitements à effectuer de manière asynchrone .
- En plus de cela , un "Observable" peut facilement être manipulé / transformé via tout un tas d'opérateurs fonctionnels prédéfinis (ex : map , filter , sort , ...)
- En outre , comme son nom l'indique , un "Observable" correspond à une mise en oeuvre possible du design pattern "observateur" (différents observateurs synchronisés autour d'un même sujet observable).

1.4. Fonctionnement (sources et consommations)

```
Source_configurée_et_initialisée
.pipe(
  callback_fonctionnelle_1 ,
  callback_fonctionnelle_2 ,
  ....
).subscribe({
  next : callback_success ,
  error : callback_error ,
  terminate : callback_terminate
})
```

```
Source_configurée_et_initialisée
.pipe(
  callback_fonctionnelle_1 ,
  callback_fonctionnelle_2 ,
  ....
).subscribe(callback_success );
```

NB: la partie .pipe() est facultative et ne sert qu'à enchaîner une éventuelle série de traitements intermédiaires .

NB : les parties *error : callback_error* et *terminate : callback_terminate* de `.subscribe()` sont **facultatives** et peuvent donc être omises si elles ne sont pas utiles en fonction du contexte.

NB : il faut (en règle générale, sauf cas particulier/indication contraire) appeler `.subscribe()` pour que la chaîne de traitement puisse commencer à s'exécuter.

1.5. Convention de nom pour "return Observable"

Certains développeurs angular/RxJs ont pris l'habitude de mettre le **suffixe \$ à la fin des noms de méthodes qui retournent en interne un objet Observable**.

Exemple partiel:

```
public getAllDevises$() : Observable<Produit[]>{
    //const url = `${this.publicBaseUrl}/devises`;
    //console.log( "url = " + url);
    //return this._http.get<Devises[]>(url);
    return of(this.tabProduit) ;
}
```

2. Fondamentaux de RxJs

2.1. Imports dans projet Angular / typescript

```
import { Observable, of } from 'rxjs';
import { map, mergeMap, toArray, filter } from 'rxjs/operators';
```

exemple d'utilisation (dans classe de Service) :

```
public rechercherProduitSimu$(prixMaxi : number) : Observable<Produit[]> {
    let tabProduit = [
        { numero : 1, label : "produit 1", prix : 50 },
        { numero : 2, label : "produit 2", prix : 30 },
        { numero : 3, label : "produit 3", prix : 80 },
        { numero : 4, label : "produit 4", prix : 500 }
    ]
    return of(tabProduit)
        .pipe(
            mergeMap(pInTab=>pInTab),
            filter((p) => p.prix <= prixMaxi),
            map((p : Produit)>=>{p.label = p.label.toUpperCase(); return p;}),
            toArray()
        );
}
```

2.2. Données statiques (tests temporaires , cas très simples)

```
let jsObject = { p1 : "val1" , p2 : "val2" } ;
of(jsObject)...subscribe(...) ;

let tabObj = [ { ...} , { ... } ] ;
of(tabObj)...subscribe(...) ;

of(value1 , value2 , ... , valueN)...subscribe(...) ;
```

2.3. Données numériques : range(startValue,endValue)

```
range(1, 10).subscribe(x => console.log(x)) ;
1
2
...
10
```

NB: la plupart des sources ci-après existent au niveau de RxJs mais sont très rarement utilisées par angular (qui fait autrement, à sa manière).

2.4. Principaux opérateurs (à enchaîner via pipe)

Rappel (syntaxe générale des enchaînements) :

Source configurée et initialisée

```
.pipe(
  callback_fonctionnelle_1 ,
  callback_fonctionnelle_2 ,
  ....
) .subscribe({
  next : callback_success ,
  error : callback_error
})
```

avec plein de variantes possibles

Principaux opérateurs :

map	Transformations quelconques (calculs , majuscules , tri ,)
flatMap , mergeMap	Tableau réactif devient flux réactif des éléments du tableau
toArray	flux réactif d' éléments devient Tableau réactif
filter	Filtrages (selon comparaison,)
tap	Traitement supplémentaire en parallèle sur les données du flux réactif mais sans changer la valeur de retour

2.4.a. map() : transformations

En sortie , résultat (retourné via return) d'une modification effectuée sur l'entrée .

Exemple 1:

```
const obsNums = of(1, 2, 3, 4, 5);
const squareValuesFunctionOnObs = map((val) => val * val);
const obsSquaredNums = squareValuesFunctionOnObs(obsNums);
obsSquaredNums.subscribe(x => console.log(x));
// affiche 1 4 9 16 25
```

Exemple 2:

```
const obsStrs = of("un", "deux", "trois");
obsStrs.pipe(
  map( (s) => s.toUpperCase() )
)
.subscribe(s => console.log(s));
// affiche UN  DEUX  TROIS
```

2.4.b. mergeMap() et toArray()

De façon à itérer une séquence d'opérateurs sur chaque élément d'un tableau tout en évitant une imbrication complexe et peu lisible de ce type :

```
observableSurUnTableau
.pipe(
  map((tableau) => {
    return tableau.map(
      (itemInTab) => { itemInTab.label = itemInTab.label.toUpperCase();
                      return itemInTab; }
    );
  })
);
```

on pourra placer une séquence d'opérateurs qui agiront sur chacun des éléments du tableau entre `mergeMap()` et `toArray()` :

```
observableSurUnTableau
.pipe(
  mergeMap(itemInTab => itemInTab),
  filter((itemInTab) => itemInTab.prix <= 300),
  map((itemInTab) => { ... }),
  toArray()
).subscribe( (tableau) => { ... } );
```

2.4.c. filter()

```
const obsVals = of(12, -15, 30, -8, 40);
obsVals.pipe(
  filter( (v) => v >= 0 )
)
```

```
.subscribe(v => console.log(v));
```

//affiche 12 30 et 40

```
range(1,10).pipe(
  filter( (v) => v % 2 === 0 )
)
.subscribe(v => console.log(v + " est une valeur paire"));
```

2.4.d. map with sort on array

```
const obsTab = of([ {numero:1,label:'produit1',prix:40.0},
  {numero:2,label:'produit2',prix:30.0},      {numero:3,label:'produit3',prix:35.0},
  {numero:4,label:'produit4',prix:15.0},      {numero:5,label:'produit5',prix:35.0}
]);
obsTab.pipe(
  map( (tab) => tab.sort( (p1,p2) => (p1.prix - p2.prix) ) )
  // map( (tab) => tab.sort( (p1,p2) => p1?p1.label.localeCompare(p2.label):0 ) )
)
.subscribe(t => console.log( JSON.stringify(t) ));
```

2.5. Passerelles entre "Observable" et "Promise"

2.5.a. Source "Observable" initiée depuis une "Promise" :

```
import { from } from 'rxjs';

// Create an Observable out of a promise :
const observableResponse = from(fetch('/api/endpoint'));

observableResponse.subscribe({
  next(response) { console.log(response); },
  error(err) { console.error('Error: ' + err); },
  complete() { console.log('Completed'); }
});
```

2.5.b. Convertir un "Observable" en "Promise"

observableXy.**toPromise()**.then(...).catch(...)
is now deprecated.
Use **lastValueFrom()** or **firstValueFrom()** instead .

Exemple :

```
async onConvertirV2(){
  try{
    this.montantConverti = await firstValueFrom(this._deviseService.convertir$(this.montant,
      this.codeDeviseSource,
      this.codeDeviseCible));
```

```
    }catch(err){  
      console.log(err);  
    }  
  }  
}
```

NB: firstValueFrom() déclenchera automatiquement un "unsubscribe" dès que possible .

2.6. Optimisations et précautions

```
myObservable.pipe(take(1)).subscribe(x => { console.log(x);});
```

Seul ou bien en fin de pipe() , l'opérateur **take(1)** permet un déclenchement automatique d'un "**unsubscribe**" pour éviter au mieux les risques de fuite de mémoire .

Par défaut, **toSignal()** se désabonne automatiquement de l'Observable lorsque le composant ou le service qui le crée est détruit .

Attention : pas de unsubscribe automatique avec seulement .pipe(shareReplay(1)) (pour hot observable partagé) , besoin du pipe "async" ou autre .

NB : le pipe **async** d'angular **déclenche également automatiquement un "unsubscribe"** lorsque le composant est détruit ce qui en fait une bonne pratique très recommandée dans la plupart des cas :

```
@Component({ ... imports: [AsyncPipe , ...], ...})
export class XxxComponent {
  montantConvertiObs$!: Observable<number>;
  ...
  onConvertir() {
    this.montantConvertiObs$ = this._deviseService.convertir$(this.montant,
      this.codeDeviseSource, this.codeDeviseCible) ;
  }
}
```

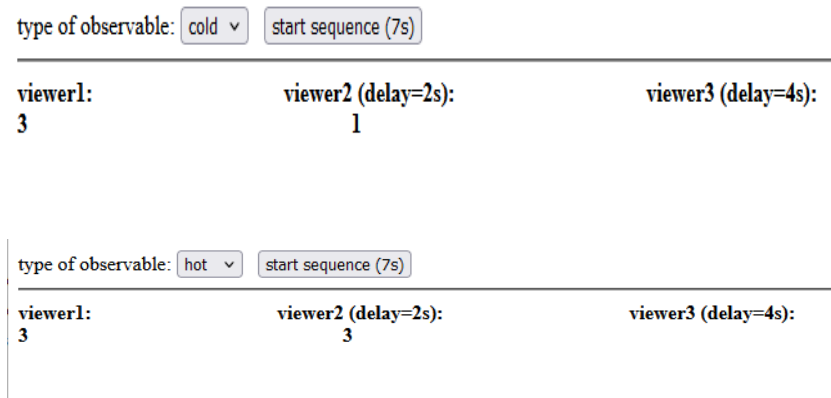
```
<input type="button" (click)="onConvertir()" value="convertir" /> <br/>
montantConverti = {{montantConvertiObs$ | async}}
```

3. Fonctionnalités avancées de RxJs

3.1. Hot Observable

Type d'observable	comportement
cold (by default) observable	Flux pas partagé / pas synchronisé . Une instance du flux observable pour chaque consommateur
hot observable	Flux partagé et synchronisé . Si plusieurs consommateurs (en mode "multicast"), ceux-ci reçoivent alors les mêmes données . Un "unsubscribe" commun/global sera automatiquement différé (en attendant la fin de la dernière consommation)

Exemple :



fonction qui enregistre un consommateur "viewer 1_ou_2_ou_3" vis à vis d'un observable *\$obs* en effectuant cet enregistrement en décalé dans le temps : *delay* ms et qui programme un unsubscribe *subscription timeout* ms après le début de l'enregistrement :

```
subscription_for_viewer=(viewerId,obs$,delay,subscription_timeout)=>{
  setTimeout(_=>{
    const subscription = obs$.subscribe(
      (v)=> {
        let msg = viewerId+": "+v;
        console.log(msg);
        document.getElementById(viewerId).innerHTML=v;
      }
    );

    setTimeout(_=> {
      subscription.unsubscribe();
    }, subscription_timeout);
  }, delay);
}
```

```
document.getElementById("btnStart").addEventListener("click" , ()=>{
  ...

  const coldObs$ = new rxjs.Observable((observer) => {
    //initialize producer:
    let currentNumber = 1;
    const producer = setInterval(_=> {
      document.getElementById("spanLastProduced").innerHTML=currentNumber;
      observer.next(currentNumber++); //envoi la valeur (++) de currentNumber toutes les 1s
    }, 1000);

    //returning function that will be called when unsubscribe:
    return () => {
      console.log("end of subscription");
    };
  });
}
```

```
        clearInterval(producer);
    }
});

//build hot observable from cold :
let hotObs$ = coldObs$.pipe(rxjs.share());

let coldOrHot = document.getElementById("selectColdOrHot").value;
let obs$=(coldOrHot=="hot"?hotObs$:coldObs$;

subscription_for_viewer("viewer1",obs$,0,7000);//sans délai
subscription_for_viewer("viewer2",obs$,2000,7000); //démarrage différé dans 2s
subscription_for_viewer("viewer3",obs$,4000,7000);//démarrage différé dans 4s

});
```

Comportement :

- en version "cold" , plusieurs instances du flux , lorsque le viewer1 reçoit 3 , le viewer 2 (au démarrage décalé) reçoit 1
- en version "hot" , une seule instance du flux , lorsque le viewer1 reçoit 3 , le viewer 2 reçoit également 3

3.2. Observable d'ordre 2

"observables of observables" = "High order observables"

```
outerObservable.pipe(
  concatMap_or_xyzMap(innerObservable),
  ....)
.subscribe(... données globalement récupérées ...);
```

Variantes	Comportements
concatMap (innerObs)	concaténation après un déclenchement d'une boucle sur le outer_observable et bufferisation . Ordre contrôlé au sein du résultat global (groupBy sur outer observable)
mergeMap (innerObs)	fusion avec sous déclenchements en parallèles au fur et à mesure (multiple sous-subscribes simultanés) Ordre incontrôlé au sein du résultat global
switchMap (innerObs)	Sous déclenchements séquentiels (subscribe , unsubscribe/cancel_new, subscribe, unsubscribe/cancel_new , ...) Interruption du traitement du sous niveau courant dès qu'arrive un nouvel élément du niveau principal
exhaustMap (innerObs)	comme switchMap mais ignore nouvelle occurrence du niveau principal si le traitement du sous niveau courant n'est pas encore terminé

Exemple :

```
let msg = "";
let wsUrlArray = [ 'https://catfact.ninja/fact' , ' https://geo.api.gouv.fr/communes?codePostal=78000',
  ' https://geo.api.gouv.fr/communes?codePostal=76000' , ' https://geo.api.gouv.fr/communes?codePostal=80000'
];

let mode =document.getElementById("selectMode").value;
let mapFunction;
switch(mode){
  case "mergeMap" : mapFunction=rxjs.mergeMap; break;
  case "switchMap" : mapFunction=rxjs.switchMap; break;
  case "exhaustMap" : mapFunction=rxjs.exhaustMap; break;
  case "concatMap" :
    default: mapFunction=rxjs.concatMap;
  }

rxjs.of(... wsUrlArray)
.pipe(
  //in this pipe : url as Observable<string> : first level of observable things :outer observable
  mapFunction(
    url => rxjs.ajax.ajax(url)
    //http.get(url) or ajax(url) return a second level of observables things (http responses) : inner observable
  ),
  rxjs.map((rxjsAjaxResp)=> "status=" + rxjsAjaxResp.status
    + " jsonData=" + JSON.stringify(rrxjsAjaxResp.response))
).subscribe({
  next : x => { msg += (x+'<br/>'); } ,
  error : e => console.log(e) ,
  complete : () => dualDisplay("spanExHighOrderObs",msg)
});
```

Au sein de cet exemple , le premier niveau d'observable (produit via of(...)) est un tableau d'url .

Le sous niveau d'observable (construit via ajax(...)) est une réponse à une api rest (ici en mode get

par défaut).

Comportements de l'exemple :

<i>mode</i>	<i>résultat</i>	<i>comportement</i>
concatMap	<pre>status=200 jsonData={"fact":"A cat can travel at a top speed of approximately 31 mph (49 km) over a short distance.", "length":86} status=200 jsonData=[{"nom":"Versailles","code":"78646","codeDepartement":"78","siren":"217806462","codeEpci":"247800584","codeRegion":"11","codesPostaux":["78000"],"population":83587}] status=200 jsonData=[{"nom":"Rouen","code":"76540","codeDepartement":"76","siren":"217605401","codeEpci":"200023414","codeRegion":"28","codesPostaux":["76000","76100"],"population":114083}] status=200 jsonData=[{"nom":"Amiens","code":"80021","codeDepartement":"80","siren":"218000198","codeEpci":"248000531","codeRegion":"32","codesPostaux":["80000","80080","80090"],"population":133625}]</pre>	Même ordre que déclenchement
mergeMap	<pre>status=200 jsonData=[{"nom":"Versailles","code":"78646","codeDepartement":"78","siren":"217806462","codeEpci":"247800584","codeRegion":"11","codesPostaux":["78000"],"population":83587}] status=200 jsonData=[{"nom":"Rouen","code":"76540","codeDepartement":"76","siren":"217605401","codeEpci":"200023414","codeRegion":"28","codesPostaux":["76000","76100"],"population":114083}] status=200 jsonData=[{"nom":"Amiens","code":"80021","codeDepartement":"80","siren":"218000198","codeEpci":"248000531","codeRegion":"32","codesPostaux":["80000","80080","80090"],"population":133625}] status=200 jsonData={"fact":"Relative to its body size, the clouded leopard has the biggest canines of all animals' canines. Its dagger-like teeth can be as long as 1.8 inches (4.5 cm).", "length":156}</pre>	ordre différent que celui du déclenchement car api geo plus rapide que catfact
switchMap	<pre>status=200 jsonData=[{"nom":"Amiens","code":"80021","codeDepartement":"80","siren":"218000198","codeEpci":"248000531","codeRegion":"32","codesPostaux":["80000","80080","80090"],"population":133625}]</pre>	Premier(s) pas rapide(s) interrompu(s) et récupération du/des dernier(s)
exhaustMap	<pre>status=200 jsonData={"fact":"During the time of the Spanish Inquisition, Pope Innocent VIII condemned cats as evil and thousands of cats were burned. Unfortunately, the widespread killing of cats led to an explosion of the rat population, which exacerbated the effects of the Black Death.", "length":259}</pre>	Attente du premier (pas rapide) et autres appels pas gérés

3.3. Old BehaviorSubject (existant avant les signaux)

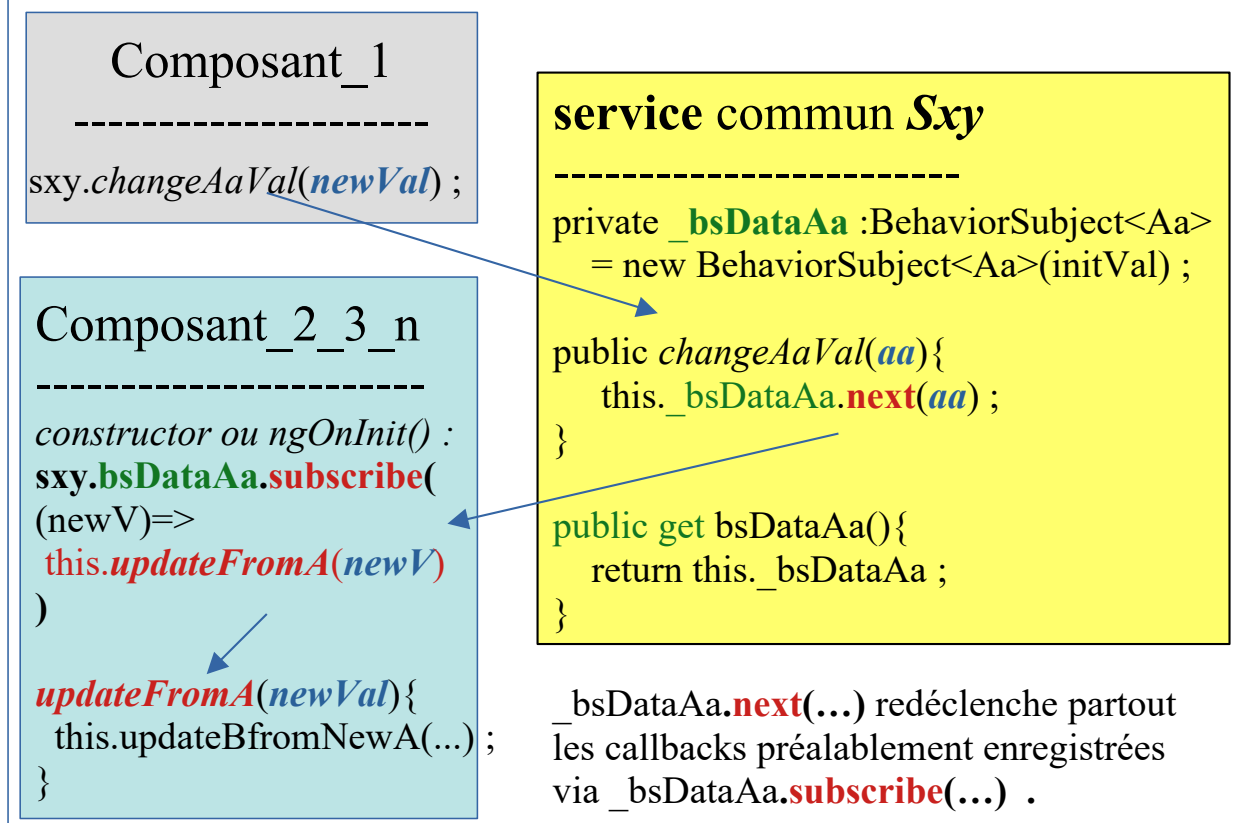
NB: un objet de type **BehaviorSubject**<...> doit avoir une valeur initiale dès sa construction. C'est une chose "**Observable**" depuis plusieurs composants de l'application.

Dès que la valeur sera modifiée, tous les observateurs seront automatiquement synchronisés.

NB: Maintenant qu'il existent des signaux qui font à peu près la même chose mais de manière plus simple et légère, il est préférable d'utiliser `signal()` et `computed()` à la place de **BehaviorSubject**.

BehaviorSubject est une implémentation RxJs/Angular du **design pattern "observateur"**.

BehaviorSubject<T> Observable



Principe de fonctionnement :

1. initialisations : chaque composant enregistre une callback (qui sera ultérieurement appelée une ou plusieurs fois) via un appel à `sxy.bsDataAa.subscribe(...)`
2. Lors de la première initialisation du **BehaviorSubject** et à chaque appel de `this._bsDataAa.next(aa)` ; les automatismes du framework RxJs vont redéclencher toutes les callbacks préalablement enregistrées
3. Via des callbacks spécifiquement adaptées à chacun des composants, chaque composant peut ainsi tenir compte de la nouvelle valeur de `aa` qui vient de changer pour réagir de manière adéquate et bien synchronisée (soit simplement recopier/afficher la valeur de `aa` qui vient de changer, soit par traitement actualiser une donnée B,C ou D devant rester cohérente avec celle de `aa`).

Signaux (plus léger , plus moderne) : à développer via l'annexe adéquate .

Combinaisons possibles "RxJs+signaux" :

- RxJs pour l'instant bien approprié pour les aspects asynchrones (ex : appels HTTP, ...)
- Signaux bien appropriés pour ré-affichages réactifs
- combinaison en mode "interop" quelquefois intéressante .

Observable combiné avec toSignal pour données systématiquement demandées :

```
import { toSignal } from '@angular/core/rxjs-interop';
...
export class XYZComponent {
  xyzService = inject(XyzService) ;

  //code systématiquement déclenché (comme si dans constructeur) :
  tabClientsObservable$ = this.xyzService.getTabInscriptions$() ;

  //code systématiquement déclenché (comme si dans constructeur) :
  tabClients = toSignal( tabClientsObservable$ , { initialValue : [] } ) ;
...
}
```

NB: pas besoin de `changeDetectorRef.markForCheck()` avec des signaux

Observable combiné avec toSignal pour données récupérées sur demande:

```
...
export class XYZComponent {
  xyzService = inject(XyzService);
  sXxx = signal<Xxx[]> ([]);

  onRetreiveXxx(){
    //pas besoin de this.changeDetector.markForCheck(); avec sXxx sous forme de signal
    this.xyzService.getXYZ$(...).subscribe({
      next: (xxx : Xxx[])=>{this.sXxx.set(xxx) ; },
      error: (err )=>{console.log("erreur:" + JSON.stringify(err)) ; }
    });
  }
}
```

<button (click)="onRetreiveXxx()">retreive xxx</button>

xxx : { {sXxx() | json} }

IV - ZoneLess et composants avancés

1. Ancien ZoneJs pour angular 2 à 20

1.1. Ancien rôle fondamental de ZoneJs

Un composant angular a techniquement 2 parties complémentaires :

- une partie "DOM" (conforme à l'api des navigateurs)
- une partie "données et mécanismes angular"

Les mécanismes internes du framework "Angular" utilisent la bibliothèque "**Zone.js**" pour gérer contextuellement certaines modifications de données au niveau de l'application prise en charge par le navigateur .

Une "Zone" est un élément abstrait qui peut être vu comme proche de "ThreadLocal" et qui correspond à "un contexte d'exécution interchangeable et durable " ou "zone mémoire bien conservée" permettant de faire un lien entre plusieurs micro-exécutions de tâches potentiellement asynchrones .

Les évènements extérieurs à Angular qui peuvent subvenir et qui sont automatiquement gérés par Zone.js et la zone angular sont:

- Les évènements Javascript comme les *clicks*, *keydown*, *submit*, *input* etc...
- Les évènements XHR provenant de l'objet [XMLHttpRequest](#),
- Les évènements provenant de *Timers* comme `setTimeout()` ou `setInterval()`.

Cette gestion automatique des événements par Zone.js et par le framework angular permet de :

- déterminer (par algorithme sophistiqué) , les modifications qui doivent ou pas se traduire par des ré-affichages actualisés
- détecter et répercuter les changements de certaines données
- assurer la cohérence de l'ensemble de l'interface graphique de l'application
- bien gérer le binding (data --- vue) .

1.2. NgZone et le contournement contrôlé des détections de changements (angular 2 à 20)

En injectant **NgZone** dans un constructeur de composant angular , on peut contrôler par programmation si une modification de données sera effectuée ou pas au dehors de la zone "angular" .

Sur **this.zone** (de type **NgZone**) on peut appeler :

- **runOutsideAngular()** pour exécuter du code en dehors de la détection de changements Angular.
- **run()** pour exécuter du code dans une zone Angular. La détection de changements sera exécutée.

La variante **runTask()** permet d'exécuter du code dans une zone Angular de façon

asynchrone. La détection de changements sera exécutée.

1.2.a. Exemple avec runOutsideAngular:

```
<p>inside-outside-zone works!</p>
cpt=<span class="enEvidence">{{cpt}}</span>
<input type="button" value="reset to 0" (click)="onResetCpt()" /><br/>
<input type="button" value="++to10 inside" (click)="onIncrCpt()" /> <br/>
<input type="button" value="++to10 outside" (click)="onIncrCptOutside()" /> <br/>
```

```
import { Component, OnInit, NgZone } from '@angular/core';

@Component({
  selector: 'app-inside-outside-zone',
  templateUrl: './inside-outside-zone.component.html',
  styleUrls: ['./inside-outside-zone.component.scss']
})
export class InsideOutsideZoneComponent implements OnInit {

  public cpt : number = 0;

  constructor(private zone: NgZone) {}

  public recursIncrCpt(doneCallback: () => void){
    this.cpt++;
    console.log(`Current cpt: ${this.cpt}`);
    if (this.cpt < 10) {
      window.setTimeout(() => {
        this.recursIncrCpt(doneCallback);
      }, 100);
    } else {
      doneCallback();
    }
  }

  public onIncrCpt(){ this.recursIncrCpt(() => console.log('inside Done!')); }
  public onResetCpt(){ this.cpt = 0; }

  public onIncrCptOutside(){
    this.zone.runOutsideAngular(() => {
      //this.recursIncrCpt(() => console.log('Done!')); //no display final 10
      this.recursIncrCpt(
        () => this.zone.run(() => {
          console.log('outside Done!'); //indirectly update UI (to final 10)
        })
      );
    });
  }
}
```

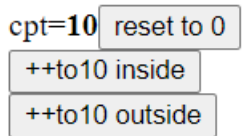
```

} //end of onIncrCptOutside

ngOnInit(): void { }
}

```

Comportement / résultats :



The screenshot shows a user interface with a text display 'cpt=10'. Below it are three buttons: 'reset to 0', '++to10 inside', and '++to10 outside'. The '++to10 inside' button is highlighted with a mouse cursor.

En cliquant sur le premier bouton (déclenchant `onIncrCpt()`), l'appel normal/ordinaire de la méthode récursive **`recursIncrCpt()`** s'effectue par défaut dans la zone angular et chaque mise à jour de `cpt` (de 1 à 10 toutes les 100 ms) est automatiquement détectée par angular et conduit à une réactualisation de l'affichage : `cpt=1` puis `cpt=2` puis `cpt=3` puis `cpt=10`.

En cliquant sur le deuxième bouton (déclenchant `onIncrCptOutside()`), l'appel de la méthode récursive **`recursIncrCpt()`** s'effectue au dehors de la zone d'angular et chaque mise à jour de `cpt` (de 1 à 10 toutes les 100 ms) n'est pas automatiquement détectée par angular et ne conduit pas à une réactualisation de l'affichage.

Il s'affiche toujours `cpt=0` avec la callback `() => console.log('Done!')`.

```

public onIncrCptOutside(){
  this.zone.runOutsideAngular(() => {
    this.recursIncrCpt(() => console.log('Done!')); //no display final 10 , cpt=0
  });
}

```

Cependant en ajustant la callback avec un appel explicite à `() => this.zone.run(...)`

```

public onIncrCptOutside(){
  this.zone.runOutsideAngular(() => {
    this.recursIncrCpt(
      () => this.zone.run(() => {
        console.log('outside Done!'); //indirectly update UI (to final 10)
      })
    );
  });
}

```

on termine via un retour en zone angular et cela provoque l'affichage de `cpt=10`

On passe ainsi directement de l'affichage `cpt=0` à `cpt=10` (les incrémentation intermédiaires 1, 2, 3 ... 9 ne sont affichées qu'à la console selon un appel explicite).

Cet exemple "cas d'école" montre le principe de fonctionnement.

Au sein d'un vrai projet angular, on peut par exemple effectuer tout un tas de prétraitement hors zone angular pour éviter des affichages inutiles et gagner en performance.

Seul le dernier traitement pourra être effectué au sein de la zone d'angular et conduire à un affichage intéressant.

2. ChangeDetectorRef

En injectant l'élément technique **ChangeDetectorRef** au sein d'un composant , on peut contrôler finement l'activation ou la désactivation de la détection automatique des changements :

- Désactiver l'exécution automatique de la détection : **ChangeDetectorRef.detach()**,
- Ré-activer la détection automatique : **ChangeDetectorRef.reattach()**,
- Forcer l'exécution de la détection : **ChangeDetectorRef.detectChanges()**,
- Vérifier si des changements sont détectés : **ChangeDetectorRef.checkNoChanges()**
- Marquer la vue du composant pour que la détection de changements soit effectuée : **ChangeDetectorRef.markForCheck()**

Exemple :

```
<p>custom-detection works with counter={{counter}}</p>
<p><input type='button' (click)='disableChangeDetection()' value='Disable change detection' /></p>
<p><input type='button' (click)='enableChangeDetection()' value='Enable change detection' /></p>
```

```
import { Component, OnInit, ChangeDetectorRef } from '@angular/core';
@Component({
  selector: 'app-custom-detection',
  templateUrl: './custom-detection.component.html',
  styleUrls: ['./custom-detection.component.scss']
})
export class CustomDetectionComponent implements OnInit {
  public counter : number = 0;
  constructor(private changeDetectorRef: ChangeDetectorRef) { }

  ngOnInit(): void { setInterval(() => { this.counter++; }, 1000); }
  disableChangeDetection(): void { this.changeDetectorRef.detach(); }
  enableChangeDetection(): void { this.changeDetectorRef.reattach(); }
}
```

Comportement :

En cliquant sur le premier bouton , conduisant à l'instruction `this.changeDetectorRef.detach()`; les changements ne sont plus détectés et l'affichage reste bloqué à une certaine valeur (ex ; 2)

custom-detection works with counter=2

Disable change detection

Enable change detection

En cliquant sur le deuxième bouton , conduisant à l'instruction `this.changeDetectorRef.reattach()`; les changements sont de nouveau détectés et l'affichage est régulièrement réactualisé (ex ; 47 puis 48 puis 49)

custom-detection works with counter=49

Disable change detection

Enable change detection

3. Stratégies de détection des changements

```
@Component({
  ... ,
  changeDetection: ChangeDetectionStrategy.OnPush
})
...
```

<i>(checkAlways)</i> Default (angular 21 et antérieur) Eager (depuis angular 22)	Suite à un quelconque changement , toute l'arborescence du composant (et de ses sous composants) est recalculée/mise à jour pour être sûr de ne rien rater .
OnPush	L'arborescence du composant et de ses sous composants n'est recalculée/mise à jour que dans un de ces cas : - un événement utilisateur (ex : click) est détecté - une valeur en <code>input()</code> ou <code>@Input()</code> a changé - utilisation du pipe <code>async</code> dans un template - un appel explicite à <code>.markForCheck()</code> est effectué

Evolution importante :

- La stratégie "**OnPush**" est devenue la stratégie par défaut en juin 2026 (depuis la version 22 d'angular).
- Pour éviter toute confusion, l'ancien **Default** a été renommé **Eager** depuis angular 22.

4. Mode ZoneLess (par défaut depuis angular 21)

Adaptation nécessaire en mode "ZoneLess" (par défaut à partir de angular 21) :

En mode moderne "**ZoneLess**" (sans utilisation de l'ancien ZoneJs) , **suite à un traitement**

asynchrone (ex : résultat d'un appel http simulé ou réel) les valeurs réactualisées en mémoire et placées dans de simples variables d'instances ne sont **pas** automatiquement **défectées** comme ayant **changées** et devant être rafraîchies du côté affichage html .

Sans signal , il faut **manuellement déclencher** une demande de rafraîchissement via

this.changeDetectorRef.markForCheck();

en s'appuyant sur un accès au service injecté ChangeDetectorRef :

private changeDetectorRef = inject(ChangeDetectorRef);

NB: `changeDetectorRef.markForCheck();` peut souvent être utilement déclenché en fin de callback enregistrée via `resObservable.subscribe({ next : ...})` .

⇒ Variante envisageable :

```
tabClients = signal<Client[]>([]);
```

et

```
.subscribe({ next: (tabCli)=>{ this.tabClients.set(tabCli); } , ...
```

NB: pas besoin de `changeDetectorRef.markForCheck()` avec des signaux

Eventuelle activation du mode "ZoneLess" au sein d'angular 20 :

```
export const appConfig: ApplicationConfig = {
  providers: [ ... ,
    provideZonelessChangeDetection() //maintenant d'office (par défaut depuis angular 21) !
  ]
};
```

plus suppression de zoneJs (zone.js) dans npm/package.json et dans partie polyfills de angular.json

5. OnPush vs ZoneLess

En pratique, ça va souvent ensemble dans les très récentes versions d'angular.

D'un point de vue formel /précis/ détaillé:

OnPush indique **quel composant doit être vérifié**. Il restreint la vérification aux composants dont les `@Input` changent, les événements DOM proviennent du template, un Signal consommé est mis à jour, ou dont le statut est marqué comme « dirty » via `markForCheck()` / `AsyncPipe`.

Zoneless indique **quand la détection des changements est planifiée et déclenchée**, en supprimant la dépendance à Zone.js. L'application ne réagit plus implicitement à chaque API asynchrone (comme `setTimeout`), mais aux mises à jour de Signals, aux flux de l'`AsyncPipe` ou aux événements utilisateur.

Autre différence :

Zoneless intervient globalement au niveau d'une application angular entière.

La stratégie **onPush** est paramétrable au niveau `@Component`

6. Éléments avancés sur composants angular

6.1. Cycle de vie précis d'un composant angular :

<i>Interfaces (à facultativement implémenter)</i>	<i>Méthodes (une par interface)</i>	<i>Moment où la méthode est appelée automatiquement par angular2</i>
(1) OnChanges	ngOnChanges()	dès changement de valeur d'un "input binding" (exemple : "propriété initialisée selon niveau parent")
(2) OnInit	ngOnInit()	à l'initialisation du composant et après les premiers éventuels ngOnChanges() et après constructeur et injections
(3) DoCheck	ngDoCheck()	permet éventuellement d'indiquer des changements si Angular ne les a pas détecté (avant un futur réaffichage)
(4) AfterContentInit	ngAfterContentInit()	est déclenchée à l'initialisation après la projection de contenu (*) (après ngOnInit() et avant ngAfterViewInit())
(5) AfterContentChecked	ngAfterContentChecked()	déclenchée après la détection de changement dans le contenu projeté
(6) AfterViewInit	ngAfterViewInit()	déclenchée après l'initialisation de la vue du composant et après l'initialisation des vues des composants enfants.
(7) AfterViewChecked	ngAfterViewChecked()	est déclenchée après détection d'un changement dans la vue du composant et dans les vues des composants enfants.
(8) OnDestroy	ngOnDestroy()	juste avant destruction d'un composant

Ces méthodes sont appelées dans l'ordre 1,2,...7 lors de l'initialisation et du premier affichage d'un composant . Par la suite seules certaines méthodes seront redéclenchées après une détection de valeurs modifiées (ngOnChanges() , ngDoCheck() , ngAfterContentChecked() , ngAfterViewChecked())

(*) NB : on appelle "**projection de contenu**" la prise en compte (au niveau d'un sous composant) d'un contenu (choisi par le parent) imbriqué entre début et fin de balise du sous composant

exemple :

```
<mat-tab-group>
  <mat-tab label="calcul tva">
    <app-tva></app-tva> <!-- contenu projeté dans composant mat-tab -->
  </mat-tab>
  ...
```

NB : La potentielle erreur "**ExpressionChangedAfterItHasBeenCheckedError**" est déclenchée en développement seulement pour indiquer qu'une erreur de conception risque de compromettre la détection de changement et empêcher la mise à jour correcte de la vue. (ex : `this.updatedValue = 'new_value'`; n'est pas à faire dans `ngAfterContentChecked()`)

Pour éviter l'erreur précédente , il suffit souvent d'englober les instructions à problème au sein d'un bloc `setTimeout(=>{...},1)` ;

Pour approfondir le sujet (mécanismes internes de angular) :

<https://cdiese.fr/angular-change-detection/>

ngDoCheck() n'est à coder que dans des cas ultra-pointus (en s'appuyant sur `KeyValueDiffers` ou `IterableDiffers`)

Lien pour approfondir le sujet (si nécessaire) :

<https://www.concretepage.com/angular/angular-ngdocheck>

6.2. Détails sur `ngOnChanges()`:

Cette *callback* est exécutée si le composant contient des propriétés en entrée (notamment avec le décorateur `@Input()`).

Cette *callback* peut être implémentée avec un argument de type `SimpleChanges`:

```
void ngOnChanges(changes: SimpleChanges): void {}
```

Elle sera alors déclenchée une seule fois (globalement pour toutes les propriétés modifiées)

Exemple :

```
ngOnChanges(changes: SimpleChanges){
  console.log( "changes=" + JSON.stringify(changes));
}
```

--> premier appel (avec valeurs initiales choisies par le parent) :

```
changes={"compteur":{"currentValue":1,"firstChange":true},
  "message":{"currentValue":"msg","firstChange":true},
  "donnees":{"currentValue":{"num":1,"label":"label"},"firstChange":true}}
```

--> appels ultérieurs (suite à changements opérés par le parent) :

```
changes={"compteur":{"previousValue":1,"currentValue":2,"firstChange":false}}
```

puis

```
changes={"message":{"previousValue":"msg","currentValue":"msg*","firstChange":false}}
```

@Input et ngOnChanges() selon modifications effectuées

template d'un composant parent :

```
counter : {{counter}} <button (click)="counter=counter+1">++</button> ,
msg : {{msg}} <button (click)="msg=msg+'*'">++</button> ,
data : ({{data.num}} <button (click)="data.num=data.num+1">++</button>,
      {{data.label}} <button (click)="data.label=data.label+'*'">++</button>
    )
<my-child [compteur]="counter" [message]="msg" [donnees]="data"> </my-child>
```

Affichage initial :

```
values in this (parent) component: counter : 1 ++ , msg : msg ++ , data : (1 ++ , label ++ )
with 3 @input/property links / [compteur]="counter" [message]="msg" [donnees]="data" :
values in this (child) component: compteur : 1 ++ , message : msg ++ , donnees : (1 ++ , label ++ )
```

Après modifications effectuées sur le parent (et automatiquement répercutées sur "child" avec appels de ngOnChanges() sur "child") :

```
values in this (parent) component: counter : 2 ++ , msg : msg* ++ , data : (1 ++ , label ++ )
with 3 @input/property links / [compteur]="counter" [message]="msg" [donnees]="data" :
values in this (child) component: compteur : 2 ++ , message : msg* ++ , donnees : (1 ++ , label ++ )
```

Après modifications effectuées uniquement sur le sous composant "child" .

pas de remontées de modifications vers le parent tant que pas de @Output / event

```
values in this (parent) component: counter : 2 ++ , msg : msg* ++ , data : (1 ++ , label ++ )
with 3 @input/property links / [compteur]="counter" [message]="msg" [donnees]="data" :
values in this (child) component: compteur : 4 ++ , message : msg*** ++ , donnees : (1 ++ , label ++ )
```

Dans cet exemple l'objet data (comportant lui même des sous parties modifiables) est initialement passé par référence (via **[donnees]="data"**) .

Cet objet de données (se retrouvant partagé entre les composants parent et enfant) conduit à un affichage automatiquement synchronisé dans les 2 sens (sans aucun appel à ngOnChanges() tant que l'on modifie les sous parties de cet objet sans changer l'objet data lui même) .

```
values in this (parent) component: counter : 2 ++ , msg : msg* ++ , data : (5 ++ , label*** ++ )
with 3 @input/property links / [compteur]="counter" [message]="msg" [donnees]="data" :
values in this (child) component: compteur : 4 ++ , message : msg*** ++ , donnees : (5 ++ , label*** ++ )
```

7. Voir si nécessaire l'annexe de rappels sur les composants (input, output, model , projection)

8. ng-template et TemplateRef

8.1. ng-template

Au sein d'un template HTML angular, un bloc `<ng-template #nomTemplate>...</ng-template>` est un bloc qui n'est pas affiché par défaut et qui le sera potentiellement par la suite.

```
...
export class WithNgTemplateComponent {
  ready=false;
  ...
}
```

```
<input type="checkbox" [(ngModel)]="ready" />ready <br/>
```

```
<!-- *ngIf="booleanExpression else alternativeTemplate" -->
```

```
<div *ngIf="ready else notReady">
```

```
  ready / done !
```

```
</div>
```

```
<ng-template #notReady>
```

```
  waiting (not ready)
```

```
</ng-template>
```

☒ ready

ready / done !

☐ ready

waiting (not ready)

Affichage via ng-container et *ngTemplateOutlet

```
<!-- ng-container peut être vu comme une div virtuelle avec possibilité de *ngIf="..."
      ou autre sans niveau d'imbrication supplémentaire -->
```

```
<ng-template #loading>
```

```
  <p>loading...</p>
```

```
</ng-template>
```

```
<ng-template #details>
```

```
  <p>details...</p>
```

```
</ng-template>
```

```
<ng-container *ngTemplateOutlet="#loading"></ng-container>
```

```
<!--affiche à cet endroit le template #loading-->
```

ng-template avec contexte :

```
....
export class WithNgTemplateComponent {
...
  ctxSemaine = {code: 1 , label : "semaine"};
  ctxWeekEnd = {code: 2 , label : "weekend"};
}
```

```
<h4>templateAvecContexte</h4>
<ng-template #templateAvecContexte let-code="code" let-label="label">
  <!-- via let-code="code" au sein de ce template la variable code
       aura la valeur de la sous partie ".code" d'un objet javascript
       interchangeable appelé context -->
  <div> code={{code}} label={{label}}</div>
</ng-template>

<ng-container *ngTemplateOutlet="templateAvecContexte;context:ctxSemaine">
</ng-container>

<ng-container *ngTemplateOutlet="templateAvecContexte;context:ctxWeekEnd">
</ng-container>
```

templateAvecContexte

code=1 label=semaine

code=2 label=weekend

ng-template avec TemplateRef :

```
import { Component, Input, TemplateRef } from '@angular/core';
...
export class PanelWithTemplateRefComponent {
  @Input()
  headerTemplate: TemplateRef<any> | undefined;

  @Input()
  footerTemplate: TemplateRef<any> | undefined;
}
```

panel-with-template-ref.component.html

```
<ng-template #defaultHeader>
  <div>defaultHeader</div>
</ng-template>

<ng-template #defaultFooter>
  <div>defaultFooter</div>
</ng-template>

<div class="withBorder">
  <ng-container *ngTemplateOutlet="headerTemplate?headerTemplate:defaultHeader"></ng-container>
  <ng-content></ng-content>
  <ng-container *ngTemplateOutlet="footerTemplate?footerTemplate:defaultFooter"></ng-container>
</div>
```

```
<h4>sous composant avec TemplateRef</h4>
<app-panel-with-template-ref>
  panel with defaultHeader and defaultFooter
</app-panel-with-template-ref>
<br/>
<app-panel-with-template-ref [headerTemplate]="details" [footerTemplate]="loading">
  panel with details template as Header and loading template as Footer
</app-panel-with-template-ref>
```

sous composant avec TemplateRef

```
defaultHeader
panel with defaultHeader and defaultFooter
defaultFooter
```

```
details...
panel with details template as Header and loading template as Footer
loading...
```

9. ViewChild / ContentChild

Vocabulaire (issu de la norme "web component") :

Le **Shadow DOM** - est un DOM interne de votre composant défini par vous (en tant que créateur du composant) et caché de l'utilisateur final. Par exemple:

```
@Component ({
  selector: 'some-component',
  template: `
    <h1>I am Shadow DOM!</h1>
    <h2>Nice to meet you :)</h2>
    <ng-content></ng-content>
  `
})
class SomeComponent { /* ... */ }
```

Le **Light DOM** - est un DOM qui est un utilisateur final de votre composant. Par exemple:

```
@Component ({
  selector: 'another-component',
  directives: [SomeComponent],
  template: `
    <some-component>
      <h1>Hi! I am Light DOM!</h1>
      <h2>So happy to see you!</h2>
    </some-component>
  `
})
class AnotherComponent { /* ... */ }
```

NB :

@ViewChild et **@ViewChildren** recherchent des éléments dans **Shadow DOM** (autrement dit dans la structure du template du composant actuel)

tandis que

@ContentChild et **@ContentChildren** les recherchent dans **Light DOM** (autrement dit dans le contenu qui sera projeté via le composant parent)

9.1. Exemple avec @ViewChild et @ViewChildren

a.child.component.ts

```
... @Component({
  selector: 'app-a-child', ...
})
export class AChildComponent {
  @Input()
  name : string ="aChild";

  public sayHello() :string{
    return "hello_from_" +this.name;
  }
}
... }
```

a.child.component.html

```
<p>a-child with name={{name}}</p>
```

with.child.component.html

```
<h3>with-child works!</h3>
message (from child 1 and 2)={{message}} <br/>
message (from all children)={{messageV2}}
<hr/>
<app-a-child name="child1"></app-a-child>
<app-a-child #c2 name="child2"></app-a-child>
<app-a-child name="child3"></app-a-child>
<p #p #p1><b>p1</b></p>
<p #p><i>p2</i></p>
<p #p>p3</p>
```

with-child works!

```
message (from child 1 and 2)=hello_from_child1 hello_from_child2
message (from all children)= hello_from_child1 hello_from_child2 hello_from_child3 p1 p2 p3
```

```
a-child with name=child1
```

```
a-child with name=child2
```

```
a-child with name=child3
```

```
p1
```

```
p2
```

```
p3
```

with.child.component.ts

```
import { Component, OnInit, ViewChild, ViewChildren,
        QueryList, ElementRef } from '@angular/core';
import { AChildComponent } from './a-child/a-child.component';

@Component({
  selector: 'app-with-child',
  templateUrl: './with-child.component.html',
  styleUrls: ['./with-child.component.scss']
})
export class WithChildComponent {

  message:string="";
  messageV2:string="";

  @ViewChild(AChildComponent)
  first_achild! : AChildComponent; // refer to first subcomponent of type AChildComponent

  @ViewChild('#c2', { static: true })
  achild_c2! : AChildComponent; // refer to subcomponent/subElement with #c2 in .html
```

```

@ViewChildren(AChildComponent)
allchilds!: QueryList<AChildComponent>; // refer to subcomponents of type AChildComponent

@ViewChildren('p', { read: ElementRef }) pList!: QueryList<ElementRef>; // all elt with #p
@ViewChild('p1', { read: ElementRef }) p1!: ElementRef; // elt with #p1

ngAfterViewInit() {
  // call method .sayHello() of child sub component :
  let msgChild1= this.first_achild?this.first_achild.sayHello():"";

  let msgChild2= this.achild_c2?this.achild_c2.sayHello():"";
  let allMsg = msgChild1 + " " + msgChild2;

  let allMsgV2 = "";
  for(let c of this.allchilds){
    allMsgV2 = allMsgV2+" " + c.sayHello();
  }

  for(let p of this.pList){
    allMsgV2 = allMsgV2+" " + p.nativeElement.innerText;
  }

  //this.message= allMsg;!-->ExpressionChangedAfterItHasBeenCheckedError"
  setTimeout(=> {
    this.message=allMsg;
    this.messageV2=allMsgV2;
  }, 0);
  console.log(allMsg);
  console.log("in p1:" + this.p1.nativeElement.innerHTML);
}
...
}

```

9.2. Exemple avec @ContentChild et @ContentChildren

with-content.component.html

```

<p>with-content works!</p>
<app-my-tabset>
  <app-my-tab title="tab1">
    <div #tabContent >div_a</div>
  </app-my-tab>
  <app-my-tab title="tab2">
    <div #tabContent >div_b</div>
  </app-my-tab>
  <app-my-tab title="tab3">
    <div #tabContent >div_c</div>
  </app-my-tab>
</app-my-tabset>

```

with-content works!

TAB1

TAB2

TAB3

div_c

with-content works!

TAB1

TAB2

TAB3

div_a

my-tab.component.html

```
<span class="aTab" (click)="onClick()" [class.selectedTab]="selected" >{{title}}</span>
```

my-tab.component.css

```
.aTab { display: inline-block; margin : 2px; padding: 2px;
        border-style: solid; border-color: blue; border-width: 1px;}
.selectedTab { font-weight: bold; background-color: lightgrey;}
```

my-tab.component.ts

```
import { Component, OnInit, Input, AfterContentInit, ContentChild,
        ElementRef, EventEmitter, Output } from '@angular/core';
@Component({
  selector: 'app-my-tab',
  templateUrl: './my-tab.component.html',
  styleUrls: ['./my-tab.component.scss']
})
export class MyTabComponent implements AfterContentInit {

  @Input() title:string="defaultTabTitle";

  @Input() selected:boolean=false;

  @Output()
  public selectionChange : EventEmitter<{value:MyTabComponent}> =
    new EventEmitter<{value:MyTabComponent}>();

  @ContentChild("tabContent")
  contentElementRef! : ElementRef; // refer to elt with #tab_content in projected content

  onClick(){
    this.selected=!this.selected;
    if(this.selected)
      this.selectionChange.emit({value:this});
  }

  constructor() { }
  ngAfterContentInit(): void {
    console.log(this.contentElementRef.nativeElement.innerText)
  }

  public tabContentAsString(){
    if(this.contentElementRef===undefined) return "";
    else return this.contentElementRef.nativeElement.innerText
  }
}
```

my-tabset.component.html

```
<ng-content></ng-content> <!-- will display tabs titles-->
<br/>
{{selectedTabContent}}
```

my-tabset.component.ts

```
import { Component, OnInit, ContentChildren, QueryList, AfterContentInit } from '@angular/core';
import { MyTabComponent } from '../my-tab/my-tab.component';

@Component({
  selector: 'app-my-tabset',
  templateUrl: './my-tabset.component.html',
  styleUrls: ['./my-tabset.component.scss']
})
export class MyTabsetComponent implements OnInit, AfterContentInit {

  selectedTabContent : string = "";

  @ContentChildren(MyTabComponent)
  tabs: QueryList<MyTabComponent>

  onSelectionChange(newSelectedTab : MyTabComponent ){
    this.tabs.forEach(tab => { tab.selected=(tab===newSelectedTab);
      if(tab === newSelectedTab )
        this.selectedTabContent = newSelectedTab.tabContentAsString();
    })
  }

  ngAfterContentInit() {
    let lastTab = this.tabs.last;
    this.tabs.forEach(tab => { console.log(tab.title);
      tab.selectionChange.subscribe(
        (evt)=>{ this.onSelectionChange(evt.value); }
      );
      tab.title=tab.title.toUpperCase();
      tab.selected=(tab===lastTab); //last tab selected by default
    })
    this.selectedTabContent = lastTab.tabContentAsString();
  }
  ...
}
```

Cet exemple "rudimentaire" de composant "onglet" est bien évidemment améliorable . Il permet cependant de montrer ce qu'il est possible de faire avec **@ContentChildren** et **@ContentChild** . En annexe (*ngx-bootstrap , angular-material*) , on trouvera des composants onglets prédéfinis.

Version améliorée des onglets personnalisés avec TemplateRef :*my-tab-component.html*

```
<span class="aTab" (click)="onClick()" [class.selectedTab]="selected">{{title}}</span>
<ng-template>
  <ng-content></ng-content>
</ng-template>
```

```
import { Component, OnInit, Input, AfterContentInit, ContentChild, ElementRef, EventEmitter, Output, TemplateRef,
ContentChildren, ViewChild } from '@angular/core';
```

```
...
```

```
export class MyTabComponent implements OnInit {
  @Input() title:string="defaultTabTitle";
  @Input() selected:boolean=false;

  @Output()
  public selectionChange : EventEmitter<{value:MyTabComponent}> =
    new EventEmitter<{value:MyTabComponent}>();
```

```
@ViewChild(TemplateRef)
```

```
tabTemplate: TemplateRef<any> | undefined ;
```

```
onClick() {
  this.selected=!this.selected;
  if(this.selected)
    this.selectionChange.emit({value:this});
}
ngOnInit(): void { }
constructor() { }

public tabContentAsTemplateRef() : TemplateRef<any> | undefined{
  return this.tabTemplate;
}
}
```

my-tab-set.component.html

```
<ng-template #defaultTabContent>
  <div>defaultTabContent</div>
</ng-template>

<ng-content></ng-content> <!-- will display tabs titles -->
<br/>
<ng-container
  *ngTemplateOutlet="selectedTabTemplateRef?selectedTabTemplateRef:defaultTabContent">
</ng-container>
```

my-tabset.component.ts

```
import { Component, OnInit, ContentChildren, QueryList, AfterContentInit, TemplateRef, AfterViewInit }
from '@angular/core';
import { MyTabComponent } from '../my-tab/my-tab.component';

...
export class MyTabsetComponent implements OnInit , AfterViewInit {

  selectedTabTemplateRef : TemplateRef<any> | undefined;
```

```
@ContentChildren(MyTabComponent)
```

```
tabs!: QueryList<MyTabComponent>
```

```
onSelectionChange(newSelectedTab : MyTabComponent ){
  this.tabs.forEach(tab => {
    tab.selected=(tab===newSelectedTab);
    if(tab === newSelectedTab )
      this.selectedTabTemplateRef = newSelectedTab.tabContentAsTemplateRef();
  })
}
```

```
ngAfterViewInit(){
```

```
  let lastTab = this.tabs.last;
  this.tabs.forEach(tab => { console.log(tab.title);
    tab.selectionChange.subscribe(
      (evt)=>{ this.onSelectionChange(evt.value); }
    );
    setTimeout(()=>{
      tab.selected=(tab===lastTab);
      //setTimeout to avoid error message "Expression has changed after it was checked"
    },1)
  })
  this.selectedTabTemplateRef = lastTab.tabContentAsTemplateRef();
}
```

```
constructor() { }
ngOnInit(): void { }
}
```

```
<app-my-tabset>
  <app-my-tab title="tab1" >
    <div>div_a</div>
  </app-my-tab>
  <app-my-tab title="tab2" >
    <div>div_b</div>
  </app-my-tab>
  <app-my-tab title="tab3">
    <div>div_c</div>
  </app-my-tab>
  <app-my-tab title="tab4" >
    <app-welcome></app-welcome>
  </app-my-tab>
</app-my-tabset>
```

tab1 tab2 tab3 **tab4**

welcome to exp-adv-app

10. Animations (angular ≥ 20)

10.1. évolutions des animations angular

NB: les anciennes versions d'angular (4 à 19) utilisait l'ancienne api *legacy* `@angular/animations` (avec `trigger`) qui est maintenant considérée comme obsolète *deprecated*.

Il faut maintenant utiliser `animate.enter="cssClassWithAnimation"`
et `animate.leave="cssClassWithAnimation"` à la place

NB:

- `animate.enter` est déclenché quand l'élément est ajouté au DOM
- `animate.leave` est déclenché quand l'élément est retiré du DOM

10.2. animate.enter et animate.leave (depuis angular 20)

html	ts	css
<pre>@if (showMessage()) { <div class="bg-red-400" animate.enter="slide-in" animate.leave="fade-out"> Angular css animations (enter: slide-in , leave: fade-out) </div> } <button (click)="toggleShow()"> Toggle showMessage</button></pre>	<pre>... export class AnimationsComponent { showMessage = signal(false); toggleShow(): void { this.showMessage.update((v: boolean) => !v); } }</pre>	<pre>.slide-in { animation: slideIn 0.5s ease-out; } .fade-out { animation: fadeOut 0.5s ease-in; } @keyframes slideIn { from { transform: translateX(+200px); opacity: 0; } to { transform: translateX(0); opacity: 1; } } @keyframes fadeOut { from { opacity: 1; } to { opacity: 0; } }</pre>

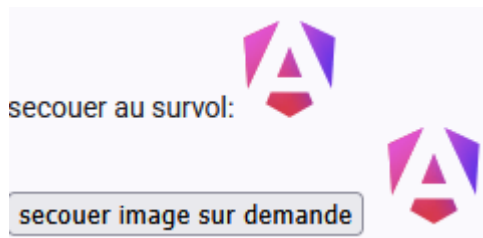
Angular css animations (enter: slide-in , leave: fade-out)

Toggle showMessage

avec en entrée/apparition un effet de glissement "slideIn" (droite vers gauche)
et en sortie/disparition un effet "fadeOut" progressif.

NB: la syntaxe des animations du côté .css (avec @keyframes) respecte à fond le standard css .

10.3. Déclenchement d'animation sur demande



.ts	.css
<pre> secouer_image_sur_demande=""; //état initial à "" (pas d'animation au premier affichage) showHideImage = signal(true); onSecouerSurDemande(){ if(this.secouer_image_sur_demande=="") this.secouer_image_sur_demande= "secouer_sur_demande" this.showHideImage.set(false); setTimeout(() => { this.showHideImage.set(true); }, 1); //a small delay (ex: 1 ms) is necessary } </pre>	<pre> @keyframes secouer-anim { 0% { transform: rotate(10deg); } 25% { transform: rotate(-10deg); } 50% { transform: rotate(10deg); } 75% { transform: rotate(-10deg); } 100% { transform: rotate(0deg); } } .secouer_au_survol:hover { animation: secouer-anim 2s; } .secouer_sur_demande{ animation: secouer-anim 1.5s; } </pre>
.html	
<pre> secouer au survol:
 <button (click)="onSecouerSurDemande()">secouer image sur demande</button> &nbsp; @if(showHideImage()){ } </pre>	

Au sein de cet exemple, l'animation au survol (*:hover*) est entièrement gérée par css (sans angular).

Par contre l'animation sur demande (ici déclenchée par un bouton) , peut s'effectuer en mettant le signal booléen **showHideImage** à **false** puis quasi aussitôt à **true** pour que le paramétrage **[animate.enter]** soit pris en compte .

La valeur de ce paramétrage est ici le contenu d'une variable **secouer_image_sur_demande** qui est initialisée à vide "" pour éviter un déclenchement automatique dès le début et qui est modifiée avec la valeur du nom de la classe d'animation **secouer_sur_demande** lorsque l'on click sur le bouton .

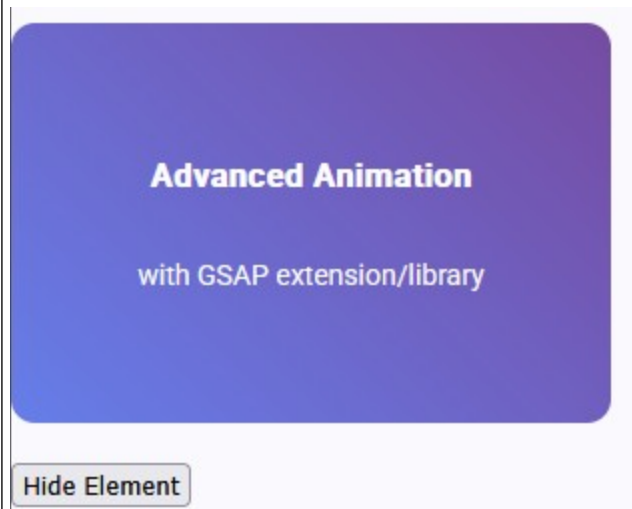
Ce genre d'animation peut attirer l'œil de l'utilisateur sur un détail important (par exemple sur un caddy qui change de contenu) .

10.4. Animation avancée avec extension gsap et fonctions événementielles

```
npm install --save gsap @types/gsap
```

<i>.html et .css</i>	<i>.ts</i>
<pre> <div class="animation-playground"> @if (showElement()) { <div class="animated-box" (animate.enter)="handleEnterAnimation(\$event)" (animate.leave)="handleLeaveAnimation(\$event)"> <h3>Advanced Animation</h3> <p>with GSAP extension/library</p> </div> } <button (click)="toggleElement()"> {{showElement() ? 'Hide' : 'Show'}} Element </button> </div> </pre>	<pre> import { gsap } from 'gsap'; showElement = signal(false); handleEnterAnimation(event: AnimationCallbackEvent): void { gsap.fromTo(event.target, { scale: 0, rotation: -180, opacity: 0 }, { </pre>

```
.animated-box {  
  width: 300px;  
  height: 200px;  
  background: linear-gradient(45deg,  
    #667eea 0%, #764ba2 100%);  
  border-radius: 12px;  
  display: flex;  
  flex-direction: column;  
  justify-content: center;  
  align-items: center;  
  color: white;  
  margin: 20px 0;  
}
```



V - LazyLoading

Principe du "lazy loading" : ne pas charger tout le code de l'application dès le démarrage , mais ne charger progressivement que le code utile à l'utilisateur (en fonction des navigations effectuées) .

Trois niveaux de Lazy loading :

- "lazy loading" historique en mode "modules" (progressivement obsolète)
- LazyLoading de niveau "composant et sous composant"
- LazyLoading fin (defer , sous partie de composant, ...)

1. Ancien Lazy loading (avec modules)

Au lieu de charger dès le démarrage (dans le navigateur) tous les composants de l'application (ce qui peut quelquefois être long/lent) , **on peut organiser la structure du code en différents modules qui ne seront chargés (en différé) que lors d'une navigation (ex : activation d'un lien hypertexte associé au routage).**

Une application angular bien structurée comportera généralement :

- **un module applicatif principal** (`src/app/app.module.ts`)
avec `RouterModule.forRoot(routes)` dans `src/app/app-routing.module.ts`
- **plusieurs sous modules applicatifs** (quelquefois dénommés "*features modules*" ou encore "*domain modules*")
(ex : `src/app/mylazyN/mylazyN.module.ts`)
avec `RouterModule.forChild(routes)` dans `src/app/mylazyN/mylazyN-routing.module.ts`
- **d'éventuels modules de composants réutilisables** (quelquefois dénommés "*widget modules*" ou encore "*shared modules*")

(ex : `src/my-util/...`)

1.1. Exemple de (sous-)module en mode lazy :

En mode "non automatisé" au sein des anciennes versions d'angular :

```
ng g module myLazy
cd src/app/my-lazy
ng g component xx
ng g component yy
```

my-lazy.module.ts

```
import { NgModule } from '@angular/core';
import { CommonModule } from '@angular/common';
...
import { Routes, RouterModule } from '@angular/router';
const childRoutes: Routes = [
  { path: 'xx', component: XxComponent },
  { path: 'yy', component: YyComponent },
  { path: '', redirectTo: 'xx', pathMatch: 'prefix' }
```

```

];
@NgModule({
  imports: [ CommonModule,
            RouterModule.forChild(childRoutes)
          ],
  declarations: [ XxComponent, YyComponent ]
})
export class MyLazyModule { }

```

Et par exemple pour naviguer relativement de xx vers yy (et vice versa) :

```
<a [routerLink]="'../yy'"> vers yy </a>
```

En mode "automatisé" au sein des versions récentes d'angular :

```
ng g module myLazy2 --route mylazy2 --module app.module
```

automatise toutes les opérations suivantes :

```

CREATE src/app/my-lazy2/my-lazy2-routing.module.ts (349 bytes)
CREATE src/app/my-lazy2/my-lazy2.module.ts (358 bytes)
CREATE src/app/my-lazy2/my-lazy2.component.html (23 bytes)
CREATE src/app/my-lazy2/my-lazy2.component.spec.ts (636 bytes)
CREATE src/app/my-lazy2/my-lazy2.component.ts (283 bytes)
CREATE src/app/my-lazy2/my-lazy2.component.scss (0 bytes)
UPDATE src/app/app-routing.module.ts (1017 bytes)

```

avec

```

...
@NgModule({
  declarations: [MyLazy2Component],
  imports: [ CommonModule, MyLazy2RoutingModule ]
})
export class MyLazy2Module { }

```

dans **my-lazy2.module.ts**

avec

```

...
const routes: Routes = [
  { path: "", component: MyLazy2Component }
];

@NgModule({
  imports: [RouterModule.forChild(routes)],
  exports: [RouterModule]

```

```

})
export class MyLazy2RoutingModule { }
dans my-lazy2-routing.module.ts

```

et avec l'ajout de

```

{ path: 'mylazy2',
  loadChildren: () => import('./my-lazy2/my-lazy2.module').then(m => m.MyLazy2Module) }

```

dans les routes de *src/app/app-routing.module.ts*

1.2. Référencement d'un sous module en mode lazy

Référencement du ou des sous module(s) "lazy" depuis le module principal de l'application :

```

import { NgModule } from '@angular/core';
import { Routes, RouterModule } from '@angular/router';
const routes: Routes = [
  { path: 'mylazy1',
    loadChildren: () => import('./my-lazy/my-lazy.module').then(m => m.MyLazyModule) ,
  { path: 'mylazy2',
    loadChildren: () => import('./my-lazy2/my-lazy2.module').then(m => m.MyLazy2Module)}
];
@NgModule({
  imports: [RouterModule.forRoot(routes)], exports: [RouterModule]
})
export class AppRoutingModule { }

```

Point clef:

pas de imports : [...] explicite/systématique de sous module mais chemin **loadChildren** exprimé de la façon suivante :

au sein d'anciennes versions de angular :

```

{ path: 'mylazy1', loadChildren: './my-lazy/my-lazy.module#MyLazyModule' } ,

```

avec un suffixe en "**#LazySubModuleName**".

au sein des versions récentes d'angular :

```

{ path: 'mylazy1',
  loadChildren: () => import('./my-lazy/my-lazy.module').then(m => m.MyLazyModule)}

```

Dans un des composants du module principal :

```

<a [routerLink]="['/mylazy1']"> lazy loading part 1 ...</a> <br/>

```

Pour visualiser l'effet lazy-loading on peut éventuellement surveiller le **chargement différé** (au moment de la navigation) de ***-chunk.js** dans la partie "network" de "Chrome Dev Tools / F12".

1.3. Eventuel "pré-chargements en tâche de fond" (background preloading)

```
import { PreloadAllModules } from '@angular/router';
...
RouterModule.forRoot(
  appRoutes,
  {
    preloadingStrategy: PreloadAllModules
  }
)
```

dans *app-routing.module.ts*

1.4. Organisation des services et modules

src/app/common/service/global.service.ts (partagé pour toute l'appli)
....

src/app/lazy1/common/service/for.lazy1.service.ts (partagé entre composants
de ce sous module 'lazy1')
....

src/app/lazy2/common/service/for.lazy2.service.ts (partagé entre composants
de ce sous module 'lazy2')
...

NB:

- si **GlobalService** est en { **providedIn** : 'root' } alors une seule instance partagée par toute l'application (singleton global)
- si **GlobalService** est en { **providedIn** : 'any' } alors une instance partagée par tous les composants d'un même (sous-)module . Et instances différentes pour module principal *app.module* et sous modules *lazy1.module* et autres

1.5. Structuration angular multi-modules

1.5.a. Module applicatif unique "app" pour petite application angular

Une application angular créée via l'assistant "*ng new my-app*" ne comporte qu'un seul module applicatif appelé "**app**". Cette structure simple suffit pour une application de petite taille.

On pourra prévoir un sous répertoire spécial (ex : "src/app/**common**") permettant de ranger tous les éléments partagés au niveau de l'ensemble des composants du module :

✓ common > data > directive > gard > interceptor > pipe > service > util > validator	<i>src/app/common/data</i> (ou <i>common/model</i>) permet de ranger des classes de données qui sont à la fois manipulées par des composants (ex : formulaires de saisies) et à la fois manipulés par des services dans le cadre des appels de WS-REST (DTO JSON) . <i>src/app/common/service</i> permet de ranger des services angular qui sont par nature partagés/partageables .
---	---

1.5.b. Modules multiples pour grande application angular

Dès que l'on a besoin de développer une application angular de moyenne ou grande taille, on a tout intérêt à décomposer l'application en modules complémentaires de façon à :

- s'y retrouver dans une structure bien carrée et modulaire.
- optimiser les performances au démarrage (chargement du code au fur et à mesure des navigations si "lazy loading") .

Structure classique (avec variantes) :

src/app/

core (ou "*root*" : module principal/noyau/racine avec singletons , home/welcome component, ...)

shared (module partagé de choses qui seront réutilisées par d'autres modules)

xxx (features module, ex: *bases*)

yyy (features module, ex: *devises*)

Chacun de ces modules pourra avoir la sous structure suivante :

...

components

gards

pipes

models (or data)

directives

services

```
... (utils, configs, ...)
```

`src/app/app.module` peut devenir `src/app/core/core.module` with `CoreModule` in `src/main.ts`

`src/app/app.routing.module` peut devenir **`src/app/core/core.routing.module`**

`src/app/app.component` peut devenir `src/app/core/components/root/root.component.ts` with `core-root` in `index.html`

```

  ▾ src
    ▾ app
      > bases
    ▾ core
      ▾ components
        > footer
        > header
        > login
        > not-authorized
        > root
        > welcome

```

Convention de noms pour les sélecteurs : **moduleName-ComponentName** (ex : *core-root* , *core-welcome* , *core-header* , ...)

dans *src/app/shared/shared.module.ts*

```

...
@NgModule({
  imports: [ CommonModule , FormsModule , HttpClientModule , ... ],
  exports: [ TogglePanelComponent , ... ],
  declarations: [ TogglePanelComponent , ... ],
...

```

dans *xyz/xyz-routing.modules.tx*

```

...
{ path: '', redirectTo: 'xyz', pathMatch: 'prefix' }
...

```

dans *core/core-routing.modules.ts*

```

...
{ path: 'ngr-welcome', component: WelcomeComponent },
{ path: '', redirectTo: '/ngr-welcome', pathMatch: 'full' },
{ path: 'ngr-login', component: LoginComponent },
{ path: 'ngr-xxx', loadChildren: () => import('../xxx/xxx.module').then(m => m.XxxModule)},
{ path: 'ngr-yyy', loadChildren: () => import('../yyy/yyy.module').then(m => m.YyyModule)},
...

```

NB : il suffit de rectifier les chemins relatifs au niveau des *import { ... } from '...'* ; pour restituer une cohérence dans le cas où certains répertoires sont renommés ou déplacés lors d'un refactoring.

2. Lazy loading de composant

Au lieu de charger dès le démarrage (dans le navigateur) tous les composants de l'application (ce qui peut quelquefois être long/lent) , **on peut organiser la structure du code en différentes parties qui ne seront chargés (en différé) que lors d'une navigation** (ex : activation d'un lien hypertexte associé au routage).

2.1. Lazy loading de petit composant

Dans `app.routes.ts` :

```
...
{ path: 'ngr-small1', loadComponent: () => import('./small1/small1.component')
                                .then(m => m.Small1Component) },
...
```

2.2. Lazy loading de gros composant (avec sous routes)

Principe clef : Si à la place d'un module (ancien @NgModule) on a un gros composant angular (avec des sous composants interchangeable via des sous routes en mode "children") on obtient alors un comportement à peu près identique à l'ancien lazy loading de niveau module mais avec un code moderne en mode standalone .

Exemple :

```
@Component({
  selector: 'app-mega1',
  imports: [RouterLink,RouterOutlet,FormsModule],
  ...
})
export class Mega1Component {...}
```

```
<div><nav>
  <a routerLink="sc1">sous composant 1</a> &nbsp;
  <a routerLink="sc2">sous composant2</a> ...
</nav>  <hr/>
  <router-outlet></router-outlet> <!-- sous composant 1 ou bien sous composant 2 ou ... -->
</div>
```

mega1.routes.ts

```

...
export const mega1Routes: Route[] = [

  { path: "",
    /* providers: [
      provideXyzDomain(config)
    ], */
    component: Mega1Component, children:
      [
        { path: "sc1", component: XxxComponent},
        { path: "sc2", component: YyyComponent},
        { path: "", redirectTo : "sc1", pathMatch: "prefix"},
      ]
  },
];

```

L'éventuelle partie `providers: [ provideXyzDomain(config) ]`, est rarement nécessaire mais possible dans certains cas très pointus .

app.routes.ts

```

import { Route } from '@angular/router';
import { WelcomeComponent } from './welcome/welcome.component';

export const appRoutes: Route[] = [
  { path: "welcome", component : WelcomeComponent } ,
  { path: 'mega1',
    loadChildren: () => import('./mega1/mega1.routes').then((m) => m.mega1Routes)
  },
  { path: "", redirectTo: "/welcome", pathMatch:"full"}
];

```

Attention , ici pas de ~~loadComponent~~ : mais `loadChildren` :

2.3. Eventuelle visualisation du lazyLoading :

```

import { RouteConfigLoadEnd, RouteConfigLoadStart, Router, Event, RouterModule } from
 '@angular/router';
...
export class App {

```

```

router = inject(Router);
asyncLoadCount = 0;

constructor(){
  this.router.events.subscribe(
    ( event: Event ) : void => {
      if ( event instanceof RouteConfigLoadStart ) {
        this.asyncLoadCount++;
        console.log(">> start loading component or children routes:
          asyncLoadCount="+this.asyncLoadCount
          + " RouteConfigLoadStart event="+JSON.stringify(event));
      } else if ( event instanceof RouteConfigLoadEnd ) {
        this.asyncLoadCount--;
        console.log("<< end loading component or children routes:
          asyncLoadCount="+this.asyncLoadCount);
      }
    }
  );
}

```

Eléments affichés à la console lors d'un "lazy loading" , en gérant les événements
"RouteConfigLoadStart" et **"RouteConfigLoadEnd"** :

```

>> start loading component or children routes: asyncLoadCount=1 RouteConfigLoadStart
event={"route":{"path":"small_or_mega_"},"type":9}
<< end loading component or children routes: asyncLoadCount=0

```

NB : on voit cela avec

```

{ path : "small_or_mega_ " ,
  loadComponent or loadChildren: () => import('....').then(m => m.....) },

```

mais pas avec

```

{ path : "small_or_mega_ " , component : SmallOrMegaComponent},

```

3. @defer (avec angular>= 17)

Automatic lazy (deferred) loading

Exemple :

```
<p>with-defer</p>

<hr/>
<h3>Timer(3s)</h3>
@defer (on timer(3s)) {
  <div class="greenBackground">Visible after 3s</div>
}
@placeholder {
  <div class="yellowBackground">Placeholder (before 3s)</div>
}

<hr/>
<div #aPagePart>aPagePart</div>

@defer (on viewport(otherPagePart) ; prefetch on idle) {
  <p class="greenBackground">final render</p>
} @placeholder(minimum 2s) {
  <div class="yellowBackground">...placeholder.. (before final rendering with lazy loading
...)</div>
}

<hr/>
**<br/>**<br/>**<br/>**<br/>**<br/>**<br/>**<br/>**<br/>**<br/>
<div #otherPagePart>otherPagePart</div>
```

Timer(3s)

Placeholder (before 3s)

Timer(3s)

Visible after 3s

aPagePart

...placeholder.. (before final rendering with lazy loading ...)

**
**
**
**

aPagePart

final render

**
**
**
**
**
**
**
**
**
**

otherPagePart

NB :

- for user experience , @defer and @placeholder are important
- for developper/debug , @loading and @error may be useful

triggers:

<div #otherPagePart>otherPagePart</div>

@defer (**on viewport**(otherPagePart)) **when #otherPagePart is visible on viewport (visible part of html page)**

@defer (**on hover**(otherPagePart)) when mouse/... over #otherPagePart

@defer (**on interaction**(aControlOfTheTemplate)) when user interact with aControlOfTheTemplate

@defer (**when** aBooleanExpression) when aBooleanExpression is true (after changing by code)

VI - Architecture micro-frontend

1. Micro-frontend avec angular

1.1. Principes de l'architecture micro-frontend

Constituer un gros frontEnd par agrégation de plusieurs petits frontEnds .

Pistes pour quelques réutilisations (s'approchant du micro-frontEnd):

- Utilisation de "*webComponents*" (composants réutilisables/paramétrables de plus ou moins grosse granularité) .

Anciennes pistes/solutions pour l'architecture "micro-frontend":

- **Single-SPA** : méta-framework où chaque micro-frontend implémente 3 méthodes (*bootstrap* pour initialisations , *mount* pour rattachement au DOM et *unmount* pour détachement au DOM) → solution originellement basé sur SystemJS (en perte de vitesse) et multiples problèmes (looks , défaut d'affichage, ...) → *ça semble tomber à l'eau* .
- **Assemblage/composition effectué coté serveur** (Server Side Include , SSR + Hydratation , ...) → *pas très flexible et solutions à priori lié à un framework précis* .

Pistes/solutions actuelles (2025, 2026,...) pour l'architecture "micro-frontend":

- **Assemblage/composition effectué dynamiquement au runtime coté navigateur** (ex : webpack module federation , native federation, ...)

Pistes pour communications entre les micro-frontends :

- custom events
- contexte commun (ex : LocalStorage , ...)
- modèle de données partagées , contrat d'interface , ...

Bases d'implémentations pour l'architecture micro-frontEnd de type "client side federation" :

Module Federation	Basé sur webpack >=5 , <i>fonctionne avec plusieurs types de frameworks (vue , angular , react, ...) mais assez complexe</i> et attention : beaucoup de technologies récentes ne s'appuient plus sur webpack (les versions récentes de Vue et Angular s'appuient sur Vite et pas webpack)
Native Federation	Basé sur une exploitation du chargement dynamique de modules ESM depuis du javascript moderne coté client . A court terme les propositions sont assez liées à un framework précis (angular ou vue ou react ou ...) et la mise en œuvre est assez simple (basée sur Vite ou autre et le framework angular ou autre)

NB: a court terme , "webpack module federation" peut avoir un intérêt pour intégrer de l'existant ou bien pour intégrer des micros frontEnds hétérogènes (react , vue, angular, ...).

1.2. Micro-frontEnd Angular avec native federation

ng add @angular-architects/native-federation

à chaque niveau (projet fédérateur et projets "micro front end") .

Et ajuster si besoin la version pour quelle corresponde à celle du projet angular.

Exemple (dans package.json , *avant éventuel npm install*) :

```
"@angular-architects/native-federation": "^21.1.1",
```

Au minimum , 2 ou 3 projets complémentaires :

Exemple: *shell/main/root* et *mfe1* , *mfe2* (*micro-front-end 1* et 2)

Variante des commandes en mode nx/monorepo :

```
npx create-nx-workspace@latest my-nx-workspace --preset=angular-monorepo
```

```
npm install @angular-architects/native-federation --save-dev
```

```
nx g @nx/angular:app apps/shell
```

```
nx generate @angular-architects/native-federation:ng-add --project shell --port 4200 --type dynamic-host
```

```
nx g @nx/angular:app apps/mfe1
```

```
nx generate @angular-architects/native-federation:ng-add --project mfe1 --port 4201 --type remote
```

et d'autre part en version nx/monorepo , *project.json* remplace angular.json

1.2.a. Paramétrages à effectuer sur le projet fédérateur (ex : shell)

- **Commenter ou supprimer** la partie *exposes* dans **federation.config.js** car inutile sur le projet fédérateur .
- Ajouter le sous répertoire assets dans src ou bien utiliser `"/public"` à coté se `"src"`.
- mettre à jour si besoin angular.json en ajoutant éventuellement `"src/assets"` à coté de `public` :

```
      "assets": [
        {
          "glob": "**/*",
          "input": "public"
        },
        "src/assets"
      ],
```

- créer un nouveau fichier **src/assets/federation.manifest.json** ou bien **public/federation.manifest.json** avec un contenu de ce genre :

```
{
  "mfe1": "http://localhost:4201/remoteEntry.json",
  "mfe2": "http://localhost:4202/remoteEntry.json"
}
```

- Modifier le fichier **main.ts** en y ajoutant 'federation.manifest.json' dans `initFederation()` :

```
import { initFederation } from '@angular-architects/native-federation';
```

```
initFederation('federation.manifest.json')
```

```
.catch(err => console.error(err))
```

```
.then(_ => import('./bootstrap'))
```

```
.catch(err => console.error(err));
```

'/assets/federation.manifest.json' ou bien 'federation.manifest.json' selon contexte

- Ajouter les routes dynamiques vers les micro-frontEnd au sein du fichier **app.routes.ts** :

```
import { loadRemoteModule } from '@angular-architects/native-federation';
import { Routes } from '@angular/router';

export const routes: Routes = [
  { path: "welcome" , component : WelcomeComponent} ,
  { path: 'mfe1',
    loadComponent: () => loadRemoteModule('mfe1', './Component').then((m) => m.AppComponent),
  },
  { path: 'mfe2',
    loadComponent: () => loadRemoteModule('mfe2', './Component').then((m) => m.App),
  },
  { path: "" , redirectTo: "/welcome" , pathMatch:"full"}
];
```

NB : il faudra peut être ajuster **(m) => m.AppComponent** en **(m) => m.App** dans le cas où l'application **mfe1** ou **mfe2** a été créée avec ng new d'**angular 21+** .

- Veiller à ce que **app.html** (ou app.component.html) comporte bien

```
<a routerLink="/welcome">welcome</a> &nbsp; ; <a routerLink="/mfe1">mfe1</a>
<!-- &nbsp; ; <a routerLink="/mfe2">mfe2</a> -->
<hr/>
<h1>{{title()}}</h1>
<hr/>
<router-outlet />
```

et pas trop de choses inutiles .

1.2.b. Paramétrages à effectuer sur les projets micro-front-ends :

Changer le numéro de port (en phase de développement) pour éviter certains conflits :

Exemple, dans **angular.json** :

```
"architect": {
  "serve-original": {
    ...
    "configurations": {
      ...
    },
    "options": {
      "port": 4201
    }
  },
}
```

Adapter si besoin le contenu du fichier app.html ou app.component.html :

```
<h3>{{title()}}</h3>
```

1.2.c. Exploitation élémentaire du micro front end angular



shell (federating host application)

micro-front-end-1

NB: en exposant et chargeant un composant à distance, on obtient un fonctionnement correct si le composant chargé imbrique de façon rigide certains sous composants (sans sous route) .

1.2.d. Exposition/importation de routes (avec paquet composants)

Pour charger d'un coup , tout un paquet de composants cohérents (avec des sous routes et navigations associés) , on aura tout intérêt à structurer les choses de la façon suivante :

Dans le projet fédérateur (ex : shell) :

```
import { loadRemoteModule } from '@angular-architects/native-federation';

export const routes: Routes = [
  { path: "welcome" , component : WelcomeComponent } ,
  { path: 'mfe1',
    loadComponent: () => loadRemoteModule('mfe1', './Component').then((m) => m.App)
    loadChildren: () => loadRemoteModule('mfe1', './routes').then((m) => m.routes)
  },...];
```

Syntaxe précise: `loadRemoteModule('microFrontEndName', './exposedName')`
`.then((m) => m.routeOrCompClassName)`

Dans un des projets "micro-front-end" (ex : mfe1) , on va créer un composant qui représentera la partie principale de ce qui exposé par le paquet de routes app.routes.ts .

Par exemple via ***ng g c mfe1 --type component***

Ce composant pourra idéalement comporter en lui de quoi naviguer (en relatif)
 + `<router-outlet/>` et sous routes (children)

C'est cet ensemble cohérent qui sera importé par le projet fédérateur .

Le composant App (ou AppComponent) du projet "mfe1" ne sera pas exposé/exporté, il ne sera utile que pour effectuer des tests isolés (seul, :4201, sans fédération).

Dans projet "mfe1"

Dans federation.config.js :

```
exposes: {
  './Component': './src/app/app.ts',
  './routes': './src/app/app.routes.ts',
},
```

Dans app.routes.ts :

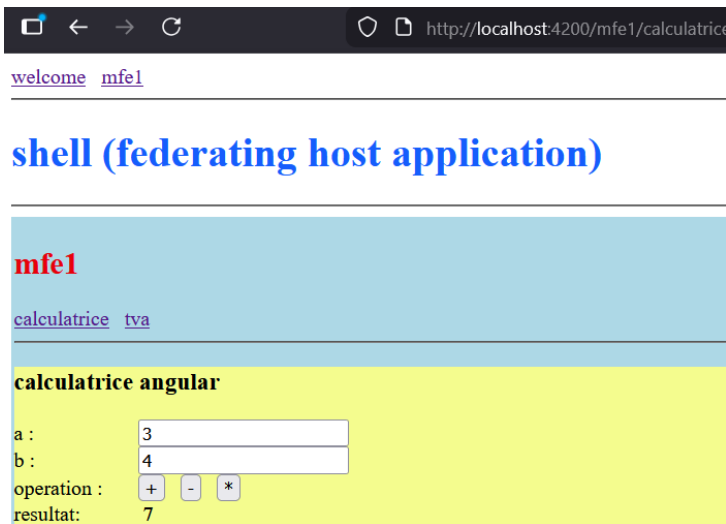
```
export const routes: Routes = [
  { path: "", component: Mfe1Component, children:
    [
      { path: "calculatrice", component: CalculatriceComponent},
      { path: "tva", component: TvaComponent},
      { path: "", redirectTo: "calculatrice", pathMatch: "prefix"},
    ]
  },
];
```

Dans App.html :

```
<router-outlet></router-outlet>
```

Dans mfe1.component.html :

```
<div class="mfe1">
  <h2 class="text-red-600">mfe1</h2>
  <nav>
    <a routerLink="calculatrice">calculatrice</a> &nbsp; <a routerLink="tva">tva</a>
  </hr>
  </nav>
  <b><router-outlet></router-outlet></b>
</div>
```



1.2.e. Pour aller plus loin sur micro-front-end et native federation

Eventuelles communications entre micro-frontends :

- Solution 1 : librairie partagée avec Service commun
- Solution 2 : Evenements personnalisés
- Solution 3 : basic 'localStorage'

Eventuelle utilisation conjointe de WebComponents :

- peut être utile pour si besoin mixer certains frameworks (vue, angular, react)
- ...

Attention: tout l'intérêt de l'architecture micro-frontend réside dans le fait d'assembler des sous parties les plus indépendantes possibles et donc toutes les communications entre micro-frontends doivent être les plus légères et les plus souples possibles (surtout pas trop d'inter-dépendances).

La solution la plus efficace et simple à mettre en œuvre consiste à :

- placer les différents projets (shell , mfe1, mfe2, **my-lib**) dans un **même workspace "NX/angular-monorepo"** .
- configurer le partage de librairie de cette façon au sein de **federation.config.js** :

```
...
module.exports = withNativeFederation({
  ...
  shared: {
    ...shareAll({ singleton: true, strictVersion: true, requiredVersion: 'auto' } ),
  },
  sharedMappings: [ '@my-nx-workspace/my-lib' ],
  skip: [ ... ] , ...
})
```


VII - Formulaires avancés et dynamiques

Principes d'un formulaire dynamique :

à partir d'un modèle descriptif coté ts (éventuellement basé sur reactiveForm/formBuilder ou bien sur signalForm) on effectue (via des boucles automatiques) une génération dynamique de formulaire de saisies du coté template html

1. Formulaires en mode reactive-form

1.1. Approche "model-driven" / "reactive-form"

dans composant angular :

```
import { FormGroup, FormControl, Validators } from '@angular/forms';
....
class ModelFormComponent {
  myForm: FormGroup;

  constructor() {
    this.myForm = new FormGroup({
      firstName: new FormControl('', Validators.required ),
      lastName: new FormControl('default_name', [ Validators.required ,
                                                    Validators.pattern("[A-Z].+") ] ),
      email: new FormControl('', [ Validators.required, Validators.email ] ),
      password: new FormControl('',[ Validators.required, Validators.minLength(8) ] )
    });
  }
}
```

avec `new FormControl( 'valeur_par_defaut', [ liste_de_valdateurs ] )`

dans `@Component()` ou `app.module.ts` :

```
import { ReactiveFormsModule } from '@angular/forms';
...
imports: [ ... , ReactiveFormsModule , ... ],
...
```

dans template HTML :

```
<form novalidate [formGroup]="myForm">
  <div class="...">
    <label>First Name</label>
    <input type="text" class="..." formControlName="firstName" >
  </div>
  <div class="...">
    <label>Last Name</label>
    <input type="text" class="..." formControlName="lastName" >
  </div>
  <div class="...">
```

```

    <label>Email</label>
    <input type="email" class="..." formControlName="email" >
  </div>    ...
  <button (click)="onSubmit()"
    [disabled]="!myForm.valid" >submit</button>
  <br/>
</form>

```

NB :

- **novalidate** (dans <form ...>) signifie *pas de validation HTML5 automatiquement effectuée par le navigateur mais simplement par l'application angular* .
- en mode "model-driven" / "reactive-form" , pas besoin de `[(ngModel)]="xxx.yyy"` mais on récupère (dans `onSubmit()` ou ...) les données saisies au sein de **`myForm.value`** .
- Pour transformer si besoin **`myForm.value`** en une instance d'une classe de données (data / model / ...) on peut par exemple s'appuyer sur un bon constructeur :

```

public onSubmit(){
  const r = this.myForm.value; //rawFormValuesObject
  this.reservation = new Reservation(r.firstName , r.lastName , r.telephone ,
                                     r.email , this.dateTimeFromDateAndLocalTime(r.date,r.time) );
  this.message="résa effectuée=" + JSON.stringify(this.reservation);
}

```

Accès aux détails d'un champ d'un formulaire contrôlé par angular :

`myForm.controls['formControlName'].errors` // **`.dirty`** , **`.valid`** , ...

Exemple (à adapter):

```

myFieldErrorMessage(fieldName:string, fieldExpectation:string=""){
  const myForm= this.xyzForm;
  //myForm.controls[fieldName].invalid && (myForm.controls[fieldName].dirty || myForm.controls[fieldName].touched)
  let errMsg="";
  const vErrors : ValidationErrors | null = myForm.controls[fieldName].errors;
  if(vErrors!=null){
    console.log(JSON.stringify(vErrors));
    if(vErrors['required']===true)
      errMsg=`${fieldName} is required`;
    else if(vErrors['minlength'])
      errMsg=`${fieldName}.minLength=${vErrors['minlength'].requiredLength}`;
    else if(vErrors['email']===true)
      errMsg=`${fieldName} should be a valid email with ...@...`;
    else
      errMsg=`${fieldName} ${fieldExpectation}`;
  }
  return errMsg;
}

```

Utilisation du coté .html :

```
<div class="f-align smallError">{{myFieldErrorMessage('lastName')}}</div>
```

ou bien

```
{{myFieldErrorMessage('lastName','must start with uppercase and other character')}}}
```

1.2. Avec l'aide de FormBuilder

En mode *model-driven* / *reactiveForm* ,
de façon à construire plus simplement le paramétrage d'un FormGroup avec FormControl et
Validateurs imbriqués , on peut éventuellement s'appuyer sur FormBuilder :

Exemple :

```
import { Component } from '@angular/core';
import { FormBuilder, FormGroup, Validators } from '@angular/forms';

@Component({ ... })
export class AppComponent {
  myForm: FormGroup;

  constructor(private _formBuilder: FormBuilder) {
    this.myForm = this._formBuilder.group({
      name: ['default_name', [Validators.required, Validators.pattern("[A-Z].+")]],
      email: ['', [Validators.required, Validators.email]],
      message: ['', [Validators.required, Validators.minLength(15)]]
    });
  }
}
```

Attention aux double [[]] :

- un premier pour ['valeur_par_defaut' , validateur(s)]
- un second imbriqué pour la liste des Validateurs

1.3. Exemple de Validateur personnalisé / spécifique :

url.validator.ts

```
import { AbstractControl } from '@angular/forms';

export function ValidateUrl(control: AbstractControl) {
  if (!control.value.startsWith('https') || !control.value.includes('.io')) {
    return { invalidUrl: true }; //return { errorKeyname : true } if invalid
  }
  return null; //return null if ok (no error)
}
```

Utilisation :

```
this.myForm = this._formBuilder.group({
  userName: ['', Validators.required],
  websiteUrl: ['', [Validators.required, ValidateUrl ]],
});
```

Exemple de comportement :

reservation

firstName:	prenom	
lastName:	nom	lastName must start with uppercase and other character
telephone:	0605040306	
email:	prenom.Nomxyz.com	email should be a valid email with ...@...
date:	2025-09-0	date yyyy-mm-dd
time:	13:24	

NB : du côté styles css pour un alignement correct (idéalement responsive) on pourra s'appuyer sur bootstrap-css ou bien au minimum sur css3 et par exemple des flex-box :

```
...
<div class="f-form-group-wrap">
  <label class="f-align">firstName:</label>
  <div class="f-align">
    <input type="text" class="f-max-size" formControlName="firstName">
  </div>
  <div class="f-align smallError">{{myFieldErrorMessage('firstName')}}</div>
</div>
...
```

styles simples améliorables :

```
.smallError { color: red; font-size:small; }
.f-form-group-wrap, .f-row-wrap { display: flex; flex-flow: row wrap; }
label.f-align { width: 12em; display: inline-block; margin-left: 0.3em; flex: 0 0 12em; }
.f-align { margin-left: 0.3em; margin-right: 0.2em; flex: 1 1 auto; }
.f-max-size { width: 98%; }
```

2. SignalForms (preview , depuis Angular 21)

2.1. Grandes lignes de l'api "signalForms"

Contrairement aux anciennes approches "template-driven" et "reactive-form" où certaines classes ng-valid , ng-invalid étaient activées automatiquement , la nouvelle api signalForms n'active plus automatiquement certaines classes css . Il faut gérer ça autrement ou coder cela soi même .

Avantages/apports de l'api "signalForms" :

- meilleurs performances (car basé sur des signaux)
- mappings/bindings html/model proches de la logique ngModel mais avec enveloppe réactive
- contraintes de validation simples à exprimer (validateurs de @angular/forms/signals)
- beaucoup de souplesse dans la gestion des erreurs de saisies

2.2. structures élémentaires (model , form) du coté .ts

Interface et/ou classe de données facultative (pour typer/structurer) le modèle les données à saisir :

src/common/data/person.ts

```
export interface PersonData {
  firstname: string;
  lastname: string;
  email: string;
  age: number;
}
```

```
export class Person implements PersonData {
  constructor(
    public firstname : string = "",
    public lastname : string = "",
    public email : string = "",
    public age :number=0){}
}
```

```
import { email, form, FormField, minLength, pattern, required } from '@angular/forms/signals';
...
```

```
@Component({
  imports: [FormField],...
})
```

```
export class LoginWithSignalFormComponent {
  message = "";
  //personModel = signal<PersonData>({ firstname: " ", lastname: " ", email : " ", age: 0});
  personModel = signal<PersonData>(new Person());
  //personModel = signal<PersonData>(new Person('jean','Bon','jean.bon@xyz.com', 40));
  //form data model as WritableSignal that will be synchronized with inputs of form
  //([formField]="personForm.firstname" is bi-directionnal)
```

```
personForm = form(this.personModel); //version simple/élémentaire sans validateur
```

```
onPerson(){
  this.message="valeurs saisies=" + JSON.stringify(this.personModel());
}
}
```

Le *model* est une enveloppe de type "WritableSignal" pour l'objet de données à saisir.
L'objet *form* est ensuite créé à partir du *model* (avec ou sans validateur).

2.3. Mappings [formField] du côté .html

La directive [formField] permet de paramétrer un **binding bi-directionnel** entre une zone de saisie (souvent <input>) et une partie de l'objet *form* enveloppant lui-même le modèle.

Le comportement du formulaire est entièrement dynamique :

Au fur et à mesure des saisies, les valeurs des champs de saisies sont accessibles via `xxxForm.fieldName().value()` et l'objet comportant globalement toutes les valeurs saisies correspond au **modèle** initial qui (sous forme de *WritableSignal*) est automatiquement mis à jour (ici `this.personModel`).

```
<h2> signalForm (person) in responsive tailwind css grid </h2>
<form class="grid grid-cols-1 md:grid-cols-2 gap-4">
  <div> <label class="block text-sm">firstname:</label>
    <input [formField]="personForm.firstname" class="w-full" />
  </div>
  <div> <label class="block text-sm">lastname:</label>
    <input [formField]="personForm.lastname" class="w-full" />
  </div>
  <div> <label class="block text-sm">email:</label>
    <input [formField]="personForm.email" class="w-full" />
  </div>
  <div> <label class="block text-sm">age:</label>
    <input type="number" [formField]="personForm.age" class="w-full" />
  </div>
</form>
<hr/>
<p>Hello {{ personForm.firstname().value() }} !</p>
<p>age: {{ personForm.age().value() }}</p> <!-- ou bien {{personModel().age}} -->
<input type="button" value="login" (click)="onPerson()" /> <br/>
message: {{message}}
```

firstname:	lastname:
jean	Bon
email:	age:
jean.bon@xyz.com	40

Hello jean!

age: 40

message: valeurs saisies={"firstname":"jean","lastname":"Bon","email":"jean.bon@xyz.com","age":40}

2.4. Valideurs et gestion des erreurs

La véritable valeur ajoutée de `[formField]="xxxForm.fieldName"` de l'api `signalForms` vis à vis d'un simple `[(ngModel)]="data.fieldName"` tient dans les contrôles de saisies .

Pour cela il faut commencer par **ajouter des validateurs dans la construction de l'objet form** du coté `.ts`:

```
import { email, form, FormField, min, minLength, pattern, required } from
'@angular/forms/signals';

...
personForm = form(this.personModel);
personForm = form(this.personModel, (schemaPath) => {
  required(schemaPath.firstname, {message: 'firstname is required'});
  required(schemaPath.lastname, {message: 'lastname is required'});
  minLength(schemaPath.firstname, 2, {message: 'firstname length must be at least 2'});
  email(schemaPath.email, { message: 'Please enter a valid email address' });
  min(schemaPath.age, 18, { message: 'You must be at least 18 years old' });
  pattern(schemaPath.lastname, /^[A-Z].+/,
    {message: 'lastname must start by uppercase , at least 2 characters'});
});
```

Du coté `.html` , via les directives `[formField]="xxxForm.fieldName"` les **contraintes de saisies issues des validateurs** seront **automatiquement prises en compte**.

Si l'on souhaite bloquer un bouton de type "submit" ou autre tant que les données ne sont pas entièrement bien saisies il suffit de spécifier `[disabled]="personForm().invalid()"` .

Validation de formulaire en mode global:

```
<input type="button" value="validate" (click)="onValidatePersonForm()" />
<!-- avec ou sans [hidden]="personForm().valid()" -->
```

et une méthode `onValidatePersonForm()` de ce genre :

```
onValidatePersonForm() {
  this.ok=true; this.message="";
  if(/*this.personForm.firstname().touched() &&*/ this.personForm.firstname().invalid()){
    this.ok=false;
    for(let e of this.personForm.firstname().errors())
      this.message += ` ${e.message} `
  }
  if(this.personForm.age().invalid()){ this.ok=false;
    for(let e of this.personForm.age().errors())
      this.message += ` ${e.message} `
  }
}
```

firstname:	lastname:
j	Bon
email:	age:
jean.bon@xyz.com	4
validate login	
message: firstname length must be at least 2 You must be at least 18 years old	

Validation plus dynamique pour chaque champ :

```

@Component({ imports: [FormField, NgClass, JsonPipe], ... })
export class LoginWithSignalFormComponent {
  ...
  messagePerson = "";
  okPerson=true;

  okEffect = effect( ()=>{ this.okPerson = ! this.personForm().invalid(); this.messagePerson="" })

  firstnameError=computed(
    ()=> this.personForm.firstname().errors().map(e=>e.message).join(" "));
  ...
  ageError=computed(()=> this.personForm.age().errors().map(e=>e.message).join(" "));

  isFieldValid(fieldName:string ){
    switch(fieldName){
      case "firstname": return ! this.personForm.firstname().invalid();
      case "lastname": return ! this.personForm.lastname().invalid();
      case "email": return ! this.personForm.email().invalid();
      case "age": return ! this.personForm.age().invalid();
      default : return false;
    }
  }

  classForField(fieldName:string) {
    let v= this.isFieldValid(fieldName);
    return {
      'ng-valid': v,
      'ng-invalid': !v,
    }
  }
}

```

```

<form novalidate class="grid grid-cols-1 md:grid-cols-2 gap-4">
  <div>
    <label class="block text-sm">firstname:</label>
    <input [formField]="personForm.firstname"
      [ngClass]="classForField('firstname')" class="w-full" />
    <span class="text-red-600">{{firstnameError}}</span>
  </div>
  ... </form> <hr/>
  person : <span [style.color]="okPerson?'green':'red'">{{ personModel() | json }}</span> <br/>

```

Dans src/styles.css (ou ailleurs) :

```

input.ng-valid { border-left: 5px solid #42A948; /* green */}
input.ng-invalid { border-left: 5px solid #a94442; /* red */}

```

NB: en mode signalForms , les classes de styles css ne sont pas obligées de s'appeler ng-valid ou ng-invalid , on pourrait les appeler autrement .

firstname:	lastname:
j	bon
firstname length must be at least 2	lastname must start by uppercase , at least 2 characters
email:	age:
jean.bonxyz.com	4
Please enter a valid email address	You must be at least 18 years old
person : { "firstname": "j", "lastname": "bon", "email": "jean.bonxyz.com", "age": 4 }	

firstname:	lastname:
jean	Bon
email:	age:
jean.bon@xyz.com	40
person : { "firstname": "jean", "lastname": "Bon", "email": "jean.bon@xyz.com", "age": 40 }	

VIII - Patterns , gestions d'états avancées (ngrx,...)

....

1. Extension ngrx

1.1. concepts et terminologie "redux" du concurrent "react"

Avec le framework react sans redux l'état d'une application est complètement décentralisé à travers tous les composants de celle-ci et les mises à jour des states des principaux composants ne suivent pas d'ordre précis à cause du temps de réponse du serveur côté API, de l'asynchronicité, de la gestion des routes, etc.

Redux (version simplifié de "flux") sert à **maintenir**, en arrière plan des "vues" de react, **un état global** à l'application bien cohérent et bien synchronisé entre tous les composants .

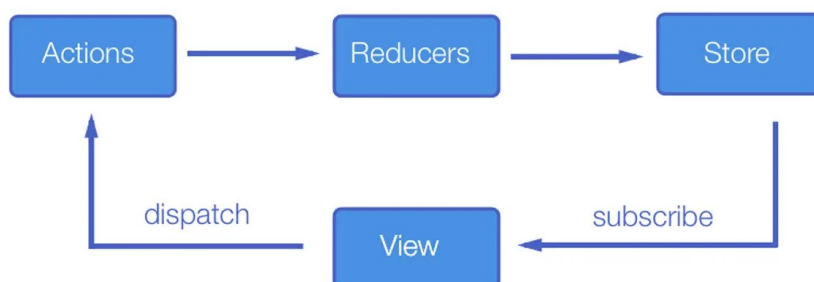
Eléments de redux	Rôles/caractéristiques
global state	objet principal (permettant de mémoriser l'état de toute l'appli) comportant de multiples sous objets <i>immutables</i> et <i>remplacés</i> .
action	objet " <i>demande de traitement</i> " permettant de modifier le "global state" (proche d'un objet " <i>command</i> " du design pattern de même nom)
reducer	objet technique sous forme de fonction pure permettant d'appliquer une action sur tout ou une partie d'un "global state existant" de façon à générer un "nouveau global state" ou une nouvelle sous-partie.
store	là où est stocké le "global state"

Le terme "*reducer*" vient du type de fonction que l'on peut appliquer à [Array.prototype.reduce\(\)](#).

Principales propriétés du global state :

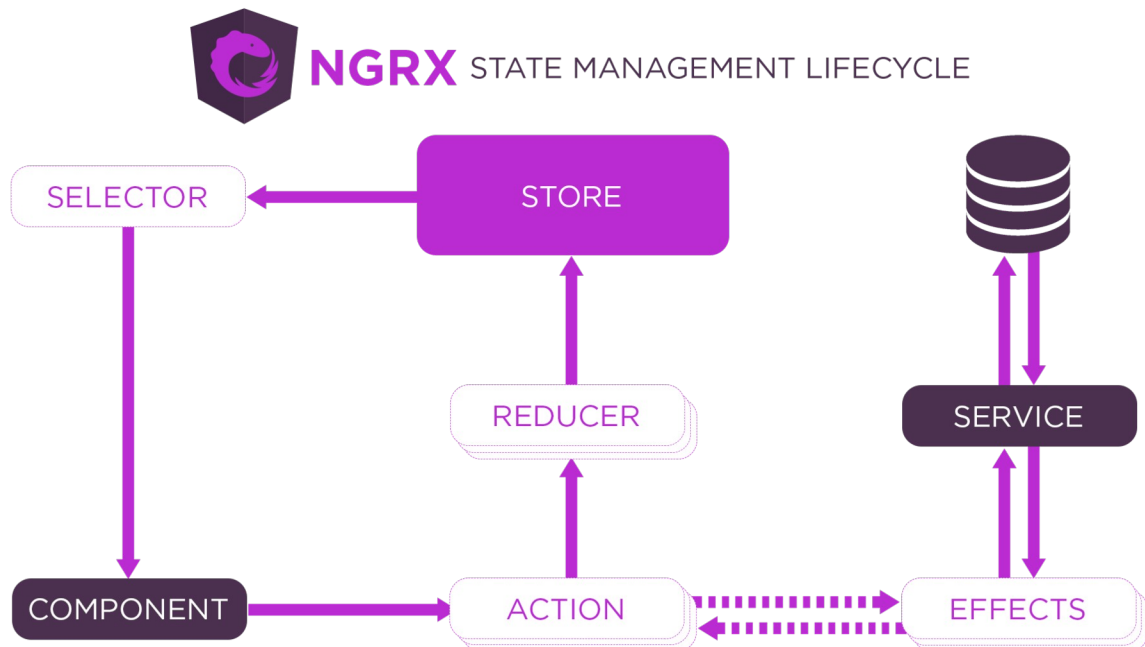
- unique (mais composé de plein de substates complémentaires : un par "reducer")
- en lecture seule (mais dont certaines parties sont remplacée par de nouvelles)

Redux



1.2. @ngrx/store (immutables , ...)

@ngrx/store est une extension permettant d'intégrer le concept de "*état global maîtrisé*" au sein d'une application angular .



- [Actions](#) describe unique events that are dispatched from components and services.
- State changes are handled by pure functions called [reducers](#) that take the current state and the latest action to compute a new state.
- [Selectors](#) are pure functions used to select, derive and compose pieces of state.
- State is accessed with the [Store](#), an observable of state and an observer of actions.

SHARI principe :

- **Shared**: state that is accessed by many components and services.
- **Hydrated**: state that is persisted and rehydrated from external storage.
- **Available**: state that needs to be available when re-entering routes.
- **Retrieved**: state that must be retrieved with a side-effect.
- **Impacted**: state that is impacted by actions from other sources.

installation de l'extension @ngrx/store :

```
npm install @ngrx/store --save
```

ou mieux encore

```
ng add @ngrx/store@latest
```

ajoutant **StoreModule.forRoot({})** au imports de *app.module.ts*

ou bien *provideStore()* dans *app.config.ts*

Exemple (proche du tutoriel officiel) :**counter.actions.ts**

```
import { createAction } from '@ngrx/store';
// just unique names :
export const increment = createAction('[Counter A_Scope] Increment');
export const decrement = createAction('[Counter A_Scope] Decrement');
export const reset = createAction('[Counter A_Scope] Reset');
```

pub.actions.ts (old v1)

```
import { createAction } from '@ngrx/store';
export const disablePub = createAction('[Pub A_Scope] Disable');
export const enablePub = createAction('[Pub A_Scope] Enable');
export const togglePub = createAction('[Pub A_Scope] Toggle');
```

pub.actions.ts (v2 via createActionGroup)

```
import { createActionGroup, emptyProps, props } from '@ngrx/store';
export const PubsActions = createActionGroup({
  source: 'Pub',
  events: {
    'disablePub': emptyProps(),
    'enablePub': emptyProps(),
    'togglePub': emptyProps(),
    'setPub': props<{ active : boolean }>(),
  },
});
```

counter.reducer.ts

```
import { Action, createReducer, on } from '@ngrx/store';
import { increment, decrement, reset } from './counter.actions';

export const initialState = 0;

export const counterReducer = createReducer(
  initialState,
  on(increment, (state) => state + 1),
  on(decrement, (state) => state - 1),
  on(reset, (state) => 0)
);
```

pub.reducer.ts

```
import { Action, createReducer, on } from '@ngrx/store';
import { PubsActions } from './pub.actions';

export const initialPub = false;

export const pubReducer = createReducer(
  initialPub,
  on(PubsActions.disablePub, (state) => false),
  on(PubsActions.enablePub, (state) => true),
  on(PubsActions.togglePub, (state) => !state),
  on(PubsActions.setPub, (state, { active }) => active)
);
```

Configuration (à l'ancienne) du store dans xyz.module.ts :

```

...
import { StoreModule } from '@ngrx/store';
import { counterReducer } from './counter.reducer';
...

@NgModule({
  ...
  imports: [
    ...
    // bind "counterReducer" to "count" (sub-)part
    // and "pubReducer" to "pub" (sub-)part of global state in @ngrx/store
    StoreModule.forRoot({ count: counterReducer , pub : pubReducer })
  ]
})
export class XyzModule { }

```

Configuration plus moderne du Store en mode "standalone app" :**app.config.ts**

```

...
import { provideStore } from '@ngrx/store';
import { provideEffects } from '@ngrx/effects';
import { counterReducer } from './common/store/reducers/counter.reducer';
import { pubReducer } from './common/store/reducers/pub.reducer';

export const appConfig: ApplicationConfig = {
  providers: [
    ...
    provideStore({ count: counterReducer , pub : pubReducer }),
    provideEffects()
  ]
};

```

time.counter.service.ts

```

import { Injectable } from '@angular/core';
import { Observable , Observer } from 'rxjs';

@Injectable({
  providedIn: 'root'
})
export class TimeCounterService {
  public getPeriodicTimeInfo() : Observable<any>{
    return Observable.create((observer :Observer<number>) => {
      const interval = setInterval(() => {
        observer.next(Date.now())
      }, 5000); //every 5000ms / 5s
    })
  }
  constructor() { }
}

```

xyz/my-counter/my-counter.component.css

```
.positif { color : blue; font-weight: bold;}
.negatif { color : red; font-style: italic;}
```

xyz/my-counter/my-counter.component.ts

```
import { Component, OnInit } from '@angular/core';
import { Store, select } from '@ngrx/store'; import { Observable } from 'rxjs';
import { TimeCounterService } from '../common/service/time-counter.service';
import { increment, decrement, reset } from '../common/store/actions/counter.actions';
import { PubsActions } from '../common/store/actions/pub.actions';

@Component({
  selector: 'app-my-counter', imports: [NgClass, AsyncPipe],
  templateUrl: './my-counter.component.html',
  styleUrls: ['./my-counter.component.css']
})
export class MyCounterComponent implements OnInit {
  count$: Observable<number>
  count_value : number=0;
  pub$: Observable<boolean>

  constructor(private store: Store<{ count: number ,
                                     pub : boolean }>,
               private timeCounterService : TimeCounterService) {
    this.count$ = store.pipe(select('count'));
    this.count$.subscribe((c)=>{this.count_value=c});
    this.pub$ = store.pipe(select('pub'));

    //reception reguliere de timeInfo toutes les 5s :
    timeCounterService.getPeriodicTimeInfo().subscribe(
      (timeInfo)=>{ console.log(timeInfo + " " + new Date(timeInfo).toLocaleTimeString());
                    this.store.dispatch(increment()); }
    )
  }

  // dispatch actions onEvents :
  onIncrement() { this.store.dispatch(increment()); }
  onDecrement() { this.store.dispatch(decrement()); }
  onReset() { this.store.dispatch(reset()); }
  onDisablePub() { this.store.dispatch(PubsActions.disablePub()); }
  onEnablePub() { /*this.store.dispatch(PubsActions.enablePub());*/
                  this.store.dispatch(PubsActions.setPub({active:true}));
                }
  onTogglePub() { this.store.dispatch(PubsActions.togglePub()); }

  ngOnInit(): void { }
  setClasses(){ return {
    "positif" : this.count_value >= 0 ,
    "negatif" : this.count_value < 0    }
  }
}
```

xyz/my-counter/my-counter.component.html

```

<p>my-counter with @ngrx/store</p>
<button (click)="onReset()">Reset Counter to 0</button> &nbsp;
<button (click)="onDecrement()">Decrement (--)</button> &nbsp;
Current Count:<span [ngClass]="setClasses()"> {{ count_value }}</span> &nbsp;
<button (click)="onIncrement()">Increment (++)</button>
<hr/>
<button (click)="onTogglePub()">TogglePub(On/Off)</button> &nbsp;
<button (click)="onDisablePub()">Disable (Off)</button> &nbsp;
Current Pub: <span>{{ pub$ | async }}</span> &nbsp;
<button (click)="onEnablePub()">Disable (On)</button>
@if((pub$ | async) == true){
  <p>pub (advertisement)</p>
}

```

my-counter with @ngrx/store

Reset Counter to 0 Decrement (--) Current Count: 3 Increment (++)

TogglePub(On/Off) Disable (Off) Current Pub: true Disable (On)

pub (advertisement)

Autre Exemple partiel "Articles" :

<pre> export interface Article { ref: string; label: string; price: number; features: string; } </pre>	<pre> export const ArticlesActions = createActionGroup({ source: 'Article', events: { 'addArticle': props<{ article: Article }>(), 'deleteArticle': props<{ ref: string }>(), }, }); </pre>
--	---

```

export const initialState: ReadonlyArray<Article> = [
  { ref: '1', label: 'Angular State Management with NgRx',
    price: 5.6, features: '345 pages'
  }
];

export const articleReducer = createReducer(
  initialState,
  on(ArticlesActions.addArticle, (state, { article }) => [...state, article]),
  on(ArticlesActions.deleteArticle, (state, { ref }) => {
    let newState: Article[] = [];
    for(let a of state){
      if(a.ref !== ref)
        newState.push(a);
    }
  })
);

```

<pre> return newState; })); </pre>	
<pre> export class ArticleComponent { cloneArticle(a: Article):Article{ return JSON.parse(JSON.stringify(a)); } articles\$!: Observable<Array<Article>>; article = { ref: '?', label: 'article_', price: 1, features: '?' } constructor(private store: Store<any>) {} ngOnInit(): void { this.articles\$ = this.store.pipe(select(state => state.article)); } onAddArticle() { this.store.dispatch(ArticlesActions.addArticle({ article: this.cloneArticle(this.article) })); } onDeleteArticle(ref:string) { this.store.dispatch(ArticlesActions.deleteArticle({ ref: ref })); } } </pre>	<pre> <!-- article list: {{ articles\$ async json }} --> @for(a of articles\$ async; track a){ {{a.ref}} {{a.label}} {{a.price}} {{a.features}} <button (click)="onDeleteArticle(a.ref)">delete</button> } <hr/> ref:<input [(ngModel)]="article.ref"/>&nbsp; label:<input [(ngModel)]="article.label"/>
 price:<input [(ngModel)]="article.price"/>&nbsp; features:<input [(ngModel)]="article.features"/>
 <button (click)="onAddArticle()">add</button> </pre>

- 1 Angular State Management with NgRx 5.6 345 pages [delete](#)
- ref2 article_2 1.6 ? [delete](#)
- ref3 article_3 1.8 ? [delete](#)

ref: label:
 price: features:

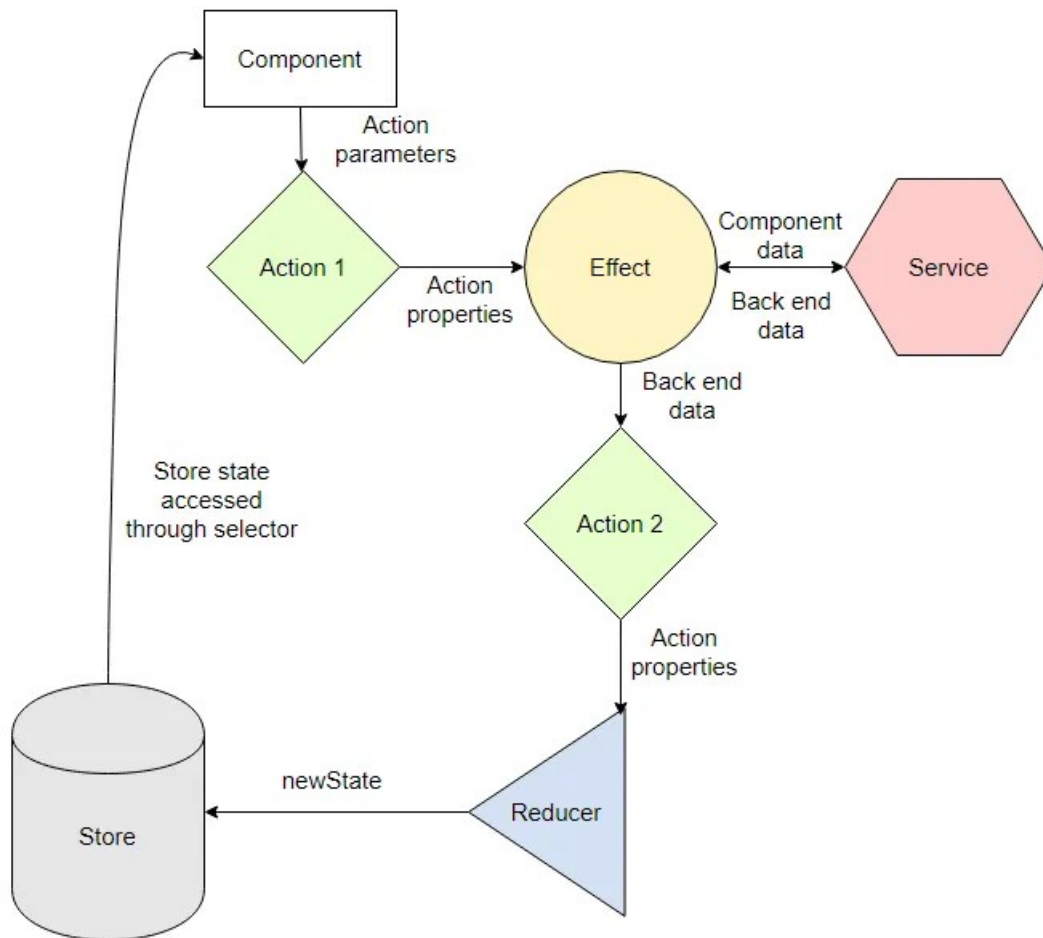
1.3. Redux ou équivalent utile pour petite appli angular ?

A priori "Non" car ça apporte beaucoup de complexité et de "petits morceaux de codes / de mécanismes" un peu éparpillés .

1.4. NgRx ou équivalent utile pour grande appli angular ?

Bien qu'apportant complexité et verbosité , l'utilisation de NgRx (ou de la variante adaptée NgXs) est un **gage de fiabilité** sur de grosse application angular au sein de laquelle on souhaite bien maîtriser l'état global .

2. Actions et Effets avec service (NgRx)



Effect au sens "side effect" of REST API call .

```
ng add @ngrx/store@latest
ng add @ngrx/effects@latest
```

user-session.ts

```
export class UserSession {
  constructor(public username:string="?",
    public connected:boolean=false,
    public roles:string=""){}
}
```

session.actions.ts

```

import { Action } from "@ngrx/store";
import { Login } from "../../common/data/login";
import { LoginResult } from "../../common/data/login-result";

export const enum SessionActionTypes {
  TRY_LOGIN = '[Session Before] tryLogin',
  NEW_VALID_SESSION = '[Session Valid] New',
  SESSION_END = '[Session Existing] End'
}

export class TryLogin implements Action {
  readonly type: string = SessionActionTypes.TRY_LOGIN;
  constructor(public login: Login) {}
}

export class NewValidSession implements Action {
  readonly type: string = SessionActionTypes.NEW_VALID_SESSION;
  constructor(public validLogin: LoginResult) {}
}

export class SessionEnd implements Action {
  readonly type: string = SessionActionTypes.SESSION_END;
  constructor() {}
}

export type SessionAction = TryLogin | NewValidSession | SessionEnd;

```

user-session-reducer.ts

```

import { LoginResult } from "src/app/common/data/login-result";
import { UserSession } from "../../model/user-session";
import { NewValidSession, SessionAction, SessionActionTypes } from "../../actions/session.actions";
const userSessionInitialState = new UserSession("?", false, "");

export function userSessionReducer(state = userSessionInitialState, action: SessionAction) {
  switch(action.type){
    case SessionActionTypes.SESSION_END:
      return userSessionInitialState;

    case SessionActionTypes.NEW_VALID_SESSION:
      let validLogin : LoginResult = (<NewValidSession> action).validLogin;
      return new UserSession(validLogin.username,
                             true,
                             validLogin.roles)
    case SessionActionTypes.TRY_LOGIN:
      //pas d'impact sur le state (simple pré-action à transformer via effect) --> as default
    default:
      return state;
  }
}

```

session-effects.ts

```

import { Injectable } from "@angular/core";
import { LoginService } from "src/app/common/service/login.service";
import { Actions, createEffect, ofType } from '@ngrx/effects';
import { NewValidSession, SessionActionTypes, SessionEnd, TryLogin } from "../actions/session.actions";
import { map, mergeMap, switchMap, tap } from "rxjs";

@Injectable()
export class SessionEffects {
  constructor(
    private actions$: Actions,
    private loginService: LoginService) {
  }

  tryLogin$ = createEffect( () => this.actions$.pipe(
    // Listen to TRY_LOGIN action
    ofType(SessionActionTypes.TRY_LOGIN),
    // Fetch payload
    map((action: TryLogin) => action.login),
    // Call API or any other desired stuff
    mergeMap(login => this.loginService.postLogin$(login).pipe(
      //tap(result => console.log("result=" + JSON.stringify(result))),
      map(result => result.ok?(new NewValidSession(result)):(new SessionEnd()))
    )
  ));
}

```

app.module.ts

```

...
imports: [ ...,
  StoreModule.forRoot({}),
  EffectsModule.forRoot([])
],
...

```

my-lazy-module.ts

```

import { EffectsModule } from '@ngrx/effects';
import { StoreModule } from '@ngrx/store';
...
imports: [ ...,
  StoreModule.forFeature("session",userSessionReducer),
  EffectsModule.forFeature([SessionEffects])
],
...

```

login.component.ts

```

import { Component } from '@angular/core';
import { select, Store } from '@ngrx/store';
import { Observable } from 'rxjs';
import { Login } from 'src/app/common/data/login';
import { UserSession } from '../common/model/user-session';
import { SessionEnd, TryLogin } from '../common/store/actions/session.actions';

@Component({
  selector: 'my-lazy2-login',
  templateUrl: './login.component.html',
  styleUrls: ['./login.component.scss']
})
export class LoginComponent {
  message="";

  userSession$: Observable<UserSession>;

  constructor(private store: Store<{session:UserSession}>) {
    this.userSession$ = store.pipe(select('session'));
    this.userSession$.subscribe((userSession)=>{
      this.message=userSession.connected?"connected":"not connected";
      console.log("message="+this.message + " userSession="+JSON.stringify(userSession));
    })
  }

  login : Login= new Login("admin1","pwdadmin1","user,admin");

  onLogin(){
    console.log("login saisi:"+ JSON.stringify(this.login));
    //Object.assign({},this.login) for several calls . Without this : readonly error
    this.store.dispatch(new TryLogin(Object.assign({},this.login)));
  }

  onLogout(){
    this.store.dispatch(new SessionEnd());
  }
}

```

login.component.html

```

<p>login </p>
username : <input [(ngModel)]="login.username" /> <br/>
password : <input [(ngModel)]="login.password" /> <br/>
roles : <input [(ngModel)]="login.roles" /> <br/>
<button (click)="onLogin()">login</button> &nbsp;
<button (click)="onLogout()">logout</button>
<hr/>
message: {{message}} <br/>
userSession : {{userSession$ | async | json}}

```

username :

password :

roles :

message: not connected

userSession : { "username": "?", "connected": false, "roles": "" }

message: connected

userSession : { "username": "admin1", "connected": true, "roles": "user,admin" }

1.1. Fondamentaux du routing angular

Page 93

1.2. Sous niveau de routage (children)

Une application angular peut comporter plusieurs niveaux de composants et sous composants.

Un `<router-outlet>` dans un composant principal permet par exemple de switcher de sous composants Xxx , Yyy , Zzz et un de ces sous composants (par exemple Yyy) peut à son tour comporter une balise `<router-outlet>` pour switcher de sous-sous-composants (Aa ou Bb).

Exemple d'url avec sous niveaux :

xxx/
yyy/aa
yyy/bb
zzz/

Exemple de configuration de routes avec sous niveau:

```
...  
const routes: Routes = [  
  { path: 'welcome', component: WelcomeComponent },  
  { path: '', redirectTo: '/welcome', pathMatch: 'full'},  
  { path: 'xxx', component: XxxComponent } ,  
  { path: 'yyy', component: YyyComponent ,  
    children: [  
      { path: 'aa', component: AaComponent },  
      { path: 'bb', component: BbComponent },  
      { path: '', redirectTo: 'aa', pathMatch: 'prefix'}  
    ]  
  }  
];
```

1.3. Routes paramétrées et navigation par code

1.3.a. configurations de routes avec paramètres logiques

```
// dans app.routes.ts  (:nomParametreLogique )
const routes: Routes = [ ...
  { path: 'xx/:categorie',   component: XxComponent }
  { path: 'yy/:yyId',       component: YyComponent }
];
```

1.3.b. Navigation avec paramètres et via lien hypertexte

```
<a [routerLink]="['/yy', numYy ]" ...> vers yy </a>
```

et

```
<a [routerLink]="['/xx', categorieXx ]" ...> vers xx </a>
```

où *numYy* et *categorieXx* sont des attributs/propriétés (avec des *valeurs variables*) de la la classe du composant à l'origine de la navigation (là où sont les liens hypertextes).

Si peu de catégories on peut envisager ce type de lien hypertexte:

```
<a [routerLink]="['/xx', 'categorieA' ]" ...> vers xx de categorie a </a>
<a [routerLink]="['/xx', 'categorieB' ]" ...> vers xx de categorie b </a>
```

1.3.c. Déclenchement d'une navigation par code (coté .ts)

...html

```
<button (click)="onNavigateYy()" > vers yy </button>
```

....ts

```
import {Router} from '@angular/router'; ...
export class .....Component {
  numYy : number = 1;
  categorieXX :string = "categorieA";
  private _router=inject( Router );
  onNavigateYy():void {
    let link = ['/yy', this.numYy]; //ou link = ['/yy'] ; si pas de paramètre
    this._router.navigate( link );
    // ou bien this._router.navigateByUrl(`/yy/${this.numYy}`); //avec quote inverse `...` !!!
  }
}
```

1.3.d. Récupération du paramètre accompagnant la navigation :

On s'intéressera dans ce paragraphe à tenir compte de la valeur d'un paramètre passé au bout de route activée .

Remarque très importante (pour comprendre et ne pas devenir fou) :

- (Cas "a") Suite à la série de navigation suivante :
yyy , **xxx** , **yyy** , des composants des classes **Yyy** , **Xxx** et **Yxx** seront ré-instanciés à chaque changement d'URL (et l'état des variables d'instance sera perdu) .
- (Cas "b") Suite à la série de navigation suivante :
detail_produit/1 , **detail_produit/2** , **detail_produit/3** (où seule change la valeur du paramètre en fin d'url) , l'instance de la classe **DetailProduit** est conservée et la **callback** enregistrée (via **subscribe**) sur **this._route.params** est automatiquement ré-appelée pour prendre en compte le changement de numéro de produit à détailler .

NB :

Dans le cas "a" , où les instances ne sont pas conservées , on peut coder simplement l'unique récupération d'un paramètre en fin de route activée via la notion de snapshot (valeurs à l'instant t):

Exemple :

```
class MyComponent {
  route = inject(ActivatedRoute) ;
  constructor() {
    const yyId: number = Number(route.snapshot.params['yyId']);
    // NB le nom logique du paramètre (ici yyId ) doit correspondre à celui
    // de la route { path: 'yy:yyId', component: YyComponent } configurée dans
    // app.routes.ts
    ...
  }
}
```

Dans le cas "b" , on a besoin d'enregistrer une callback potentiellement réappelée plusieurs fois (à chaque changement de valeur du paramètre) :

```
import {Component , OnInit} from '@angular/core';
import { ActivatedRoute, Params} from '@angular/router';
import { Location } from '@angular/common';
...
export class YyComponent implements OnInit{
  yyId: number = 0;
  y : Yy | undefined; message : string = "...";
  private _yyService =inject(YyService)
  private _route=inject(ActivatedRoute)
  private _location= inject( Location)

  constructor(){}
  ngOnInit() {
    this._route.params.subscribe(
      (params: Params) => {
```

```

        this.yId = Number(params['yyId']); //où 'yyId' est le nom du paramètre
        // dans l'expression de la route avec path: 'yy/:yyId'
        // du fichier app.route.ts
        this.fetchYy();
    }
    );
}
fetchYy(){
    this._yyService.getYyObservable(this.yId).subscribe(yy=>this.y = yy ,
        error => this.message = <any>error);
}
goBack(): void {    this._location.back(); /* retour arrière dans historique des navigations */ }
}

```

2. Route conditionnée par gardien

Il est possible de créer (et enregistrer dans une route) des éléments techniques (ex : CanActivateRouteGuard) qui implémentent un rôle "Can...." (ex : CanActivate) pour contrôler si une route peut ou pas être activée selon (par exemple) une authentification utilisateur effectuée ou pas .

NB : en cas de route bloquée, il n'y a pas d'automatisme de type lien hypertexte grisé ou invisible. Une telle fonctionnalité doit éventuellement/potentiellement être codée en complément .

Dans la définition des routes , on pourra déclarer une liste de gardiens à utiliser :

```

...
{ path: 'dashboard',
  component: DashboardComponent,
  canActivate: [CanActivateRouteGuard]
}, ...

```

2.1. Historique de l'évolution des gardiens dans angular

- Avant la version 7.1 d'angular les gardiens étaient basiques : valeur de retour booléenne et blocage sans explication
- Depuis Angular 7.1 , la méthode canActivate() d'un gardien peut non seulement retourner un booléen mais aussi un objet de type **UrlTree** pour effectuer une redirection vers un composant explicatif de type "**NotAuthorizedComponent**" .
- Au sein des anciennes versions d'angular (ex : v6, v10, ...) , un gardien était codé comme une classe implémentant une interface (ex : implements CanActivate)
- Au sein des versions récentes d'angular (ex : v17, v21, ...) , les gardiens sont maintenant codés comme des fonctions légères .

2.2. Gardien moderne

auth-gard.gard.ts

```
import { CanActivateFn, createUrlTreeFromSnapshot } from '@angular/router';
import { of } from 'rxjs';

export const authGardGuard: CanActivateFn = (route, state) => {
  let token = sessionStorage.getItem("authToken");
  console.log("authGardGuard with token="+token);
  if(token!=null && token!= "" && token != "null") return true;
  //else return false;
  else return of(createUrlTreeFromSnapshot(route, ['./ngr-login']));
  //ou bien './ngr-not-authorized'
};
```

avec par exemple *not-authorized.component.html*

```
<h3>not-authorized</h3>
<p>login obligatoire pour accéder à la partie demandée</p>
<a routerLink="/login">login</a>
```

et *app-routing.module.ts* ou *app.routes.ts* comportant

```
{ path : "ngr-not-authorized" , component : NotAuthorizedComponent },
```

NB : Au sein des version récentes d'angular ,

ng g guard auth

génère un gardien codé sous forme de fonction fléchée plutôt que sous forme d'une classe implémentant l'interface CanActivate.

Un code adapté pourra par exemple utiliser la fonction **createUrlTreeFromSnapshot** de manière à exprimer une route de redirection de manière relative à la route d'origine .

2.3. Eventuelles données accompagnant un gardien

dans app-routing.module.ts

```
...  
{  
  path: 'super-user-stuff',  
  component: SuperUserStuffComponent,  
  canActivate: RoleGuard,  
  data: {roles: ['SuperAdmin', ...]}  
}
```

dans RoleGuard.ts :

```
...  
canActivate(route: ActivatedRouteSnapshot, state: RouterStateSnapshot)  
  : Observable<boolean> | Promise<boolean> | boolean | UrlTree {  
  let roles = route.data.roles as Array<string>;  
  ...  
}
```

3. router-outlet annexe (complexe)

A un niveau donné (principal ou "child") , il peut y avoir des <router-outlet> annexes (avec des noms précis) .

Ceci permettra (suite à une navigation) , de switcher à la fois :

- le composant principal
- un ou plusieurs(s) panneau(x) annexe(s) (exemple : menu latéral ou autre).

Exemple (dans app.component.html):

```
<div class="row">
  <div class="col-sm-3">
    <router-outlet name="aside"></router-outlet> <!-- secondary outlet with name -->
  </div>
  <div class="col-sm-9">
    <router-outlet></router-outlet>
    <!-- main outlet without specified name , default_name="primary" -->
  </div>
</div>
```

Dans **app-routing.module.ts**

```
const routes: Routes = [

  { path: 'asidePanelA', component: AsidePanelAComponent , outlet : 'aside'},
  { path: 'asidePanelB', component: AsidePanelBComponent , outlet : 'aside'},

  { path: 'ngr/welcome', component: WelcomeComponent },
  { path: '', redirectTo: '/ngr/welcome(aside:asidePanelA)', pathMatch: 'full'},
  . . .
```

URL complète pour naviguer (dans barre d'url):

http://localhost:4200/**ngr/welcome(aside:asidePanelA)**

http://localhost:4200/**ngr/welcome(aside:asidePanelB)**

Navigation via lien angular :

```
<a [routerLink]="[ { outlets: { primary : ['/ngr/welcome'] , aside : ['asidePanelB'] } } ]">
welcome</a>
```

Navigation via code typescript :

```
router.navigate([ { outlets: { primary : ['/ngr/welcome'] , aside : ['asidePanelB'] } } ])
```

(à éventuellement adapter) .

4. Resolver et autres gardiens

interfaces/rôles de gardiens	rôles/utilités
CanActivate	activation possible d'une route ?
CanActivateChildren	activation possible des "sub-routes" ?
CanDeactivate	pour forcer à ne pas bouger tant que modifications pas sauvegardées
Resolve	permet de différer l'activation d'une route tant que certaines données ne sont pas chargées
CanLoad	gardien pour modules asynchrones

4.1. Resolver

Un "resolver" est une **abstraction** et correspondant à un **petit objet technique** dont le rôle est **de préparer un chargement de données** avant de pouvoir les afficher au sein d'un composant.

Sans resolver :

Via une navigation angular , on arrive sur un composant qui affiche une liste d'élément vide au départ . La méthode ngOnInit() du composant ensuite à un service de récupérer certaines données et celles-ci seront affichées en différé (après téléchargement et éventuelles consolidations et autres post-traitements) .

Avec resolver :

La route est associé à un "resolver" qui va :

- déléguer le chargement des données à un service (via HttpClient)
- effectuer éventuellement certains post-traitements (restructuration des données , appels secondaires , ...)
- bloquer l'activation de la route tant que tout n'est pas prêt .

Ainsi , lorsque que le "resolver" aura fini son travail, les données récupérées/préparées seront alors associées à la route activée et le composant associé à cette route aura juste besoin de les afficher.

Ancien Exemple (à l'époque où les *resolver* étaient codés comme des classes) :

```
import { Resolve, ActivatedRouteSnapshot } from '@angular/router';
...
@Injectable({
  providedIn: 'root'
})
export class NewsResolver implements Resolve<Publication[]> {
  constructor(private _publicationService : PublicationService) {}
  resolve(): Observable<Publication[]> {
```

```

console.log("via NewsResolver ...");
return this._publicationService.getListePublicationObservable$();
}
}

```

Resolver moderne codé sous forme de fonction fléchée légère :

```

import { ActivatedRouteSnapshot, ResolveFn, RouterStateSnapshot } from '@angular/router';
...
export const newsResolver : ResolveFn< Publication[] > =
(route: ActivatedRouteSnapshot, state: RouterStateSnapshot) => {
  const publicationService = inject(PublicationService);
  console.log("via NewsResolver ...");
  return publicationService.getListePublicationObservable$();
}

```

dans *app.routes.ts* :

```

...
{ path: 'ngr/news', component: NewsComponent , resolve: { publications: newsResolver } } , ...

```

dans *news.components.ts* :

```

...
activatedRoute = inject(ActivatedRoute);

ngOnInit() {
  this.activatedRoute.data.subscribe({
    next : ( data ) => {
      const newsResolverData = data as { publications: Publication[] }
      this.tabPublicationNews=newsResolverData.publications; //...
    } ,
    error : (error) => { console.log( " error : " + error ) ; }
  });
}
...

```

Limitation des resolvers :

Un "resolver" va permettre d'attendre le téléchargement des données JSON (depuis une Api REST) pour celles-ci soient bien prêtes lorsqu'un composant en aura besoin.

Ceci dit , si ces données JSON comportent des références (url relatives) vers de gros fichiers "images" ou "vidéos" , ces fichiers ne seront traités (téléchargés et affichés progressivement) qu'au niveau du composant angular (après l'action du resolver et sans aucun lien avec celui-ci) .

X - Tests "angular" (unitaires & e2e)

1. Différent types de tests autour de angular

1.1. Vue d'ensemble / différents types et technologies de tests

Rappel du contexte: une application "Angular2+" correspond avant tout à la partie "Interface graphique" d'une architecture n-tiers et s'exécute au sein d'un navigateur web avec interprétation d'un code "typescript" transformé en "javascript".

Les principales fonctionnalités à tester sont les suivantes :

- **test unitaire d'un service** (avec éventuel "mock" sur accès aux données)
- **test unitaire d'un composant graphique** (avec éventuel mock sur "service")
- **test d'intégration complet (end to end)** englobant un dialogue HTTP/ajax/XHR avec des web services REST en arrière plan et sans avoir à connaître la structure interne de l'application (vue comme un boîte noire , vue de la même façon que depuis l'utilisateur final).

Pour tester du code javascript , la technologie de référence est "**jasmine**". Avec quelques extensions pour Angular2+, cette technologie pourrait suffire à mettre en place des tests unitaires simples .

Etant donné que l'on souhaite également tester unitairement des composants graphiques (en javascript) qui s'exécutent dans un navigateur, on a également besoin d'une technologie de test qui puisse interagir avec un navigateur (chrome, firefox, ...) et c'est là qu'intervient ou qu'intervenait "**karma**".

Depuis la version Angular 21 les tests unitaires sont par défaut lancer via "**vitest**" qui émule/simule le comportement de l'api DOM d'un navigateur .

Pour effectuer des tests globaux en mode "**end-to-end**" / "**boîte noire**" , on pouvait utiliser jusqu'à la version 11 ou 12 d'angular la technologie spécifique "**protractor**" qui permettait d'intégrer ensemble "**selenium**" et "**angular**" à travers des tests faciles à lancer.

A partir de Angular 12 ou 13, la technologie protractor n'est plus intégrée dans Angular et il est conseillé d'utiliser la technologie **cypress** (encore plus simple) à la place .

La technologie "**Playwright**" de Microsoft est également préconisée pour des tests e2e .

Cypress est plus simple que Playwright et Playwright est plus perfectionné et plutôt adapté pour des tests complexes/avancés .

2. Test "end-to-end / cypress"

2.1. Présentation de cypress

Cypress est une technologie JavaScript un peu plus moderne que Selenium permettant de

déclencher des tests "end-to-end".

Les principales différences entre Cypress et Selenium sont que :

- Selenium fonctionne au-dehors d'un navigateur et les ordres sont donnés au navigateur via l'intermédiaire d'un WebDriver
- Cypress fonctionne directement en JavaScript dans un navigateur : il est donc beaucoup plus rapide et peut beaucoup plus facilement interagir avec l'arbre DOM et XHR / Ajax

Selenium a cependant été utilisé massivement par un très grand nombre de testeurs. Selenium reste une RÉFÉRENCE incontournable.

Le "Test Runner" Cypress est open-source et fourni sous licence MIT

2.2. Installation de cypress

```
npm install --save-dev cypress
```

ou bien tout simplement `npm install` si cypress est déjà présent dans le fichier `package.json`

Attention : Prévoir de longues minutes de téléchargement ...

Eventuels réglages pour éviter un conflit avec angular :

Au sein d'un projet angular , il faut ajouter ceci à la fin du fichier `tsconfig.json` de manière à ce que l'installation de cypress ne perturbe pas les tests unitaires d'angular :

```
{
  ...
  "compilerOptions": {
    ....
  },
  "angularCompilerOptions": {
    ...
  },
  "exclude": [
    "cypress.config.ts",
    "cypress/**/*.ts"
  ]
}
```

2.3. Lancement de cypress

```
npx cypress open
```

Remarque : Le premier lancement va permettre la création de `cypress.json` , du sous-répertoire cypress et de certains sous-répertoires avec quelques exemples

Après un lancement de `cypress open` , il faut en théorie choisir un test à lancer au sein de la fenêtre de sélection. Nous devons cependant écrire préalablement notre propre test. À arrêter (en fermant cette fenêtre) et à relancer ultérieurement.

2.4. Écriture d'un nouveau test Cypress

Au sein du répertoire `.../cypress/integration` ou `.../cypress/e2e` selon version ,
créer le nouveau fichier `myTest.spec.js` ou bien `myTest.spec.cy.js` (*selon version*)
comportant un code de ce genre (à adapter) :

```
//NB: il faut préalablement lancer ng-serve ou autre

describe('My Angular Tests', () => {
  it('good conversion', () => {

    //partir de index.html
    cy.visit("http://localhost:4200/index.html")

    //cliquer sur le lien comportant 'basic'
    cy.contains('basic').click()
    cy.wait(50)
    // Should be on a new URL which includes 'basic'
    cy.url().should('include', 'basic')

    cy.get('a[href="/ngr-basic/calculatrice/simple"]').click()

    // Get an input, type data into it
    //and verify that the value has been updated
    cy.get('input[name="a"]')
    .clear()
    .type('9')
    .should('have.value', '9')

    cy.get('input[name="b"]')
    .clear()
    .type('6')
    .should('have.value', '6')

    //declencher click sur bouton soustraction
    cy.get('input[type="button"][value="-"]')
    .click()

    //vérifier que la zone d'id spanRes comporte le texte '3'
    cy.get('#spanRes')
    .should('have.text', '3')

  })
})
```

2.5. Lancement d'un test Cypress

1. Lancer d'abord l'application web à tester (via *ng serve* ou *lite-server* ou autre)
et d'éventuels "backends" en arrière plan.
2. `npmx cypress open` puis sélectionner le test à lancer

ou bien

```
npx cypress run --spec "cypress/e2e/myTest.spec.cy.js" --browser firefox >test_report.txt
```

Rapport test_report.txt généré : ...

```
| Cypress: 12.2.0
| Browser: Firefox 108 (headless)
| Specs: 1 found (myTest.spec.cy.js)
| Searched: my-app\cypress\e2e\myTest.spec.cy.js
Running: myTest.spec.cy.js
My Angular Tests
  ✓ good conversion (1519ms)

1 passing (3s)
```

NB : pour éviter un conflit entre les tests cypress et les tests unitaires d'angular, il faut ajouter ceci en bas de **tsconfig.json** :

```
,
  "exclude": [
    "cypress.config.ts",
    "cypress/**/*.ts"
  ]
```

3. Tests "end-to-end / playwright"

3.1. Présentation de playwright

Playwright est une technologie de test "e2e" créée par *Microsoft* à partir de 2020.

Un peu plus récente que cypress (de 2017) et beaucoup plus récente que Selenium (de 2004) , la technologie Playwright se démarque de "cypress" en fonctionnant au dehors d'un navigateur.

Inconvénient : moins d'interaction fine / visualisation

Avantage : plus de possibilités (ex : tests complémentaires avec différents navigateurs) .

D'un point de vue performances , playwright est un peu plus léger et rapide que "cypress".
Les fonctionnalités essentielles sont néanmoins très similaires.

3.2. Installation de playwright

```
npm init playwright@latest
```

cette commande interactive va soit créer un nouveau projet avec playwright soit ajouter playwright à un projet existant (par exemple angular) .

Il faudra choisir un répertoire pour les tests de playwright (ex : playwright/e2e ou autre) et répondre "y" à la question "install playwright browsers" .

3.3. Lancement de playwright

```
npx playwright test
```

et pour visualiser le rapport de tests :

```
npx playwright show-report
```

Site officiel de playwright : <https://playwright.dev>

3.4. Écriture d'un nouveau test Cypress

Au sein du répertoire choisi lors de l'installation de playwright (ex : *playwright/e2e*) , créer un nouveau fichier *myTest.spec.ts* comportant un code de ce genre (à adapter) :

```
//NB: il faut préalablement lancer ng-serve ou autre

import { test, expect } from '@playwright/test';

test('good soustraction', async ({page}) => {

    await page.goto("http://localhost:4200/ng-basic");

    await page.fill('input[name="a"]','9');
    await page.fill('input[name="b"]','6');

    await page.click('input[type="button"][value="-"]');

    await expect(page.locator("#spanRes")).toHaveText("3");

})
```

4. Attention à la cohérence des tests

Les commandes d'angular-cli (ng new , ng g component , ...) génèrent par défaut des fichiers de tests pour chaque composant et chaque service .

Si on ne s'intéresse jamais aux tests (on ne les code pas, on ne les lance pas) , des fichiers *xyz.spec.ts* inutilisés (et souvent devenus incohérents par rapport au code des composants) ne sont que des fichiers inutiles .

Si par contre, on tient à lancer quelques tests unitaires, il vaut mieux que ceux-ci soient maintenus de manière cohérente avec le code , sinon --> erreurs de tous les cotés (manque import , ...) .

Coder et maintenir à jour un fichier *xyz.spec.ts* représente un gros travail (il faut y passer du temps).

Conséquence :

- il vaut mieux développer sans test , qu'avec des tests incohérents et plein de bugs .
- ne générer les fichiers *xyz.spec.ts* que si on les code et on les maintien à jour .
- une restructuration (*refactoring*) d'une application angular peut conduire à beaucoup de tests à retoucher pour que tout reste globalement cohérent (bon courage!!!) .

NB : Pour générer un nouveau composant *xyz* sans test unitaire au départ on peut utiliser l'option **--skipTests=true** de **ng g component xyz** .

Et l'on pourra ajouter le fichier *xyz.spec.ts* ultérieurement si besoin

5. Tests unitaires élémentaires

5.1. Tests unitaires en javascript

Pour écrire un test unitaire en javascript , les 4 technologies les plus utilisées sont :

- **jasmine**
- **mocha + chai**
- **jest** (*utilisé avec react*)
- **vitest**

Angular a longtemps fait le choix d'utiliser **jasmine** .

Depuis la version **21** , angular utilise par défaut **vitest** .

<i>package.json</i> (avant version 21)	<i>package.json</i> (angular 21+)
<pre>... "devDependencies": { ... "@types/jasmine": "~4.0.0", "jasmine-core": "~4.1.0", "karma": "~6.3.0", "karma-coverage": "~2.2.0", "karma-chrome-launcher": "~3.1.0", "karma-jasmine": "~5.0.0", "karma-jasmine-html-reporter": "^1.7.0",...</pre>	<pre>"devDependencies": { "@angular/build": "^21.1.0", "@angular/cli": "^21.1.0", ... "typescript": "~5.9.2", "vitest": "^4.0.8" }</pre>

Les **assertions** de *jasmine* ou de *vitest* s'expriment via **expect(...).to...**

(BDD syntax : Behavior Driven Development) :

```
expect(object_or_var)
.toEqual(expected) //deep equality for objects and arrays
.toContain(val) //check if string or array contains specific item or substring
.toBe(value) //exact equality
.toBeCloseTo(value,nbDecimale) //test de valeur proche pour nombre à virgule flottante
.toBeDefined()
.toBeNull()
.toBeTruthy()/toBeFalsy()
.toHaveBeenCalled()
...
```

5.2. Test unitaire élémentaire (jasmine)

basicTest.spec.ts

```
describe('premiers tests', () => {
  it('1+1=2', () => expect(1+1).toBe(2));
  it('2+2=4', () => expect(2+2).toBe(4));
});
```

Ces spécifications de tests au format "**jasmine**" correspondent à un fichier d'extension ".spec.ts" (ou ".spec.js") et chaque partie "`() => expect(...).toBe(...)`" correspond à une assertion à vérifier.

Lancement d'un test unitaire jasmine (sans angular) :

```
npm install -g jasmine (si nécessaire)
tsc --skipLibCheck
jasmine dist\out-tsc\src\app\basicTest.spec.js
```

--> affiche (en cas de succès) :

```
Started
..
2 specs, 0 failures
Finished in 0.014 seconds
```

ou bien (ici en cas d'échec volontaire via `expect(2+2).toBe(6)`) :

```
Failures:
1) premiers tests 2+2=4
  Message:
    Expected 4 to be 6.
  Stack:
    Error: Expected 4 to be 6.
      at <Jasmine>
      at UserContext.it (D:\tp\local-git-mycontrib-repositories\tp_angular8+\basic-app\dist\out-
tsc\src\app\basicTest.spec.js:3:37)
      at <Jasmine>
2 specs, 1 failure
Finished in 0.013 seconds
```

5.3. Test unitaire élémentaire (vitest)

sum.ts

```
export function sum(a:number, b:number) :number{
  return a + b
}
```

basicVitest.spec.ts

```
import { expect, test, describe, it } from 'vitest'
import { sum } from './sum'

describe('premiers tests', () => {
  test('1+1=2', () => { expect(1+1).toBe(2) });
  it('2+2=4', () => { expect(2+2).toBe(4) });
  it('sum(1,2)=3', () => { expect(sum(1, 2)).toBe(3) });
});
```

NB : Avec vitest la fonction test() est utilisable à la place de it() , à priori it = alias de test (comme avec jest) .

npm install -g vitest

vitest src/.../basicVitest.spec.ts

```

✓ src/app/basic/calculatrice/basicVitest.spec.ts (3 tests) 2ms
✓ premiers tests (3)
  ✓ 1+1=2 1ms
  ✓ 2+2=4 0ms
  ✓ sum(1,2)=3 0ms

Test Files  1 passed (1)
Tests       3 passed (3)
Start at    10:18:34
Duration    35ms

PASS Waiting for file changes...
press h to show help, press q to quit

```

5.4. Tests structurés (avec jasmine ou vitest) :

```

describe('tests structures avec jasmine', () => {
  let s : string;
  let x,y,z ;

  beforeAll(()=>{
    console.log("beforeAll called once");  s="abc";
  });

  beforeEach(()=>{
    console.log("beforeEach called again");  x=1;y=2;
  });

  describe('tests de calcul', () => {
    it('1+2=3', () => expect(x+y).toBe(3));
    it('1*2=2', () => expect(x*y).toBe(2));
  });

  describe('autres tests', () => {
    it('s=abc', () => expect(s).toBe('abc'));
    it('s comporte ab', () => expect(s).toContain('ab'));
  });

  afterEach(()=>{  console.log("afterEach called again"); });

  afterAll(()=>{  console.log("afterAll called once"); });

});

```

NB :

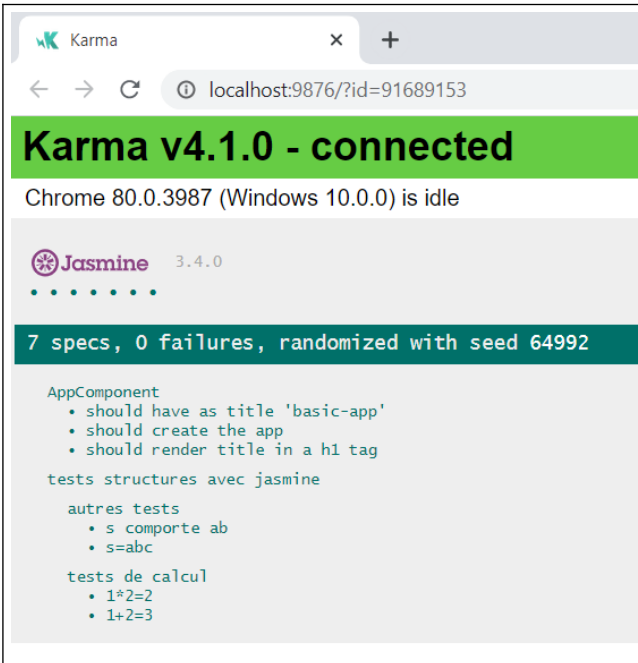
- **beforeEach()** et **afterEach()** sont appelées avant et après l'exécution de chaque **it()** *pour les describe() de même niveau.*
- Souvent , un niveau de **describe()** correspond à un **objet/composant/sujet à tester** et la fonction **it()** signifie **"it should behave like that"** .

6. Lancement tests "Angular" avec ng test

ng test

Cette commande (de angular CLI) permet de lancer tous les tests unitaires "angular" (fichiers `xyz.spec.ts`) via jasmine+karma ou via vitest (de façon à interagir avec un navigateur émulé ou pas).

Les résultats des tests déclenchés s'affichent de cette façon avec "karma+jasmine" ou "vitest" :



```

✓ my-app src/app/basic/tva/tva.component.spec.ts (3 tests) 137ms
✓ TvaComponent (3)
  ✓ should create 61ms
  ✓ tva(200,20)=40 from model 40ms
  ✓ tva(200,10)=220 from IHM 35ms

Test Files 1 passed (1)
Tests 3 passed (3)
Start at 10:29:23
Duration 5.00s (transform 77ms, setup 432ms, import 79ms, tests 137ms)

PASS Waiting for file changes...
press h to show help, press q to quit

```

avec vitest (depuis angular 21)

Par défaut, **ng test** détecte tous les tests unitaires (`.spec.ts`) et les lance en mode « **watch** ».

Chaque modification enregistrées d'un code source provoque une ré-exécution des tests.

On peut compléter la commande "ng test" avec des options, comme :

- n'exécuter le test qu'une fois (`--watch=false`)
- générer un rapport de couverture de test (`--code-coverage`)

Exemple :

```
ng test --watch=false
```

et

```
ng test --watch=false --include=**/service/*.spec.ts
```

pour ne lancer que les tests unitaires sur les services

7. Tests unitaires "angular"(composants, service, ...)

7.1. Rare test unitaire isolé (sans extension Angular)

Exemple :

Service élémentaire à tester (calcul de tva):

compute.service.ts

```
import { Injectable } from '@angular/core';
@Injectable()
export class ComputeService {
  tva(ht:number ,taux_tva_pct:number){
    return ht * taux_tva_pct/100.0
  }
}
```

Test unitaire :

compute.service.spec.ts

```
import { ComputeService } from './compute.service';
// Isolated unit test = Straight Jasmine test
// no imports from Angular test libraries
describe('ComputeService without the TestBed', () => {
  let service: ComputeService;

  beforeEach(() => { service = new ComputeService(); });

  it('20%tva sur 200 ht = 40', () => {
    expect(service.tva(200,20)).toBe(40);
  });
});
```

ng test -->

0 failures

ComputeService without the TestBed
• 20%tva sur 200 ht = 40

NB : Ce service de calcul à tester étant extrêmement simple, l'instance à tester a pu être créée par un simple new . Ce cas est très rare. Bien souvent , les services à tester ont des dépendances vis à vis d'autres services ou éléments du framework angular.

7.2. Test de service simple avec TestBed

Le test précédent peut être ré-écrit en s'appuyant sur "*TestBed*". Ceci permet d'obtenir des instances d'éléments à tester bien initialisés par le framework angular et ses mécanismes d'injection de dépendances :

compute.service.spec.ts

```
import { TestBed } from '@angular/core/testing';

import { ComputeService } from './compute.service';

describe('ComputeService', () => {
  beforeEach(() => TestBed.configureTestingModule({
    /* providers: [ ComputeService ] already provided in root via @Injectable() */
  }));

  it('should be created', () => {
    const service: ComputeService = TestBed.inject(ComputeService);
    //NB: avant angular 9 , TestBed.get(ComputeService) ;
    //était moins bien que TestBed.inject() car le type de retour était any
    expect(service).toBeTruthy();
  });

  it('20%tva sur 200 ht = 40', () => {
    const service: ComputeService = TestBed.get(ComputeService);
    expect(service.tva(200,20)).toBe(40);
  });
});
```

7.3. Test unitaire de composant angular avec TestBed

Exemple :

calculatrice.component.ts

```
import { Component, OnInit } from '@angular/core';
import { ActivatedRoute, Params } from '@angular/router';

@Component({
  selector: 'app-calculatrice', imports: [FormsModule],
  templateUrl: './calculatrice.component.html',
  styleUrls: ['./calculatrice.component.css']
})
export class CalculatriceComponent implements OnInit {

  modeChoisi : string = "simple"; //"simple" ou "sophistiquee"

  a : number = 0;
  b : number = 0;
  res : number = 0;

  onCalculer(op:string){
    switch(op){
      case "+":
        this.res = Number(this.a) + Number(this.b); break;
      case "-":
        this.res = Number(this.a)- Number(this.b); break;
      case "*":
        this.res = Number(this.a) * Number(this.b); break;
      default:
        this.res = 0;
    }
  }

  constructor(route : ActivatedRoute) {
    route.params.subscribe(
      (params: Params)=> {
        this.modeChoisi = params['mode'];
      }
    )
    //NB: params['mode'] car { path: 'calculatrice/:mode', ... },
  }

  ngOnInit(): void {
  }
}
```

calculatrice.component.html

```
<div class="c1">
  <h3>calculatrice angular</h3>

  <label>a </label> <input type="number" name="a" [(ngModel)]="a" > <br>
  <label>b </label> <input type="number" name="b" [(ngModel)]="b" > <br>
  <label>operation </label>
    <input type="button" value="+" (click)="onCalculer('+')" > &nbsp;
    <input type="button" value="-" (click)="onCalculer('-')" > &nbsp;
    <input type="button" value="*" (click)="onCalculer('*')"
      [style.visibility]="modeChoisi=='sophistiquee'? 'visible': 'hidden'" >
  <br>
  <label>resultat:</label>
  <span [style.font-weight]="res>0?'bold':'normal'"
    [class.negatif]="res<0" id="spanRes">{{res}}</span> <br>
</div>
```

Exemple de test (récent): `calculatrice.component.spec.ts`

```
// à lancer via ng test --include=**/calculatrice/*.spec.ts
```

code initial généré par l'assistant *ng generate component* :

```
import { ComponentFixture, TestBed } from '@angular/core/testing';
import { CalculatriceComponent } from './calculatrice.component';

describe('CalculatriceComponent', () => {
  let component: CalculatriceComponent;
  let fixture: ComponentFixture<CalculatriceComponent>;

  beforeEach(async () => {
    await TestBed.configureTestingModule({
      imports: [CalculatriceComponent]
    })
    .compileComponents(); // compile asynchronously template and css , return promise

    fixture = TestBed.createComponent(CalculatriceComponent);
    component = fixture.componentInstance;
    await fixture.whenStable();
  });

  it('should create', () => {
    expect(component).toBeTruthy();
  });
})
```

La librairie `@angular/core/testing` comporte entre autres la classe fondamentale **TestBed** qui permet de **créer un module/environnement de test** (un peu comme `@NgModule` ou `@Component`) à partir de la méthode **configureTestingModule()**.

Il est conseillé d'appeler cette méthode à l'intérieur de `beforeEach()` de façon à ce que chaque test soit indépendant des autres (avec un contexte ré-initialisé).

Code enrichi tenant compte du paramètre **ActivatedRoute** du constructeur du composant :

```
import { of } from 'rxjs'
import { ActivatedRoute } from '@angular/router';
...
beforeEach(async () => {
  await TestBed.configureTestingModule({
    imports: [CalculatriceComponent],
    providers: [ {
      provide: ActivatedRoute, useValue: { params: of([mode: 'simple']) },
    } ]
  }).compileComponents();
  ...
})
```

Ceci permet de spécifier ici la valeur 'simple' pour le paramètre mode de `ActivatedRoute`.

Seulement après une bonne et définitive configuration, la méthode **TestBed.createComponent()** permet de créer une chose technique **fixture** de type **ComponentFixture<ComponentType>** sur laquelle on peut invoquer :

.componentInstance de façon à accéder à l'instance du composant à tester (variable *component*)

.debugElement.nativeElement de façon à accéder au nœud d'un arbre DOM lié au template du composant à tester.

test intermédiaire (en grande partie basé sur une manipulation directe du modèle) :

```
it('5+6=11 from model', async () => {
  component.a=5;
  component.b=6;
  component.onCalculer('+');//à ne pas oublier d'appeler si pas de dispatchEvent
  fixture.changeDetectorRef.markForCheck(); //REQUIRED in ZoneLess mode !!!

  //fixture.detectChanges(); //not OK for ZoneLess mode
  await fixture.whenStable(); //OK with async for ZoneLess mode
  expect(component.res).toBe(11); //test computed value in model
  const compNativeElt = fixture.debugElement.nativeElement;
  let spanResElt = compNativeElt.querySelector('#spanRes');
  console.log("from model, 5+6, res:" + spanResElt.textContent);
  //spanResElt.innerText avec jasmine+karma, spanResElt.textContent avec vitest
  expect(spanResElt.textContent).toContain('11');
});
```

NB :

await fixture.whenStable();

est souvent nécessaire pour laisser le temps aux mécanismes d'angular de mettre à jour les éléments de la vue (arbre DOM) de manière bien synchronisée par rapport à l'évolution du modèle (nouvelles valeurs calculées, ...).

Depuis angular 21 (avec le mode "zoneLess" par défaut), il faut quelquefois appeler explicitement *fixture.changeDetectorRef.markForCheck()* (surtout en l'absence de signaux).

test unitaire basé sur des interactions et affichages "DOM" :

```
it('10-3=7 from IHM', async () => {
  const compNativeElt = fixture.debugElement.nativeElement;
  let aInputElt = compNativeElt.querySelector("input[name='a']");
  aInputElt.value=10;
  aInputElt.dispatchEvent(new Event('input'));

  let bInputElt = compNativeElt.querySelector("input[name='b']");
  bInputElt.value=3;
  bInputElt.dispatchEvent(new Event('input'));
```

```

let moinsButtonElt =
  compNativeElt.querySelector("input[type='button'][value='-']");
//moinsButtonElt.dispatchEvent(new Event('click'));
moinsButtonElt.click();

await fixture.whenStable(); //OK with async for ZoneLess mode

let spanResElt = compNativeElt.querySelector('#spanRes');
console.log("from IHM, 10-3, res:" + spanResElt.textContent);
expect(spanResElt.textContent).toContain('7');
});

```

NB: `moinsButtonElt.click();` est équivalent à `moinsButtonElt.dispatchEvent(new Event('click'));`
Cela permet de déclencher un événement "DOM" (interaction simulée de l'utilisateur) .

Variantes (pour anciennes versions d'angular) :

pour l'époque "avec NgModule" (pas standalone) :

```

...
 TestBed.configureTestingModule({
   imports: [ FormsModule , AppRoutingModuleModule] ,
   declarations: [ CalculatriceComponent ]
 })
...

it('10-3=7 from IHM', () => {
  ...
  fixture.detectChanges();
  ...
  expect(spanResElt.innerText).toContain('11');
});
...

```

NB : la méthode `fixture.detectChanges()` permet d'explicitement (re)synchroniser la vue HTML en fonction des changements effectués au niveau du modèle.

L'équivalent plus moderne `await fixture.whenStable();` fonctionne mieux (surtout en mode ZoneLess) et nécessite l'usage conjoint du mot clef `async` :

```

it('-----', async () => {
  ...
  await fixture.whenStable();
  ...
});

```

7.4. Tester de composant avec mock de service

Lorsqu'un composant angular s'appuie sur un service injecté par constructeur ou via inject on a

- soit la possibilité de tester "composant+service_injecté"
- soit la possibilité de tester plus unitairement le composant en simulant le comportement du service injecté

Par exemple si le composant *TvaWithServiceComponent* utilise un service de type *TvaService* on a alors ces deux variantes possibles :

```
//utilisation directe (souvent par défaut) du réel service de calcul :
 TestBed.configureTestingModule({ ...
    providers:    [ TvaService ] ,
...} ;)
```

```
beforeEach(async () => {
    //stub Service for test purposes (will be cloned and injected)
    let tvaServiceStub = {
        tva(ht : number, tauxTvaPct : number ) : number{
            return ht * tauxTvaPct / 100;
        }
    };

    await TestBed.configureTestingModule({...
        providers: [ {provide : TvaService,
                       useValue : tvaServiceStub } ],
    }).compileComponents();
```

Il est en général conseillé de demander à injecter un service de type "stub" ou "mock" (simulant le comportement du réel service et permettant de se focaliser sur le composant angular à tester)

7.5. Spy et mock avec vitest

Rappel : depuis angular 21 , les tests unitaires sont interprétés par vitest (et plus par jasmine).

Exemple :

calculator.ts

```
export class Calculator {
    add(a: number, b: number) { return a + b; }
    ...}
```

calculator.spec.ts

```
import {describe, it, expect, vi} from 'vitest';
import {Calculator} from "../calculator";

describe("Vitest spy/mock Fundamentals", () => {

    it("shows how Vitest spies work", () => {
        const calculator = new Calculator();
        const spyAdd = vi.spyOn(calculator, "add");
        const resultAdd = calculator.add(2, 3);
```

```

expect(resultAdd).toBe(5); //original/same behavior
expect(spyAdd).toHaveBeenCalledOnce();
expect(spyAdd).toHaveBeenCalledWith(2, 3);
const resultAdd2 = calculator.add(3, 3);
expect(spyAdd).toHaveBeenCalledTimes(2);
expect(spyAdd).toHaveBeenCalled();
spyAdd.mockClear(); //only reset recording calls
const resultAdd3 = calculator.add(4, 5);
expect(spyAdd).toHaveBeenCalledOnce();
})

it("shows how Vitest mocking works", () => {
  const calculator = new Calculator();
  const mockAdd = vi.spyOn(calculator, "add").mockReturnValue(5);
  // the result is always 5, due to mocking :
  const result2a = calculator.add(5, 5);
  expect(mockAdd).toHaveBeenCalled();
  expect(mockAdd).toHaveBeenCalledWith(5, 5);
  expect(result2a).toBe(5);
  mockAdd.mockImplementation((a:number,b:number)=>a-b)
  const result2b = calculator.add(8, 5);
  expect(result2b).toBe(3); //8-5=3
  mockAdd.mockReset(); //clear history of called and restore original/real code
  //mockAdd.mockRestore(); //clearReset() AND disconnect/deactivate link between calculator objet and mockAdd
  //mockXyz.mockRestore() or vi.resetAllMocks() is often called in beforeEach

  const result3 = calculator.add(5, 5);
  expect(result3).toBe(10);
  expect(mockAdd).toHaveBeenCalled(); //cannot be called after mockAdd.mockRestore();
})
})

```

7.6. partial and asymmetric matchers

```
import { expect, test, vi } from 'vitest'
```

```

test('the shopping list has milk in it', () => {
  const shoppingList = ['milk', 'bread', 'eggs', 'butter']
  expect(shoppingList).toContain('milk')
  expect(shoppingList).toEqual(expect.arrayContaining(['eggs','milk']))//containing sublist any order
})

```

```

test('partial object matcher', () => {

const user = {
  id: 1,
  firstname: 'jean',
  lastname : 'Bon',
  email: 'jean.bon@xyz.com',
}

```

```
// We only care about lastname and email here
expect(user).toMatchObject({
  lastname: 'Bon',
  email: 'jean.bon@xyz.com',
});
```

```
test('there is no I in team', () => {
  expect('team').not.toMatch(/I/) //regexp
})
```

```
test('object has property', () => {
  const user = {
    name: 'Alice',
    address: { city: 'Paris', zip: '75001' }
  }

  expect(user).toHaveProperty('name')
  expect(user).toHaveProperty('name', 'Alice')
  expect(user).toHaveProperty('address.city', 'Paris')
  expect(user).toHaveProperty('address.zip')
})
```

```
function createUser(){
  return {
    id: 2,
    firstname: 'alex',
    lastname : 'Therieur',
    email: 'alex.therieur@xyz.com',
    roles : ['user','admin'],
    address: { zip: '75001' , city : 'Paris'}
  };
}

test('user has the right shape via asymmetric matchers', () => {
  const user = createUser()

  expect(user).toEqual({
    id: expect.any(Number),
    firstname: expect.any(String),
    lastname: expect.stringMatching(/[A-Z].*/),
    email: expect.stringContaining('@'),
    roles: expect.arrayContaining(['admin']),
    address: expect.objectContaining({ zip: expect.any(String) , city : expect.any(String)})
  })
})
```

```
function divide(a:number,b:number) {
  if (b === 0) {
    throw new Error('Cannot divide by 0')
  }
}
```

```

    return a/b;
  }

  test('divide by zero throws exception', () => {
    // Check that it throws exception
    expect(() => divide(3,0)).toThrow()

    // Check the error message with regexp
    expect(() => divide(3,0)).toThrow(/divide by 0/)
  })

```

7.7. Valeurs en input/output au niveau des tests unitaires

```

fixture = TestBed.createComponent(HeaderComponent);
component = fixture.componentInstance;
fixture.componentRef.setInput('titre', 'my-ng-app');//sinon 'titre_par_defaut'

```

ou bien

```

fixture = TestBed.createComponent(HeaderComponent, {
  bindings: [
    inputBinding('titre', () => 'my-ng-app')
  ]
});

```

en mode "two way binding / model" :

```

let leChoix = signal('red');//valeur initiale

beforeEach(async () => {
  ...
  fixture = TestBed.createComponent(SimpleSelectorComponent, {
    bindings: [
      inputBinding('values', () => ['red', 'green', 'blue']),
      twoWayBinding('choix', leChoix),
    ]
  });
  ...
});

```

Vérification d'émission en output via spy on emit :

```

const outputChoixEmitSpy = vi.spyOn(component.choix, 'emit');
const select: HTMLSelectElement =
  fixture.debugElement.query(By.css('#selection')).nativeElement;
select.value = select.options[1].value; // <-- select second possible value (at index 1)
select.dispatchEvent(new Event('change'));
await fixture.whenStable();
expect(outputChoixEmitSpy).toHaveBeenCalledOnce();
expect(outputChoixEmitSpy).toHaveBeenCalledWith('green'); //expected output value = 'green'

```

XI - Déploiements avec ou sans SSR/SSG

...

1. Application isomorphe et optimisation SSR

Autrefois appelée "**angular-universal**" et maintenant rebaptisée "**ssr**" dans les versions 17+ , l'extension "**ssr**" pour angular permet d'**obtenir un démarrage plus rapide ainsi qu'un bien meilleur référencement naturel ou SEO (Search Engine Optimization)** .

La nouvelle version "**ssr**" est plus élaborée/performante et elle est basée sur de l'hydratation.

1.1. Objectif de "ssr"

Pour être visité par un grand nombre d'utilisateurs, un site web se doit de remplir deux conditions essentielles.

- La page d'accueil soit s'afficher le plus rapidement possible.
- L'application angular doit être bien référencé par les moteurs de recherche.

La technique qui permet de le faire porte un nom.

- Le **Rendu Côté Serveur** ou **Server Side Rendering (SSR)** en anglais.

Les moteurs de recherche ont à l'heure actuelle du mal à interpréter le javascript.

Par défaut une application angular est de type "SPA (Single Page Application)" et "CSR" (Client Side Rendering) . Le titre principal de la page d'accueil est caché dans un bundle javascript produit par "ng build" mais n'apparaît pas dans index.html .

--> On a donc besoin d'ajouter un mécanisme SSR pour obtenir un bon référencement .

On ne peut cependant pas remettre en cause tous la logique de fonctionnement "SPA/CSR" d'angular qui fonctionne très bien pour naviguer rapidement d'un sous composant à l'autre et qui offre un très bon comportement dynamique (réactions rapides).

"**Angular SSR**" est en fait une technologie dite d'**isomorphisme** :

Au lieu de n'utiliser qu'un rendu coté serveur , "Angular SSR" va servir à générer coté serveur une sorte de pré-rendu "HTML/CSS" immédiatement/rapidement affichable par un navigateur de la page principale , son entête , son pied de page et son sous composant d'accueil (ex : WelcomeComponent) .

Le "pré-rendu" SSR n'est qu'un point de départ de l'application (très rapidement téléchargé et affiché) . Les mécanismes "javascript/SPA/CSR" sont téléchargés et chargés en mémoire en tâche de fond (le temps que l'utilisateur passe à admirer le pré-rendu affiché) .

Par la suite, les mécanismes SPA/CSR prennent le relai et l'application angular fonctionne comme d'habitude en dynamique javascript coté client et sans SSR.

Depuis la version 16 d'angular , cette prise de relai a été beaucoup optimisée et elle passe par une "**Hydration**" "javascript/eventListener" des pages statiques préalablement pré-rendues.

Ce "pré-rendu" SSR est en tout point identique à celui qui serait générer en pur javascript par les mécanismes habituels CSR , on parle alors en terme d'**isomorphisme** : même forme prise par les rendus SSR et CSR .

1.2. Positionnement de angular-ssr

La technologie @angular/ssr est à la fois :

- liée au projet angular (add-on)
- vouée à être exécutée coté serveur via **nodeJs/express**

1.3. intégration de angular ssr

Au sein d'un projet existant angular , on lancera la commande suivante de façon à ajouter la fonctionnalité "ssr" :

ng add @nguniversal/express-engine *pour anciennes versions*

ng add @angular/ssr *depuis angular 17+*

Cette commande va ajouter certains fichiers et modifier certains fichiers existants de l'application angular. Il se peut que l'extension ssr soit déjà ajoutée dès la création de l'application (proposition de ng new).

1.4. 3 niveaux (SSG, SSR, CSR)

SSG (Static Site Generation)	Pré-rendu généré lors du build : utile pour les parties qui ne changent pas beaucoup/souvent (ex : partie principale de la page d'accueil) .
SSR (Server Side Rendering)	Rendu effectué dynamiquement du coté serveur (nodeJs et main.server.ts)
CSR (Client Side Rendereing)	Mode ordinaire/par défaut du fonctionnement d'angular coté client

NB: dans versions récentes d'angular , une fois l'extension @angular/ssr installée , le fichier **app.routes.server.ts** permet de préciser les routes qui seront prises en compte

- soit en mode **RenderMode.Client** (CSR)
- soit en mode **RenderMode.Server** (CSR)
- soit en mode **RenderMode.PreRender** (SSG)

Ce fichier permet également de préciser des valeurs de paramètres de routes pour les pré-rendus.

Par exemple , avec un environnement de production correct, on peut par exemple avoir un temps de réponse de l'ordre de 33ms en SSG et 100ms en SSR .

Par contre , en mode SSG , on aura pas de "new du jour" (dynamique , en SSR seulement) ni de statistique à jour ("nombre de ... du moment "), etc , etc .

Exemple de fichier *app.routes.server.ts* :

```
import { RenderMode, ServerRoute } from '@angular/ssr';

export const serverRoutes: ServerRoute[] = [
  {
    path: 'welcome',
    renderMode: RenderMode.Prerender
  },
  {
    path: 'aaa',
    renderMode: RenderMode.Prerender
  },
  {
    path: 'bbb',
    renderMode: RenderMode.Server
  },
  {
    path: '**', /* tout le reste en CSR */
    renderMode: RenderMode.Client
  }
];
```

1.5. Utilisation de Angular-SSR

npm run build (alias npm pour "*ng build* ")

génère la sous arborescence suivante :

```
dist/my-app/browser/index-csr.html et ....js  (partie habituellement générée par ng build)
dist/my-app/server/server.mjs  (partie supplémentaire générée en mode ssr)
```

En mode "extension ssr activée" , la fonctionnalité **"Static Site Generation" (SSG)** ou **"Prerendering"** génère directement (dans le répertoire *dist/my-app/browser*) des versions **".html sans javascript"** pour la page d'accueil et les principales routes "non lazy" .

Lancement direct de l'application angular générée en mode "ssr":

```
npm run serve:ssr:my-app
```

ou bien (directement sans un des alias de package.json) :

```
node dist/my-app/server/server.mjs
```

1.6. Pièges à éviter en mode SSR :

- Le rendu "SSR" (coté serveur "sans navigateur") ou bien SSG n'est pas capable d'utiliser les objets globaux qui n'existeront que plus tard au runtime dans votre navigateur comme *document*, *window*, *localStorage*, etc.

Fausse solution : désactiver "ssr" (ok que si on ne veut pas s'en servir).

On pourrait ajouter "ssr": *false*, "prerender": *false*

dans angular.json près des lignes 72,73

```
"development": {
  "optimization": false,
  "extractLicenses": false,
  "sourceMap": true,
  "ssr": false,
  "prerender": false
}
```

Bonne solution : Tester le contexte (CSR ou SSR) avant d'utiliser un objet global du navigateur (document ou localStorage, ...) via un bloc de code de ce genre :

```
import { Component, inject, PLATFORM_ID } from "@angular/core";
import { DOCUMENT, isPlatformBrowser, isPlatformServer } from "@angular/common";

export class XyzComponent {
  private readonly platform = inject(PLATFORM_ID);
  private readonly document = inject(DOCUMENT);

  constructor() {
    if (isPlatformBrowser(this.platform)) {
      console.warn("browser / csr");
      // Safe to use document, window, localStorage, etc. :-
    }

    if (isPlatformServer(this.platform)) {
      console.warn("server / ssr");
      // Not smart to use document here, however, we can inject it ;-
    }
  }
}
```

Points à principalement encadrer comme cela :

- `localStorage.setItem(...)` et `getItem(...)`
- appels automatiques de WS-REST en mode GET au sein de `ngOnInit()` ou autres

Ce besoin étant récurrent, autant le coder dans un sous service réutilisable :

```
import { Injectable } from '@angular/core';
import { inject, PLATFORM_ID } from "@angular/core";
import { isPlatformBrowser, isPlatformServer } from "@angular/common";

@Injectable({
  providedIn: 'root'
})
export class MyStorageUtilService {
  private readonly platform = inject(PLATFORM_ID);

  public setItemInLocalStorage(key:string, value: string | null | undefined){
    if (isPlatformBrowser(this.platform)) {
      localStorage.setItem(key,value?"");
    }
  }

  public setItemInSessionStorage(key:string, value: string | null | undefined){
    if (isPlatformBrowser(this.platform)) {
      sessionStorage.setItem(key,value?"");
    }
  }

  public getItemInLocalStorage(key:string):string|null {
    return isPlatformBrowser(this.platform)?localStorage.getItem(key):null
  }

  public getItemInSessionStorage(key:string):string|null {
    return isPlatformBrowser(this.platform)?sessionStorage.getItem(key):null
  }
  constructor() { }
}
```

Exemple d'utilisation :

preferences.service.ts

```
import { Injectable } from '@angular/core';
import { MyStorageUtilService } from './my-storage-util.service';

@Injectable({
  providedIn: 'root'
})
export class PreferencesService {
  readonly myStorageUtilService = inject(MyStorageUtilService);

  //public couleurFondPreferee :string = 'lightgrey'; //v1 : public
  private _couleurFondPreferee :string="?"; //v2 : private + get/set + localStorage

  public get couleurFondPreferee() {
    return this._couleurFondPreferee;
  }
}
```

```
public set couleurFondPreferee(c:string){
  this._couleurFondPreferee=c;
  //localStorage.setItem('preferences.couleurFond',c);
  this.myStorageUtilService.setItemInLocalStorage('preferences.couleurFond',c);
}

constructor() {
  //const c = localStorage.getItem('preferences.couleurFond');
  let c :string | null = this.myStorageUtilService.getItemInLocalStorage('preferences.couleurFond');
  this._couleurFondPreferee = c?c:'lightgrey';
}
}
```

1.7. Hydratation automatique

Hydratation = passage progressif d'une page statique vers page dynamique (structure DOM stable , javascript progressif).

Attention : pour que l'hydratation progressive se passe bien il faut que l'**arbre DOM soit bien structuré et stable** . Par exemple pour cela il faut bien respecter la structure complète d'un tableau HTML en explicitant les parties **<thead>** et **<tbody>** (quelquefois considérées comme implicites).

Fonctionnement de l'hydratation :

1. La première requête envoyée du navigateur au côté serveur (node + appli_angular_ssr) retourne très rapidement une page principale statique (par exemple constituée de index.html + composant principal + sous composants essentiels (header, menu, ..))
2. Le navigateur télécharge cette page statique déshydratée , créer un arbre DOM et affiche cela très rapidement
3. L'hydratation progressive consiste ensuite à charger le code "javascript" de l'application . Lorsque ce code va s'initialiser , ça va petit à petit hydrater l'application avec des variables en mémoire , des fonctions événementielles sans changer la structure principale de l'arbre DOM (id conservé , arborescence stable, ...)
4. l'application fonctionne ensuite normalement de manière pleinement dynamique en mode "CRS"

Perception délicate de l'hydratation :

Normalement, un bon isomorphisme ssr/csr et une bonne hydratation donne une impression de totale continuité (parfaitement transparente) et l'utilisateur ne voit pas du tout l'effet de l'hydratation.

D'ailleurs , le paramétrage provideClientHydration(**withEventReplay()**) de app.config.ts va jusqu'à mémoriser certaines interactions précoces de l'utilisateur (ex : click sur menu) pour les rejouer en très léger différé une fois que le code javascript aura été chargé pour gérer cela .

NB: Paramétrer (même temporairement) une éventuelle hydratation incrémentale peut être un moyen de visualiser un peu la phase d'hydratation généralement transparente et invisible.

1.8. Hydratation incrémentale et configurable

Pour les cas pointus , on peut paramétrer une hydratation incrémentale (c'est à dire différée à un affichage réel ou une interaction utilisateur via `@defer (hydrate on ...){ .... } @placeholder { static before dynamic }`

Activation de l'hydratation incrémentale :

app.config.ts

```
import { provideClientHydration, withIncrementalHydration } from '@angular/platform-browser';
...
//provideClientHydration(withEventReplay())
provideClientHydration(withIncrementalHydration())
...
```

NB : `withIncrementalHydration()` s'appuie en interne sur `withEventReplay()` activé d'office.

Pour vérifier en mode dev que l'hydratation incrémentale est activée:

```
ng serve --no-hmr
```

Car sans l'option `--no-hmr` , l'hydratation incrémentale est court-circuitée .

HMR = Hot Module Replacement = rechargement automatique du code en mode dev (par défaut via `ng serve`).

NB : ceci permet entre autre de visualiser dans la console web du navigateur un message de ce genre :

```
Angular hydrated 4 component(s) and 60 node(s), 0 component(s) were skipped. 1 defer block(s)
were configured to use incremental hydration
```

Exemple de paramétrage d'hydratation incrémentale :

my-ssg.component.html

```
...
<h4>with incremental hydration</h4>
@defer (hydrate on timer(8s)) {
  <div id="timeDiv" class="bg-red-200">
    rendered at {{clockService.currentDateTimeUtcString()}}
  </div>
}
```

Effets :

Affichage initial (temporaire) : <i>horaire du build</i>	with incremental hydration rendered at Tue, 24 Feb 2026 08:56:00 GMT
environ 8s plus tard, nouvel affichage : <i>horaire de l'hydratation</i>	with incremental hydration rendered at Tue, 24 Feb 2026 08:56:09 GMT

Comportements importants :

- Le rendu initial (SSG ou SSR) initialement téléchargé ne s'affiche (s'il en a le temps) qu'une seule fois au démarrage complet de l'application (que l'on peut provoquer via un "refresh" du navigateur).
- Une fois l'hydratation effectuée (ici au bout de 8s) , le nouveau contenu/rendu sera mémorisé et conservé tout le temps de fonctionnement de l'application même si on navigue vers un autre composant et que l'on revient où l'on est .

Comportements importants et pouvant sembler déroutants :

- Le contenu des blocs `@defer (hydrate on ....) { ... }` sont entièrement interprétés (sans aucun différé) lors de la phase de pré-rendu SSR ou SSG
- Si l'on souhaite visualiser une différenciation (un peu artificielle) du pré-rendu on peut éventuellement imbriquer un bloc de type :

```
@defer (hydrate on timer(8s)) {
  <div id="timeDiv" class="bg-red-200">
    rendered at {{clockService.currentDateTimeUtcString()}}
    @if(isClientSide){
      <p class="text-blue-600">rendu CSR</p>
    }@else {
      <p class="text-green-600">pre-rendu temporaire</p>
    }
  </div>
}
```

avec du côté .ts :

```
readonly platform = inject(PLATFORM_ID);
readonly isClientSide = isPlatformBrowser(this.platform);
```

rendered at Tue, 24 Feb 2026 09:17:50 GMT pre-rendu temporaire	rendered at Tue, 24 Feb 2026 09:17:58 GMT rendu CSR
---	--

Remarque: les 8s de pause et les différenciations ci dessus n'étaient là que pour visualiser le phénomène d'hydratation incrémentale .

Au sein de la plupart des applications on ne souhaite qu'un démarrage hyper-rapide puis un fonctionnement de plus en plus dynamique avec un contenu bien actualisé .

Quelques variations :

@defer (hydrate on hover) {...}	Déclenchement au survol de la zone via pointeur de souris
hydrate on viewport	Dès l'affichage visible
hydrate on interaction	Dès la première interaction de l'utilisateur
hydrate on idle	Lorsque le navigateur en aura le temps (pour choses secondaires , pas importantes)
hydrate on immediate	Pour contenu prioritaire
... when condition, ... never	Paramétrages et comportements complexes

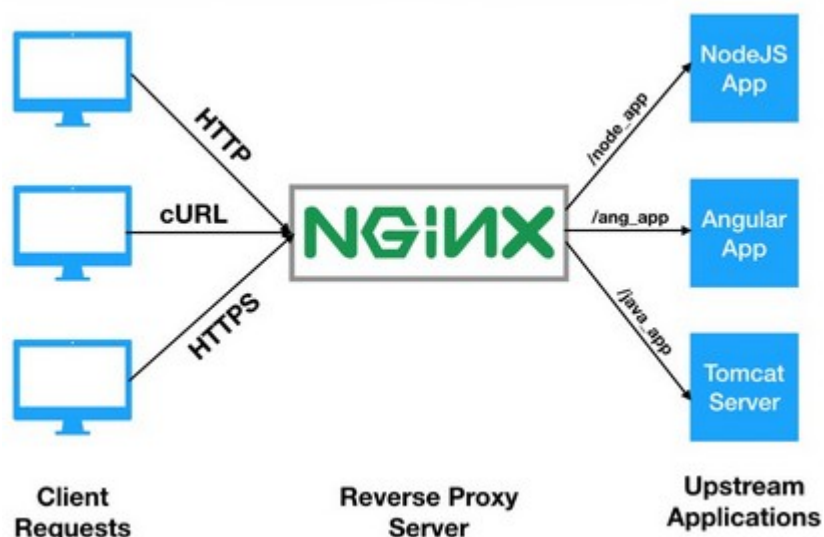
Pour approfondir les nombreuses limitations et subtilités :

<https://v21.angular.dev/guide/hydration>

Par exemple , en mode "zoneLess" , la temporisation du bloc `@defer (hydrate on timer(8s)) { ...}` semble avoir un effet de bord sur d'autres parties de la page ("refresh" à priori complet) .

1.9. SSR d'un point de vue topologie de production :

- Sans l'extension SSR, une application angular est structurée par une simple page html et quelques bundles rattachés (css, js , ...) et tout cela peut être directement pris en charge par un serveur HTTP (ex : nginx , apache http server, ...)
- **Avec l'extension SSR**, une **application angular** (quelquefois packagée sous forme de bundle) est ordinairement **gérée par nodeJs** et on a souvent un serveur HTTP intermédiaire qui joue au minimum le rôle de terminaison SSL (pour HTTPS) et de reverse-proxy :



Exemple de configuration partielle "nginx reroutant vers angular ssr app et autres backends" :

xyz-nginx.conf

```
...
    #qcm-api : (backend for qcm-app) , uses mongoDB
    location ~ ^/qcm-api/(.*){
        resolver 127.0.0.11;
        proxy_pass http://backend-qcm-api.api.service:8230/qcm-api/$1?$args;
    }
...
    #qcm-app : (angular frontend en mode ssr , dans conteneur docker basé sur nodejs )
    location ~ ^/qcm-app/(.*){
        resolver 127.0.0.11;
        proxy_pass http://qcm-app.frontend:4000/$1?$args;
    }
...
```

Exemple de configuration docker pour une application @angular/ssr :

Dockerfile	docker-compose.yml
FROM node:22 # Create app directory inside docker image WORKDIR /usr/src/app # Install app dependencies COPY package*.json ./ #RUN npm install # Bundle app source (src, dist, ...) COPY . . #setting ENV-VARIABLE ENV PORT=4000 EXPOSE 4000 CMD ["npm", "run" , "serve:ssr:qcm-app"]	networks: mynetwork: external: true services: qcm-app: #build : ../../local-git-repositories/qcm-app image: didierdefrance69/qcm_app:1 container_name: frontend-qcm-app restart: always environment: PORT : 4000 networks: mynetwork: aliases: - qcm-app.frontend ports: - "4000:4000" extra_hosts: - "host.docker.internal:host-gateway"

XII - Aspects divers et optimisations

Principales optimisations (angular) :

- Component-level Lazy Loading
- Data Binding, mode manuel : ChangeDetectorRef (imposé avec angular21/ZoneLess)
- trackBy pour *ngFor (maintenant imposé sur @for)
- Budgets (à optimiser via micro-frontend)
- Gestion des bonnes souscriptions RxJS (à surveiller)

ANNEXES

XIII - Annexe – Standalone, signaux et rappels

1. Structure d'une application angular 2+

1.1. Programmation "orientée objet" et "modularité par classe"

Selon une logique proche de celle de nodeJs , une application "Angular" peut s'appuyer sur les mots clefs **"export"** et **"import"** (de TypeScript et de ES2015) de façon à être construite sur une base **modulaire** .

Chaque composant élémentaire est un fichier à part. (dans le cas d'un composant visuel , il pourra y avoir des fichiers annexes ".html" , ".css") .

Il **exporte quelque-chose** (classe , interface , données , ...).

Un composant angular doit importer un ou plusieurs autre(s) module(s) ou classe(s) / composant(s) s'il souhaite **avoir accès aux éléments exportés** et les utiliser. Le référencement d'un autre composant s'effectue généralement via un chemin relatif commençant par **"/"** .

Le cœur d'**angular** est codé à l'intérieur de **librairies de composants prédéfinis regroupés en modules**.

Les modules des **"librairies"** prédéfinies d'angular sont par convention préfixés par **"@angular"** .

Exemples :

```
import {Component}      from '@angular/core';
import {HttpClient}     from '@angular/common/http';
import {Observable}     from 'rxjs';
import {BrowserModule} from '@angular/platform-browser'
import {OnInit}         from '@angular/core';
import {Router, RouteParams} from '@angular/router';
...
```

1.2. Rappels des principaux types de composants "angular":

Module fonctionnel <i>maintenant facultatif</i>	Ensemble de composants et/ou de services (généralement placés dans un même répertoire) et chapeauté par une classe décorée via @NgModule .
Component <i>(composant visuel)</i>	Composant visuel de l'application graphique (associé à une vue basée elle même sur un template) – généralement spécifique à une application NB: un composant angular est souvent vu comme une nouvelle balise/sélecteur (ex : <code><app-header></app-header></code>)
Directive	Elément souvent réutilisable permettant de générer ou altérer dynamiquement une partie de l'arbre DOM . NB: une fois programmée, une directive peut s'utiliser sur tout un tas de balise/élément html (ex: sur <code><p></code> , sur <code><div></code> , sur <code></code> ...) Une directive s'utilise souvent comme un nouvel attribut spécial (ex : <code>[highlight]="yellow"</code> , <code>*ngIf="..."</code>)

	2 types de directives : "attribut" (fréquentes) et "structurelles" (rares)
Service (<i>invisible , potentiellement partagé</i>)	Un service est un élément invisible de l'application (qui rend un certain service) en arrière plan (ex : accès aux données , configuration, calculateur , ...)

2. Modules applicatifs (maintenant facultatifs)

NB :

- **Au sein des versions récentes d'angular (17, 18, ...) les modules sont devenus facultatifs. Une application sans module est qualifiée "standalone" .**
- **Au sein des anciennes versions d'angular (2,...,16) , les modules étaient obligatoires (au moins un module principal app.module.ts)**
- **Beaucoup de bibliothèques de composants réutilisables (ex : primeNg, ngx-bootstrap , @angular/material , ...) sont encore structurées en modules (paquets de composants) Et on a donc encore souvent besoin d'importer des modules (paquets de choses).**

Dans les grandes lignes :

- Un module angular correspond souvent à un répertoire (ex : xyz) et comporte un fichier **xyz.module.ts** avec **@NgModule()** comme décorateur principal .
- **En important un module , on importe (d'un seul coup) un paquet d'éléments.**

Exemple :

```
imports : [CommonModule]
```

peut être vu comme un raccourci pour

```
imports[ NgIf,NgFor,JsonPipe,DecimalPipe,...]
```

2.1. Standalone application (sans module)

NB: ceci a longtemps été impossible au sein des anciennes versions d'angular (de 2 à 14,15 ou 16). Ce n'est possible et vraiment au point que depuis la version 17.

Via `bootstrapApplication(MyMainComponent)`; de `@angular/core` on peut démarrer une application angular sans module à partir d'un simple composant en mode standalone.

src/main.ts en version standalone

```
import { bootstrapApplication } from '@angular/platform-browser';
import { appConfig } from './app/app.config';
import { AppComponent } from './app/app.component';

bootstrapApplication(AppComponent, appConfig)
  .catch((err) => console.error(err));
```

src/app/app.config.ts //fichier de configuration global à l'ensemble de l'application

```
import { ApplicationConfig } from '@angular/core';
import { provideRouter } from '@angular/router';

import { routes } from './app.routes';
import { provideClientHydration } from '@angular/platform-browser';

export const appConfig: ApplicationConfig = {
  providers: [provideRouter(routes), provideClientHydration()]
};
```

src/app/app.routes.ts //fichier de configuration des routes (navigations)

```
import { Routes } from '@angular/router';
import { WelcomeComponent } from './welcome/welcome.component';

export const routes: Routes = [
  { path: 'welcome', component: WelcomeComponent },
  { path: '', redirectTo: '/welcome', pathMatch: 'full' },
];
```

Changement de comportement de angular-cli depuis v17

ng new simpleApp (standalone par défaut depuis v17)

ng new --no-standalone appWithModules

=====

NB: au sein d'une application "standalone", chaque composant est autonome (`@Component` avec `{ standalone : true , ...}`) et doit lui même renseigner finement ses dépendances (ex : RouterLink , FormsModule , DecimalPipe , NgFor , ...)

2.2. Standalone components

En mode **standalone**, un composant n'appartient plus à un module ; il est complètement indépendant et **défini lui même tous les importations dont il a besoin**.

```
@Component({
  standalone: true, ... })
export class MyAngularStandaloneComponent {}
```

NB : Le paramétrage standalone n'existait pas avant la version 14 (comme si il avait la valeur false par défaut). Au sein des versions 15 à 18 , il faut explicitement préciser standalone : true .

A partir de la version 19, le paramétrage standalone est maintenant implicitement avec la valeur true par défaut et on est donc obligé de spécifier *standalone : false* dans les cas où c'est utile (en mode module à l'ancienne).

Exemples d'importations classiques :

```
imports: [RouterOutlet , HeaderComponent, FooterComponent],
```

```
imports: [RouterLink , DatePipe , AsyncPipe , DecimalPipe , JsonPipe , UppercasePipe],
```

```
imports: [FormsModule , NgIf , NgFor],
```

3. Signaux (depuis angular 16+)

NB: Les signaux sont apparus avec la version 16 d'angular . Ils ont été en phase "developper-preview" au sein des versions 17,18 . L'essentiel est enfin stabilisé depuis angular 19.

Les **signaux** forment un **mécanisme d'actualisation réactive synchrone**. Il sont **vus comme des fonctions d'accès à des données** . Les signaux peuvent être composés/calculés à partir de sous-signaux (vus comme des sous fonctions).

Principales sortes de signaux :

Types de signaux	Fonctions de construction	Caractéristiques
WritableSignal	signal()	Sous objet de données évolutives avec signalement automatique des mises à jour
Signal	computed() , effect() en void	Signal de haut niveau basé (par calcul ou autre) sur des signaux de bas niveau
En remplacement de @Input() , @Output() , ...	input() , output() , model()	Signaux en remplacement de @Input() , @Output() de manière à obtenir de meilleures performances

3.1. Signaux ordinaires (WritableSignal , signal())

Exemple :

```
import { Component, Signal, computed, effect, signal } from '@angular/core';

@Component({
  selector: 'app-with-signal', standalone: true, imports: [],
  templateUrl: './with-signal.component.html', styleUrls: ['./with-signal.component.scss']
})
export class WithSignalComponent {
  //sCount=signal<number>(0);
  sCount = signal(0); //new WritableSignal With initial value to 0
  //other possible signal types : String, boolean , object , array , ...

  public onIncrement(){
    //NB: signalName as function call to get value ,
    // .set() to update/change value with synchronization
    this.sCount.set(this.sCount() + 1);
  }

  public onDecrement(){
    //set(newValue or .update(currentValue->newValue)
    //this.sCount.set(this.sCount() - 1);
    this.sCount.update ( count => count-1);
  }

  public onSCountChange(event : Event){
    const input = event.target as HTMLInputElement;
    this.sCount.set(Number(input.value));
  }
}
```

```
<span [style.color]="sCountColor()">sCount={{sCount()}}</span>
&nbsp; <button (click)="onIncrement()">++</button>
&nbsp; <button (click)="onDecrement()">--</button> <br/>
<hr/>
sCount = <input type="number" [value]="sCount()" (change)="onSCountChange($event)" />
```

sCount=15

++

refresh

--

sCount =

15

- avec signaux --> meilleures performances , meilleure détection automatique des changements et réactualisation automatique des affichages .

3.2. Signal avec ngModel

```
@Component({...
  imports: [FormsModule, UpperCasePipe , ...],...
})
export class WithSignalComponent {
```

```
name1="abc";
name2AsSignal=signal("abc");
name3AsSignal=signal("abc");
}
```

```
name1:<input [(ngModel)]="name1" /> {{name1 | uppercase}} (without signal)<br/>
name3AsSignal:<input [(ngModel)]="name3AsSignal()" (ngModelChange)="name3AsSignal.set($event)" />
{{name3AsSignal() | uppercase}} <br/>
<!-- more simple equivalent with some recent angular version : -->
name2AsSignal:<input [(ngModel)]="name2AsSignal" /> {{name2AsSignal() | uppercase}} <br/>
```

name1: ABCD (without signal)
 name3AsSignal: ABCD
 name2AsSignal: ABCD

→ la tendance actuelle de l'évolution d'angular consiste à petit à petit s'éloigner de l'ancien système de détection des changements (avec zoneJs) et d'utiliser le mécanisme des signaux à la place.

3.3. Signaux calculés (computed())

Exemple :

```
import { Component, Signal, computed, effect, signal } from '@angular/core';
...
export class WithSignalComponent {
  sCount = signal(0);

  //NB: la fonction computed() (de@angular/core) permet de définir un nouveau signal calculé
  // à partir d'autres signaux → réactualisation automatique .

  sSquare /* :Signal<Number> */ = computed(() => this.sCount() * this.sCount());
  sCountColor /* :Signal<String> */ = computed(() => this.sCount()>=0?'green':'red');
}
```

```
...
sSquare={{sSquare()}} <br/>
```

sCount=8

sCount =

sSquare=64

NB: une valeur, en lecture seule, d'un signal "computed()" est automatiquement mise en cache et n'est par la suite recalculée que si un des signaux en dépendance vient à changer.

3.4. Effets (à utiliser qu'avec parcimonie)

Exemple :

```
import { Component, Signal, computed, effect, signal } from '@angular/core';
...
export class WithSignalComponent {
  message="";
  sCount = signal(0); //new WritableSignal With initial value to 0

  //la fonction effect() (de @angular/core) permet d'enregistrer une callback (traitement quelconque)
  //qui sera automatiquement appelée dès qu'un signal changera de valeur

  logsCountEffect = effect(()=>{ this.message="sCount="+this.sCount();
                                console.log(this.message);});
  // logsCountEffect n'est utile que syntaxiquement (pour coder l'enregistrement de l'effet
  // comme un attribut de la classe) et pour un éventuel this.logsCountEffect.destroy()
}
```

```
<span [style.color]="sCountColor()">sCount={{sCount()}}</span> &nbsp; <br>
<button (click)="onIncrement()">++</button> &nbsp; <button (click)="onDecrement()">--</button> <br>
<hr/> message(as an effect)={{message}}
```

message(as an effect)=[5] sCount=4

3.5. assez complexe linkedSignal()

Dans les grandes lignes , **linkedSignal()** est une variante de computed() qui est **modifiable** (pas en lecture seule).

Le comportement par défaut (avec syntaxe simple) est le suivant :

1. - première valeur extraite/calculée = première valeur (souvent sous partie) selon valeur initiale du signal de base
2. - nouvelle valeur temporaire = valeur directement modifiée via .set()
3. - valeur recalculée/réactualisée (comme étape 1) si le signal de base est réactualisé

Une variante syntaxique très complexe , on peut éventuellement recalculer la nouvelle valeur en prenant en compte l'éventuelle ancienne valeur temporaire (prioritaire ou pas, ...) selon un algorithme complètement libre.

Exemple (extraction d'une sous partie ".firstname"):

```
class User{
  constructor(public firstName:string="?", public lastName:string="?", public email:string="?"){
  }
  ...
  // Signal de base :
  user = signal<User>({
```

```
firstName: 'jean',
lastName: 'Bon',
email: 'jean.Bon@xyz.com'
});
```

// Signal lié (un peu comme computed() mais en lecture/écriture) :
 firstName = **linkedSignal**(()=>this.user().firstName); //basic syntax

1) `user: { "firstName": "jean", "lastName": "Bon", "email": "jean.Bon@xyz.com" } firstName: jean`

2) après "update_firstName" et firstName.set(...), valeur temporaire :

```
newFirstName: first_name update_firstName update_user
user: { "firstName": "jean", "lastName": "Bon", "email": "jean.Bon@xyz.com" } firstName: first_name
```

3) après "update_user" et réactualisation selon signal de base (comme computed()) :

```
newFirstName: first_name update_firstName update_user
user: { "firstName": "alain", "lastName": "Therieur", "email": "alain.Therieur@xyz.com" } firstName: alain
```

Variante complexe :

```
firstName = linkedSignal<User,string>({
  source : this.user ,
  computation: (newUserFromSignal , previousSourceAndValue) =>{
    let computedFirstName="?";
    console.log("newUserFromSignal="+JSON.stringify(newUserFromSignal));
    console.log("previousSourceUser="+JSON.stringify(previousSourceAndValue?.source));
    console.log("previousFirstnameValue="+previousSourceAndValue?.value);
    //computedFirstName = newUserFromSignal.firstName; //default behaviour
    if(previousSourceAndValue?.value.includes("_"))
      computedFirstName= previousSourceAndValue?.value;
    else
      computedFirstName = newUserFromSignal.firstName;
    return computedFirstName;
  } });
```

L'algorithme ci dessus permet de choisir entre :

- la valeur temporairement affectée (**previousSourceAndValue?.value**)

- la valeur issue de la réactualisation du signal de base (`newUserFromSignal.firstName`)
- une éventuelle valeur librement calculée en fonction des deux informations :
 - ici on choisi de garder la valeur temporaire si elle contient un " _ "

3.6. Ré-évaluation des signaux si changements

Pour des raisons d'optimisation, les effets et les rafraîchissements/réactualisations d'affichage ne sont effectués que lorsqu'un signal change réellement de valeur.

Autrement dit, appeler `signalXy.set(meme_valeur)` ne déclenche rien !

Conséquences :

- les signaux fonctionnent parfaitement bien avec des objets "immuables" (string, number, ...)
- besoin de re-crée quelquefois des instances pour déclencher des réactualisations dans certains cas pointus .

```
// Refreshable<T> wrapper for handling immutability of signal value
class Refreshable<T>{
    public timestamp: Date;
    constructor(public value:T){
        this.timestamp=new Date(); //with time
    }
    time(){ return this.timestamp.getTime(); }
    refresh(){ return new Refreshable<T>(this.value); }
}
```

NB: un bon rafraîchissement ce n'est pas qu'un ré-affichage juste à côté (via `{{signalName() | json }}`) mais un **bon re-déclenchement** des autres `computed()` et `effect()` basés sur le signal d'origine .

Pour cela , **il faut absolument que la valeur passée à `signalXy.set()` soit changée !!!**

Aucun soucis pour une valeur primitive mais **nouvelle instance, littérale ou pas (via new), pour un objet .**

D'autre part , pour éviter des re-déclenchement inutiles de `computed()`, `effect()`, ... lorsque des objets littéraux ont par hasard les mêmes valeurs internes , on peut préciser une fonction de test d'égalité (en mode "deep") en tant qu'option facultative de la fonction `signal()` :

```
sXy2 = signal({ ref:"ref2" , value:0 } ,
    { equal : (a,b)=> (a.ref === b.ref) && (a.value === b.value)});
```

Bien que possible, ce paramétrage pointu est rarement utile/nécessaire.

3.7. C'est quoi techniquement , un signal angular ?

- D'un point de vue "vision publique" : un signal est une valeur avec une sémantique de changement explicite. Dans Angular, un signal élémentaire est représenté par une **fonction getter sans argument** renvoyant sa valeur actuelle.
- Certains signaux sont en **lecture seule**. Cela signifie qu'il est possible de propager des valeurs réactives aux consommateurs sans leur donner la capacité de modifier la valeur eux-mêmes (*typiquement le cas d'un signal de type `computed()` ou `input()`*) .

3.8. input(), output(), model()

en tant que signaux pour interactions avec sous-composants .

sub.component.ts

```
import { Component, input, model, output } from '@angular/core';
import { FormsModule } from '@angular/forms';

@Component({
  selector: 'app-sub', imports: [FormsModule], standalone : true,
  templateUrl: './sub.component.html', styleUrls: ['./sub.component.scss']
})
export class SubComponent {
  public ic = input(0); //input counter as input signal
  public oc = output<number>(); //output counter as output signal
  public mc = model(0); //model (input+output) counter as model signal

  public ocVal=0;

  onOcIncr() {
    this.ocVal++;
    this.oc.emit(this.ocVal);
  }
}
```

sub.component.html

```
<div class="sc">  
<span>sub component</span> <br/>  
<span>ic (input counter signal): {{ic0}}</span>&nbsp;&nbsp;&nbsp;<br/>  
<button (click)="onOcIncr()">increment oc (output counter signal) : {{ocVal}}</button> <br/>  
mc = <input [(ngModel)]="mc" /> (model counter signal)  
</div>
```

parent...html

```

...
<app-sub [(mc)]="pMcS" [ic]="sCount()" (oc)="onOc($event)" ></app-sub>
<span>pOcVal={{pOcVal}}</span> <br/>
<span>pMcS:<input [(ngModel)]="pMcS" /> {{pMcS}}</span>

```

parent...ts

```
pMcS=signal(100); //parentMcSignal value
pOcVal=0; //parent oc value
...
public onOc( ocVal: number){
    this.pOcVal=ocVal;
}
```

```

sub component
ic (input counter signal): 5 increment oc (output counter signal) : 11
mc = 10555 (model counter signal)
pOcVal=11
pMcS: 10555 10555

```

NB : input() , output() et model() seront approfondis au sein d'un chapitre ultérieur .

3.9. Typage indirect des signaux :

Au sens type de données "typescript" , on pourrait éventuellement directement déclarer le type précis d'un signal avec une **syntaxe déconseillée de ce genre** :

```

//liste des objets à afficher :
tabObjectsRef :InputSignal<object[] | null>
= input(null,{transform: (tabObjects)=><any> tabObjects });

//objet sélectionné (null au début):
public selectedObjectRef :ModelSignal<any>= model(null);

```

Mais il est bien plus élégant/pratique/conseillé de laisser typescript déduire le type exact du signal en se basant si besoin sur l'aspect générique des fonctions d'initialisations des signaux :

```

//liste des objets à afficher :
tabObjectsRef = input<object[] | null>(null);

//objet sélectionné (null au début):
public selectedObjectRef = model<any>(null);

```

3.10. Signaux avec services et RxJs

NB:

- Des signaux partagés peuvent être placés dans des services

Signal (angular 16+) vs BehaviorSubject(RxJs)

- Les signaux (synchrones) sont généralement mieux (plus simples, plus performants) dans la majorité des cas.
- Les "Subjects" (de RxJs) sont plus complexes/perfectionnés (asynchrones, multiples abonnés, ...) et sont à réserver/utiliser pour les cas pointus.

```
import { interval } from 'rxjs';
import { toSignal } from '@angular/core/rxjs-interop';
...
myIntervalObs = interval(1000); //observable emitting 1,2,3,.. every 1000ms
myIntervalSignal = toSignal(this.myIntervalObs, { initialValue: 0 } )
```

```
...
from observable : {{ myIntervalObs | async }} <br/>
from signal : {{ myIntervalSignal() }}
```

from observable : 10

from signal : 10

4. Code flow (depuis angular 17)

Depuis angular 17, il est possible d'utiliser de nouvelles syntaxes (appelées "**code flow**") au sein des templates html.

Principaux intérêts:

- **pas besoin de imports:[NgIf, NgFor]**
- **performances améliorées**

Ainsi, au lieu d'utiliser `*ngIf="..."` ou bien `*ngFor="let v of values"` placées sur des balises de type "`<p>`" ou "`<div>`" ou autres on peut directement utiliser `@if(...)` `@else{...}` dans le texte du template (au dehors des balises html) .

4.1. @if(...) { ... } @else{ ... }

```
<input type="checkbox" [(ngModel)]="withDetails"> withDetails
<hr/>
@if(withDetails) {
```

```

<div>
  <p>details ...</p>
</div>
} @else {
  <p>no details</p>
}

```

4.2. @for(...){ ... }@empty{...}

```

<table border="1">
  <tr><th>v</th><th>v*v</th></tr>
  @for(v of values; track v){
    <tr><td>{{v}}</td><td>{{v*v}}</td></tr>
  }
  @empty {
    <tr><td>?</td><td>?</td></tr>
  }
</table>

```

NB :

- La partie `@empty {}` est facultative et n'est utilisée que si le tableau est vide .
- La partie `; track v` (ou bien `; track obj.id`) est obligatoire et permet de préciser l'identifiant de chaque entrée de la collection . C'est un point clef pour obtenir de bonnes performances.

Attention: depuis la version 17 d'angular , `*ngIf` et `*ngFor` ne sont plus préconisés car ça nécessite `imports:[NgIf,NgFor, ...]` et c'est moins performant que `@if()` et `@for()`

```

@for (u of users; track u.id ; let i = $index) {
  <p>at index {{i}} : {{u | json}} </p>
}

```

Via `let i = $index` on peut récupérer l'index de l'élément durant le parcours de la collection.

Comme `$index` il existe aussi `$count` (nombre d'éléments de la collection) et sous forme de booléens `$first` , `$last` , `$even` (pair) , `$odd` (impair) .

4.3. @switch(...){ @case(...){ ... } }

```

<select [(ngModel)]="detailLevel" >
  <option>none</option>
  <option>low</option>
  <option>high</option>
</select> <br/>
@switch (detailLevel) {
  @case("none"){ <div>detailLevel=none [no details]</div>}
  @case("low"){ <div>detailLevel=low , few details</div>}
  @case("high"){ <div>detailLevel=high , much details , ...</div>}
}

```

```
}
```

NB: il existe `@default { }`

4.4. @let

Depuis la version 18.1 d'angular , on peut déclarer et utiliser des variables locales dans les templates html via `@let` et un `;` à ne pas oublier.

Exemples :

```
@let value = (val$ | async) ?? 0;
```

```
<div>{{ value }}</div>
```

```
<ul>
  @for(user of users(); track user.id) {
    @let isAdmin = checkIfAdmin(user);
    <li>User is admin: {{ isAdmin }}</li>
  }
</ul>
```

NB : au sein de cet exemple , la fonction `checkIfAdmin` ne sera exécutée que si son paramètre (dépendance) change.

XIV - Annexe – Rappels sur input/output , projection

1. input() , ouput() , model() , <ng-content>

1.1. Vue d'ensemble sur input et output

input() et output() pour composants réutilisables

dans composant parent

```
<c1 [p1]=" 'valeurP1' " (eventA)="onEvtA($event)"></c1>
```

```
@Component({selector : 'c1',...})
```

```
SousComposant 1 {
```

```
  p1 = input('defaultValue')
```

```
  eventA = output<...>();
```

```
  onInternalEvt(){
    this.eventA.emit(...);
  }
}
```

*et éventuelles
répercussions
sur*

Autre(s)
sousComposant(s)

Principe fondamental (commun au framework concurrent "react") :

Au sein d'une arborescence de composants ,

- les ordres/directives/paramétrages descendent
- les événements remontent

Evolution importante :

- Avant angular 17, les signaux input() et ouput() n'existaient pas encore et on devait utiliser des décorateurs @Input et @Output
Ces décorateurs sont encore utilisables au sein des versions récentes d'angular mais sont moins bien car plus complexes, moins flexibles et moins performants .
Les anciennes syntaxes ont été déplacées dans une annexe.

- A partir de la version 17 d'angular, il est fortement recommandé d'utiliser **input()** et **output()** sous forme de signaux .

1.2. input() signal (alternative à @Input)

Au sein des versions récentes d'angular , on peut remplacer @Input par la **fonction input()** produisant un signal.

Principal avantage: meilleure synchronisation (plus performante).

Code du côté "sous composant" :

```
import { Component , inject, input } from '@angular/core';
...
export class HeaderComponent {
  titre = input("titre_par_defaut"); //input<string or ...>(...)
  ...
}
```

```
... {{titre()}}...
```

Exemple d'utilisation depuis un composant parent : *app.component.ts*

```
import { Component } from '@angular/core';
...
@Component({
  selector: 'app-root',
  imports: [RouterOutlet , HeaderComponent , FooterComponent],
  templateUrl: './app.component.html'
})
export class AppComponent {
  title /* :string */ = 'my-standalone-app';
}
```

app.component.html

```
<app-header [titre]="title" > </app-header>
```

```
...
```

```
my-standalone-app welcome basic
```

Variantes d'utilisation :

```
<app-header></app-header> <!-- avec valeur par défaut -->
<app-header titre="monAppli"></app-header> <!-- avec valeur fixe -->
<app-header [titre]=" 'monAppli' "></app-header> <!-- syntaxe complexe possible -->
<app-header [titre]="titre"></app-header> <!-- avec valeur variable
                                     (copie dynamique de title) -->
```

1.3. output() en tant que signal depuis angular 17

output() (au niveau d'un sous-composant) permet de déclarer un **événement** qui sera potentiellement remonté et géré par un composant parent/englobant.

Depuis angular 17 le `output()` sous forme de signal remplace l'ancien décorateur `@Output()`.

reglette.component.ts

```
import { Component, input, output } from '@angular/core';
@Component({...})
export class RegletteComponent {

  width /*:string*/ = input("100"); //largeur paramétrage (100px par défaut)

  // @Output()
  // changeEvent = new EventEmitter<{value:number}>();
  changeEvent = output<number>();

  onInternalCurscur(event : Event){
    const evt : MouseEvent = <MouseEvent> event;
    const valX = evt.offsetX;
    const pctCurscur = (valX / Number(this.width())) * 100 ; //en %
    //console.log("pctCurscur="+pctCurscur);
    this.changeEvent.emit(pctCurscur);
  }
}
```

reglette.component.html

```
<div class="reglette" (click)="onInternalCurscur($event)"
  [style.width.px]="width()">
</div>
```

reglette.component.css

```
.reglette {
  width : 100px; height : 40px;
  background: linear-gradient(to right, white, blue);
  border-style : solid; border-width : 2px; border-color : blue;
}
```

Exemple d'utilisation au niveau d'un composant parent :

```
...
export class DemoComponent {
  valeurCurscur /*:number*/ =0; //en %

  onChangeCurscur(eventValue : number){
    this.valeurCurscur = eventValue;
  }
  ...}
```

```
<app-reglette width="200" (changeEvent)="onChangeCurseur($event)"></app-reglette>
curseur = {{valeurCurseur}}
```



curseur = 49

si click dans le milieu de la réglette



curseur = 1

si click dans le début de la réglette (coté gauche)



curseur = 97

si click dans la fin de la réglette (coté droit)

1.4. model() bi-directionnel

La fonction **model()** permet de créer un **signal bidirectionnel** (combinant à peu près les fonctionnalités de input() et output()) un peu comme de ngModel prédéfini d'angular .

Exemple de synchronisation entre vision interne et vision externe (composant parent) :

☐ with gradient

curseur = 0

☐ avecDegradeCouleur

☒ with gradient

curseur = 43.5

☒ avecDegradeCouleur

Code simplifié de l'exemple allant à l'essentiel :

reglette.component.ts

```
import { Component, input, model } from '@angular/core';
export class RegletteComponent {
  gradient=model<boolean>(false); //avec dégradé de couleur ?
  ...
  onCheckedChange() {
    const valeurActuelle = this.gradient(); const nouvelleValeurInversee = ! valeurActuelle ;
    this.gradient.set( nouvelleValeurInversee );
  }
}
```

reglette.component.ts

```
<div ...>
  <input type="checkbox"
    [checked]=gradient() (change) = "onCheckedChange()" />with gradient
</div>
```

ou bien plus simplement encore :

```
... <input type="checkbox" [(ngModel)]=gradient" />with gradient
```

Utilisation externe (depuis un composant parent) de la nouvelle version du composant reglette avec gradient (dégradé de couleur paramétrable : true or false) :

```
avecDegradeCouleur=false;
```

et

```
<app-reglette ... [(gradient)]="avecDegradeCouleur" ></app-reglette>
```

```
<input type="checkbox" [(ngModel)]="avecDegradeCouleur" /> avecDegradeCouleur
```

☐ with gradient

curseur = 0

☐ avecDegradeCouleur

☒ with gradient

curseur = 43.5

☒ avecDegradeCouleur

⇒ le comportement est une **synchronisation bidirectionnelle** !

Si on coche *avecDegradeCouleur* au niveau parent , le *gradient* interne bascule immédiatement et inversement .

Tout comme (ngModelChange) pouvant accompagner [(ngModel)] il est possible d'ajouter une méthode événementielle (avec traitement quelconque , ex : affichage console) lorsque la valeur du signal de type model changera au cours du temps :

```
onGradientChange(){
  console.log("nouvelle valeur de avecDegradeCouleur="+this.avecDegradeCouleur);
}
```

et

```
<app-reglette ... [(gradient)]="avecDegradeCouleur"
  (gradientChange)="onGradientChange()" ></app-reglette>
```

1.5. Projection d'éléments imbriqués (<ng-content>)

Un composant angular peut (en tant que nouvelle balise telle que *toggle-panel* ici) , incorporer à son tour certains sous éléments (ex : <div ...> ou autre sous-sous composant angular).

Exemple :

```
<toggle-panel title="panel1" >
  <app-part1></app-part1> <!-- ou ... , vu comme ng-content dans toggle-panel -->
</toggle-panel>

<toggle-panel title="panel2" >
  <div>contenu du panneau 2</div> <!-- vu comme ng-content dans toggle-panel....html -->
</toggle-panel>
```

Le (ou le paquet de) sous-composant(s)/sous-balises imbriqué au niveau de l'utilisation d'un composant réutilisable sera vu via la balise spéciale **<ng-content></ng-content>** au sein du template HTML (code interne) de ce composant.

Cette fonctionnalité était appelée "transclusion" au sein des directives "angular Js 1.x" , elle est maintenant appelée "**projection de contenu**" au sein des composants angular 2+ .

Exemple concret : composant réutilisable "togglePanel" basé sur des styles *bootstrap* et fontes *bootstrap-icons* :

```
import { Component, input , model } from '@angular/core';
@Component({
  selector: 'toggle-panel',
  templateUrl: './toggle-panel.component.html' , styleUrls: ['./toggle-panel.component.scss']
})
export class TogglePanelComponent {
  panelOpenState=model(false);

  title /* : string */ = input( 'default panel title' );
  constructor() { }
}
```

version du template html basée sur les classes css de bootstrap 5 :

```
<div class="card mb-2">
  <div class="card-header text-white bg-primary"
    (click)="panelOpenState.set(!panelOpenState())" >
    {{title()}} <i [class.bi-arrow-down-circle-fill]="!panelOpenState()"
      [class.bi-arrow-up-circle-fill]="panelOpenState()" ></i>
  </div>
  <div class="card-body collapse" [class.show]="panelOpenState()">
    <ng-content></ng-content> <!-- remplacé par le contenu imbriqué -->
  </div>
</div>
```

version du template html basée sur les classes de tailwind css:

```
<div class="mt-1 mb-1">
<h4 class="mb-0 p-1 bg-blue-600 rounded-t-sm">
<a class="text-white no-underline" (click)="panelOpenState.set(!panelOpenState())" >
<span class="bg-white text-blue-600 m-1 px-1 font-bold"
[style.display]="panelOpenState()?'none':'inline-block'">+</span>
<span class="bg-white text-blue-600 m-1 px-1 font-bold"
[style.display]="panelOpenState()?'inline-block':'none'">-</span>{{title()}}
</a>
</h4>
<div class="border-2 border-blue-600 p-1 rounded-b-sm hidden"
[style.display]="panelOpenState()?'block':'none'">
<ng-content></ng-content> <!-- remplacé par le contenu imbriqué -->
</div>
</div>
```

version du template html basée sur les classes css spécifiques :

```
<div class="my-card">
  <h4 class="my-card-header my-bg-primary">
    <a class="my-text-light" (click)="panelOpenState.set(!panelOpenState())" >
      <span class="my-icon" [style.display]="panelOpenState()?'none':'inline-block'">+</span>
      <span class="my-icon" [style.display]="panelOpenState()?'inline-block':'none'">-</span>{{title()}}
    </a>
  </h4>
  <div class="my-card-body my-collapse" [class.my-show]="panelOpenState()"
    <ng-content></ng-content> <!-- remplacé par le contenu imbriqué -->
  </div>
</div>
```

toggle-panel.component.scss

```
.my-card { margin-top: 0.1em; margin-bottom: 0.1em; }
.my-card-header { border-top-left-radius: 0.3em; border-top-right-radius: 0.3em;
  padding: 0.1em; margin-bottom: 0px;}
a { text-decoration: none;}
.my-card-body {border: 0.1em solid blue; border-bottom-left-radius: 0.3em;
  border-bottom-right-radius: 0.3em; padding: 0.2em;}
.my-bg-primary { background-color: blue;}
.my-text-light { color : white;}
.my-icon { color : blue; background-color: white; margin: 0.2em;
  padding-left: 0.2em; padding-right : 0.2em;
  min-width: 1em; font-weight: bold;}
.my-collapse { display : none;}
.my-show { display : block ;}
```

Exemple d'utilisation :

```
<toggle-panel title="calculatrice">
  <app-calculatrice></app-calculatrice> <!-- ou ... , vu comme ng-content dans toggle-panel -->
</toggle-panel>

<toggle-panel title="calcul tva">
  <app-tva></app-tva> <!-- vu comme ng-content dans toggle-panel....html -->
</toggle-panel>
```

calculatrice ⬇	calculatrice ⬆ calculatrice a: 0 ⬆ b: 0 ⬆ additionner (a+b) soustraire (a-b) res: 0
---	---

XV - Annexe – Angular library & workspace

...

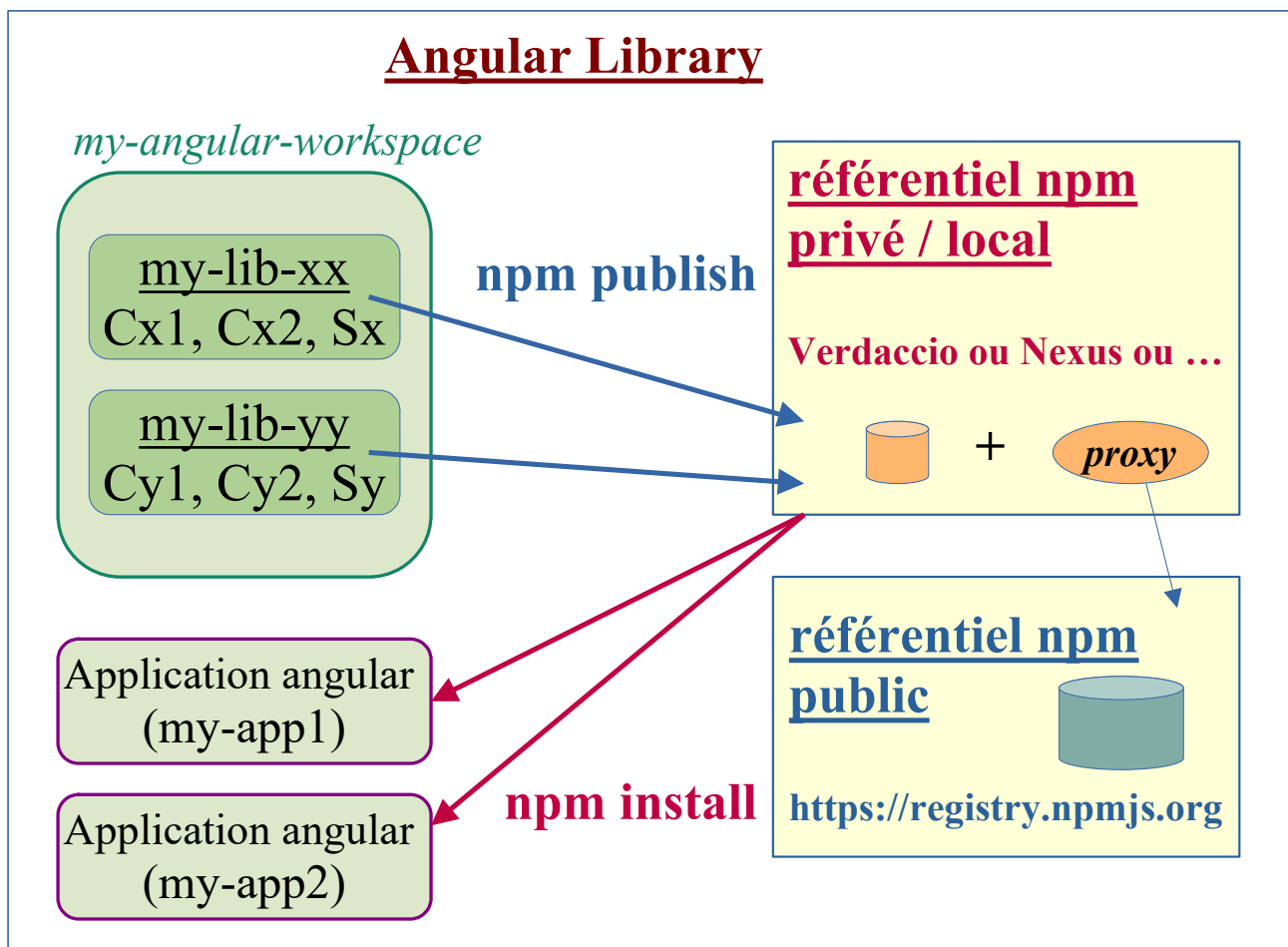
1. Notion de "angular library"

Une **librairie angular** correspond à un **sous projet réutilisable**.

Pouvant contenir des services utilitaires et des composants réutilisables (avec éventuels css),
une **librairie angular** est publiée dans un **référentiel npm** (public ou bien local/privé).

Ceci permet de mettre en commun tout un tas d'éléments réutilisables entre plusieurs projets angular développés par une même entreprise.

Ca peut également servir à coder une extension angular publique qui pourra ensuite être téléchargée et réutilisée par un très grand nombre de développeurs.



2. Préparation publication de librairies

2.1. Publication locale via verdaccio

NB : la solution hyper locale *npm link* comporte beaucoup de limitations/bugs .

Une réelle publication vers <https://www.npmjs.com/> impose un changement de version à chaque fois que le package est republié (ex 0.0.1 --> 0.0.2).

En phase de développement, on a besoin d'un référentiel npm local simple et léger de manière à republier fréquemment et localement un package sans être obligé de changer sa version.

verdaccio est un "simple/light npm local repository manager/proxy"

```
npm install -g verdaccio
```

C:\Users\xyz\AppData\Roaming\verdaccio\config.yaml : config file after first launch

<http://localhost:4873/> : default verdaccio url

Lancement et arrêt du serveur (référentiel/proxy) verdaccio :

```
verdaccio
```

Ctrl-C

Configuration du référentiel npm :

```
npm adduser --registry http://localhost:4873
```

```
REM user1/pwd1 user1@xyz.com
```

npm set registry <http://localhost:4873>

(replace default "npm set registry <https://registry.npmjs.org>")

npm login (ex: user1 or didier-defrance)

password (ex: pwd1 or .2.n...9)

email (ex: user1@worldcompany.com or didier@d-defrance.fr)

npm whoami

→ maintenant prêt pour **npm publish**

NB: si dernière version sérieusement publiée sur <https://registry.npmjs.org>

est 0.0.n alors en phase de dev (sur local-verdaccio) : **0.0.N+1 = version minimum acceptée**

et si *npm publish* est refusé par verdaccio alors **npm unpublish** avant npm publish

2.2. Publication sur <https://registry.npmjs.org>

```
npm set registry https://registry.npmjs.org
```

npm login (ex: user1 or didier-defrance)

password (ex: pwd1 or .2.n...9)

email (ex: user1@worldcompany.com or didier@d-defrance.fr)

npm whoami

→ maintenant prêt pour **npm publish**

3. Programmation de librairie(s) angular

Les *librairies angular* sont souvent placées dans des *workspaces angular* (paquets de projets)

3.1. Création de la structure (workspace + librairies)

Création du workspace (méga projet) :

```
ng new my-ngx-util-libspace --create-application=false
```

#to generate *my-ngx-util-libspace* with *package.json* , ... *without src* , *scr/app*

Création d'une librairie (sous projet) :

```
cd my-ngx-util-libspace
```

```
ng g library d2f-ngx-commons
```

#==> generate *projects/d2f-ngx-commons* into *my-workspace*

#and update *my-ngx-util-libspace/angular.json* with "*d2f-ngx-commons*" project

Au sein de **ng-package.json**

de *projects/d2f-ngx-commons*

ajouter

```
"assets": ["/assets"]
```

pour publier un sous répertoire *assets* (à coté de *src*) comportant des futurs fichiers *.css* ou *.scss*

projects > d2f-ngx-commons

Nom

- assets
- src
- karma.conf.js
- ng-package.json
- package.json
- README.md
- tsconfig.lib.json
- tsconfig.lib.prod.json
- tsconfig.spec.json

Création d'autres librairies et d'un éventuel petit projet de test:

```
cd my-ngx-util-libspace
```

```
ng g library d2f-ngx-components
```

```
ng g library d2f-ngx-crud
```

```
ng g library d2f-ngx-layout
```

```
ng g app test-app (just for quick testing of local  
css and libraries features, )
```

```
#ng serve (to run from workspace directory)
```

Structure globalement construite :

my-ngx-util-libspace > projects

Nom

- d2f-ngx-commons
- d2f-ngx-components
- d2f-ngx-crud
- d2f-ngx-layout
- test-app

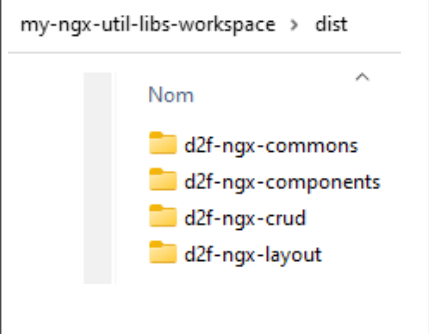
NB : placer un petit projet de test (ici *test-app*) directement dans le workspace , ça sert simplement à effectuer des tests élémentaires mais ça comporte plein de limitations et ça ne teste pas tout !

angular.json (de niveau *workspace global*) :

```
{
  "$schema": "./node_modules/@angular/cli/lib/config/schema.json",
  "version": 1,
  "newProjectRoot": "projects",
  "projects": {
    "d2f-ngx-commons": {
      "projectType": "library",
      "root": "projects/d2f-ngx-commons",
      "sourceRoot": "projects/d2f-ngx-commons/src",
      "prefix": "lib",
      "architect": {
        "build": {
          ...
        },
        "defaultConfiguration": "production"
      },
      "test": {
        ...
      }
    },
    "d2f-ngx-components": {....
  },
  "d2f-ngx-crud": {...
  },
  "d2f-ngx-layout": {....
  },
  "test-app": {
    "projectType": "application",
    ...
    "root": "projects/test-app",
    "sourceRoot": "projects/test-app/src",
    "prefix": "app",
    "architect": {
      "build": {
        ....
      }
    }
  }
}
```

3.2. Construction et publication des librairies

Une fois que les librairies auront été codées, on pourra les construire et publier via les lignes de commandes suivantes :

<pre>cd my-ngx-util-lib-space ng build d2f-ngx-commons ng build d2f-ngx-components ng build d2f-ngx-crud ng build d2f-ngx-layout</pre>		- assets - esm2020 - fesm2015 - fesm2020 - lib - index.d.ts - package.json - public-api.d.ts - README.md
--	--	--

Après préparation et réglage du contexte npm (éventuel démarrage de verdaccio , npm set registry , npm login, npl whoami)

```
cd dist/d2f-ngx-commons
#npm unpublish
npm publish
```

et idem pour autres librairies

Exemple de script de publication :

publish_local_d2f_ngx_commons.bat

```
set MY_NGX_LIB=d2f-ngx-commons
set TEMP_LIB_VERSION=0.0.4
cd dist\%MY_NGX_LIB%
REM with verdaccio local npm registry
npm unpublish %MY_NGX_LIB%@%TEMP_LIB_VERSION% && npm publish

REM with https://www.npmjs.com remote npm registry , no unpublish but increment version
```

3.3. Paramétrage et codage d'une librairie

Fichiers de configuration d'une librairie :

my-ngx-util-libs-workspace/projects/d2f-ngx-commons/*package.json*

```
{
  "name": "d2f-ngx-commons",
  "version": "0.0.4",
  "author": "didier defrance",
  "description": "commons utilities (classes , services , helpers, css) for angular projects" ,
  "repository" : {
    "type" : "git" ,
    "url" : "https://github.com/didier-mycontrib/angular17plus"
  },
  "peerDependencies": {
    "@angular/common": "^21.1.0",
    "@angular/core": "^21.1.0",
    "rxjs": "~7.8.0"
  },
  "dependencies": {
    "tslib": "^2.3.0"
  }
}
```

NB: une "**peerDependency**" est une **dépendance non transitive**.

Cela signifie ici que la librairie *d2f-ngx-commons* s'appuie sur la version 21.1.0 d'angular mais que la future application qui va utiliser *d2f-ngx-commons* pourra éventuellement être basée sur une autre version d'angular (tant que celle-ci reste compatible).

Autrement dit, l'application utilisatrice devra reformuler (à sa façon) ces dépendances non transitives.

my-ngx-util-libs-workspace/projects/d2f-ngx-commons/*ng-package.json*

```
{
  "$schema": "../../node_modules/ng-packagr/ng-package.schema.json",
  "dest": "../../dist/d2f-ngx-commons",
  "assets": ["/assets"],
  "lib": {
    "entryFile": "src/public-api.ts"
  }
}
```

my-ngx-util-libs-workspace/projects/d2f-ngx-commons/*src/public-api.ts*

```
/* Public API Surface of my-ngx-commons */

export * from './lib/d2f-ngx-commons.service';
export * from './lib/d2f-ngx-commons.component';
export * from './lib/d2f-ngx-commons.module';
export * from './lib/util/util';
export * from './lib/data/user-session';
export * from './lib/helper/object-helper';
export * from './lib/service/user-session.service';
...
```

Pour **anciennes librairies** avec des modules "angular" (avant époque standalone) :

my-ngx-util-libs-workspace/projects/d2f-ngx-commons/*src/lib/d2f-ngx-commons.module.ts*

```
import { NgModule } from '@angular/core';
import { D2fNgxCommonsComponent } from './d2f-ngx-commons.component';
```

```
@NgModule({
  declarations: [ D2fNgxCommonsComponent ],
  imports: [ ],
  exports: [ D2fNgxCommonsComponent ]
})
export class D2fNgxCommonsModule { }
```

Pour le reste , code angular classique (services , composants avec .ts ;,html,.css, classes de données ,) à comme d'habitude commencer via assistant "ng g"

Eventuelles liaisons .scss entre diverses librairies :

in each ng-package.json of d2f-ngx-xyz library (but not d2f-ngx-commons)

```
{
  ...,
  "lib": {
    "entryFile": "src/public-api.ts",
    "styleIncludePaths": [
      "../d2f-ngx-commons/assets/scss"
    ]
  }
}
```

in order to find .scss (not .css) to import during ng build of library

Unfortunately , .scss are transformed in static css during the phase of library building.

values of scss variables must be set at this stage , not after !

Point important sur inter-dépendances entre librairies :

Lorsque plusieurs librairies sont placées dans le même workspace , il est possible qu'une librairie réutilise certains styles ou composants d'une autre librairie.

Pour l'accès à un fichier css d'une autre librairie du même workspace, un import en relatif n'est pas idéal mais ça fonctionne. Exemple :

```
@import ".././.././.././d2f-ngx-commons/assets/css/d2f-common-dialog.css";
```

Pour l'accès à un composant angular (.ts , ...) d'une autre librairie du même workspace, **un import du package préalablement construit** via **ng build d2f-ngx-components** (et disponible dans dist/) fonctionne bien. Exemple :

```
import { MyMatTogglePanelComponent } from 'd2f-ngx-components';
```

4. Utilisation d'une librairie angular

Une fois publiée au sein d'un référentiel npm (public ou local), une librairie personnalisée (codée par nous même) s'utilise de la même manière qu'une librairie standard/prédéfinie.

Préparer si besoin le contexte npm (éventuel démarrage de verdaccio , npm set registry ,
npm login, npm whoami)

Dans autre appli "**my-app**" hors du workspace des librairies :

```
npm install -s d2f-ngx-commons
npm install -s d2f-ngx-components
npm install -s d2f-ngx-crud
npm install -s d2f-ngx-layout
```

Selon le format des éventuels styles en accompagnement (.css ou .scss) on ajoutera si besoin des configurations de ce genre dans **angular.json** (*pour importation globale*):

```
"styles": [
  "projects/test-app/src/styles.scss",
  "node_modules/@angular/material/prebuilt-themes/indigo-pink.css",
  "node_modules/d2f-ngx-commons/assets/css/d2f-common-layout.css"
  ou bien "projects/d2f-ngx-commons/assets/css/d2f-common-layout.css"
],
```

ou bien en mode .scss :

```
"stylePreprocessorOptions": {
  "includePaths": [
    "projects/d2f-ngx-commons/assets/scss/" ou bien
    "node_modules/d2f-ngx-commons/assets/scss"
  ]
},
```

NB : pour effectuer (sans saas) , des *@import locaux précis* depuis un composant applicatif angular il faut :

- du côté "librairie angular" avoir exporter le fichier css (dans **librairie-xyz/package.json**) via

```
"exports": {
  "./s1.css": "./assets/css/s1.css"
}
```

- du côté "projet utilisateur" utiliser cette syntaxe :

@import "librairie-xyz/s1.css";

Pour bien utiliser **tailwind css 4** à la fois dans une librairie angular et dans une application , il faut ajouter @source dans **src/styles.css** du projet "appli utilisatrice" :

```
/* @import "tailwindcss"; */
@import 'tailwindcss/theme';
@import 'tailwindcss/utilities';
@source "../node_modules/librairie_xyz";
```

Si nécessaire, pour prendre en compte des changements/mises à jour au niveau des librairies utilisées (sans changement de version officielle) :

```
npm update d2f-ngx-commons d2f-ngx-components ...
```

XVI - Annexe – OAuth2 avec angular

1. Sécurisation d'une application "angular"

1.1. Quelques considérations générales sur la sécurité

Même transformé de ".ts" en ".js", une application Angular n'est pas véritablement compilé, son code source est accessible et éventuellement sujet à interception / modification.

--> jamais d'éléments confidentiels dans le code de l'application (pas de "salt", pas de "default_password", ...)

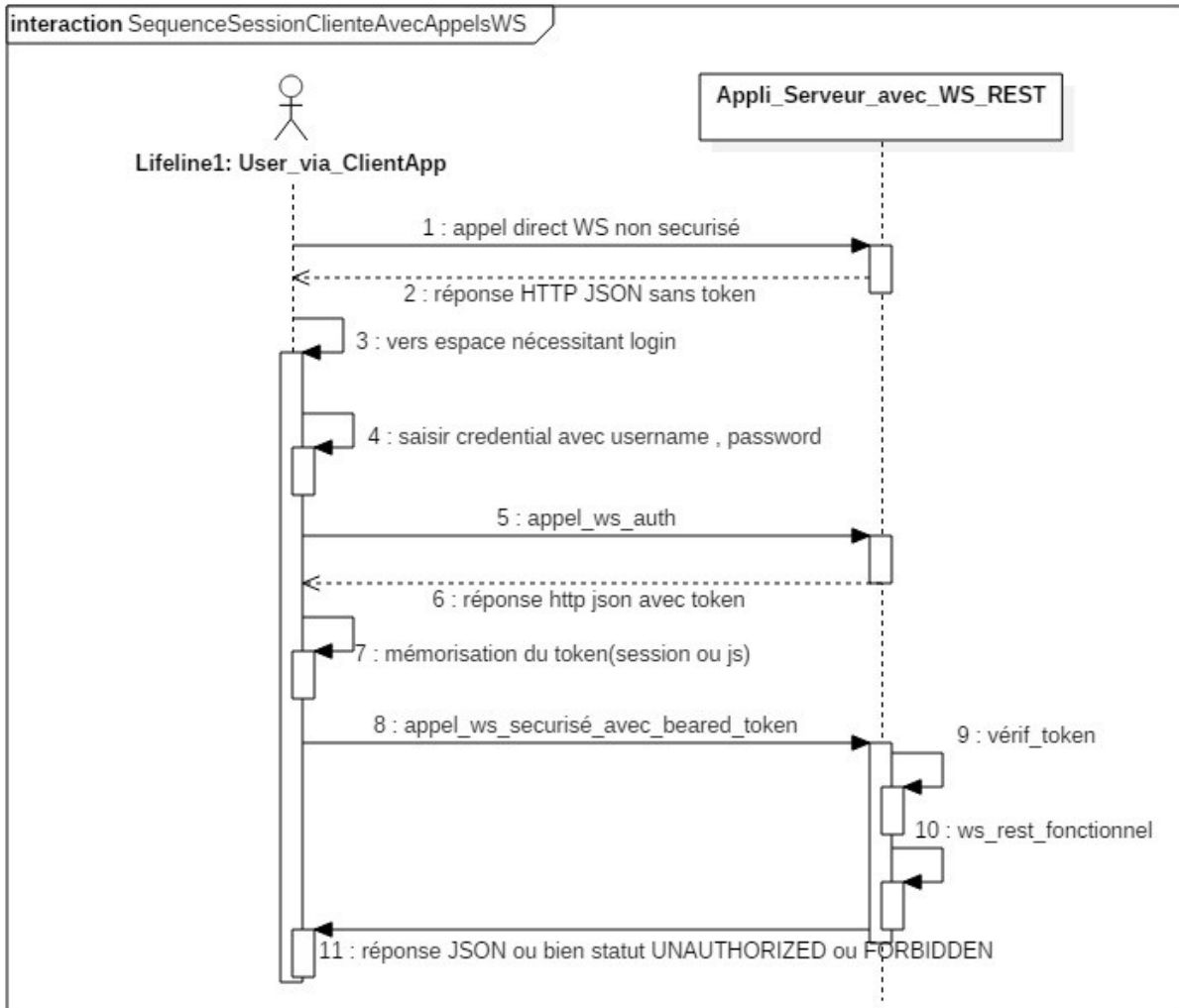
--> le côté serveur ne doit avoir qu'une confiance limitée aux requêtes angular.
Il est bon de vérifier fréquemment "token" et autres.

1.2. Conseils sur la structure d'une application angular sécurisée

- HTTPS / SSL dès qu'il faut échanger des informations confidentielles (username, password), ...
- Un service d'authentification
- Des gardiens pour certaines routes

2. Token pour appels aux Web-services REST

2.1. Pseudo session avec "token" plutôt que cookie :



Des jetons de sécurité ("token") sont généralement employés pour gérer l'authentification d'un utilisateur et d'une application dans le cadre d'une communication sous forme de WS-REST.

Le jeton de sécurité est généré si le couple (username,password) transmis est correctement vérifié coté serveur.

Ce "token" (véhiculé au format "string") pourra prendre la forme d'un uuid (universal unique id , exemple: e51cd176-a522-454c-9c0a-36ca74cdb2d0) ou bien être conforme au format JWT (Json Web Token).

Dans le cas d'un token sophistiqué de type jwt , le token généré comporte déjà en lui (de manière cryptée/extractible) certaines informations utiles (subject , roles ,) et donc pas besoin de map coté serveur.

Le protocole HTTP a normalisé la façon dont le token doit être retransmis au sein des requêtes

émises du client vers le serveur (après l'authentification préalablement effectuée) :

Il faut pour cela utiliser le champ "Authorization :" de l'entête HTTP pas en mode "Basic" mais en mode "Bearer" (signifiant "au porteur" en français).

exemple (postman) :

Authorization ●	Headers (1)	Body	Pre-request Script	Tests
Key		Value		
Authorization		Bearer e51cd176-a522-454c-9c0a-36ca74cdb2d0		
New key		Value		

Status: 200 OK

si ok

Status: 403 Forbidden

ou bien

si faux jeton (invalid token)

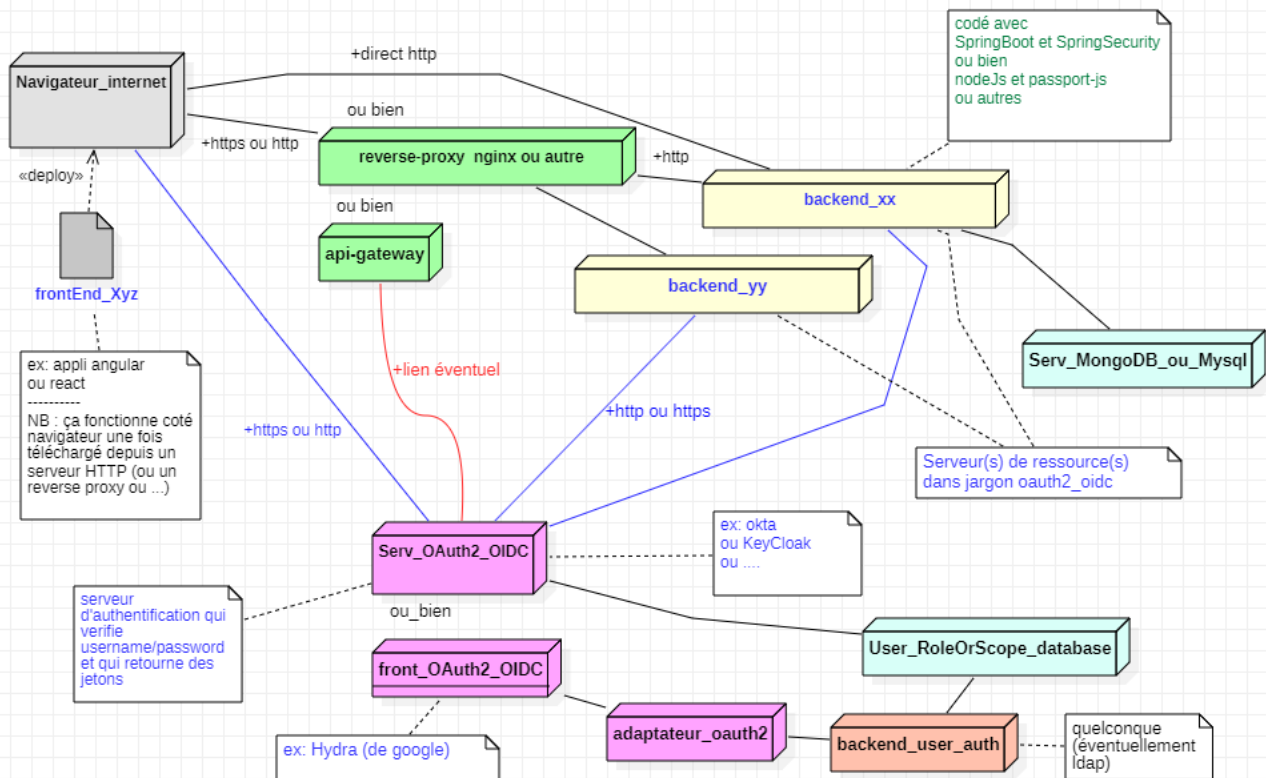
Status: 401 Unauthorized

ou bien

si pas de jeton

3. Authentification via OAuth2/OIDC

Authentification avec OAuth2 et/ou OIDC



- 1) utilisation de la partie publique du frontEnd
 - 2) le frontEnd (souvent dans une popup) délègue le login (et éventuels consentements) au serveur d'authentification OAuth2_OIDC
 - 3) le serveur OAuth2_OIDC présente un formulaire de login (avec éventuels choix de scopes) et retourne si ok des jetons après échange de code
 - 4) le frontEnd mémorise le ou les jeton(s) (dans session utilisateur ou sessionStorage ou autre)
 - 5) le frontEnd retransmet le jeton dans tous les appels de Web-Services privés ("bearer token" Http)
 - 6) le serveur "backend_xx_ou_yy" (appelé "serveur de ressources") communique si besoin avec le serveur OAuth2_OIDC pour vérifier la validité du jeton
 - 7) en fonction d'éventuels scopes (droits d'accès) contenus dans le jeton le serveur "backend_xx_ou_yy" accepte ou pas de répondre à la requête
- L'entreprise "Okta" propose un service "OAuth2_OIDC" clef en main à distance (en mode saas)
 Le serveur "Keycloak" (basé sur jboss-wildfly) est un serveur "OAuth2_OIDC" open source avec une console d'admin correcte
 Certaines extension de Spring servent à établir simplement une communication entre un serveur de ressources basé sur java/springBoot et un serveur d'authentification OAuth2/OIDC .
 Certaines extensions pour angular (telles que "angular-oauth2-oidc") servent à établir une délégation de login entre un frontEnd angular et un serveur d'authentification OAuth2/OIDC .

3.1. Accès à un serveur d'autorisation depuis angular

```
npm install angular-oauth2-oidc --save
```

src/silent-refresh.html

```
<html>
<body>
  <script>
    var checks = [/[\?|&|#]code=/, [/[\?|&|#]error=/, [/[\?|&|#]token=/, [/[\?|&|#]id_token=/];

    function isResponse(str) {
      if (!str) return false;
      for(var i=0; i<checks.length; i++) {
        if (str.match(checks[i])) return true;
      }
      return false;
    }

    var message = isResponse(location.hash) ? location.hash : '#' + location.search;

    if (window.parent && window.parent !== window) {
      // if loaded as an iframe during silent refresh
      window.parent.postMessage(message, location.origin);
    } else if (window.opener && window.opener !== window) {
      // if loaded as a popup during initial login
      window.opener.postMessage(message, location.origin);
    } else {
      // last resort for a popup which has been through redirects and can't use window.opener
      localStorage.setItem('auth_hash', message);
      localStorage.removeItem('auth_hash');
    }
  </script>
</body>
</html>
```

angular.json

```
...
"assets": [
  ...,
  "src/silent-refresh.html"
],
...
```

src/app/common/data/user_in_session.ts

```
export class UserInSession {
  constructor(public username = "",
              public authenticated = false,
              public roles = "",
              public grantedScopes : string[] = []) {}
}
```

Dans une application angular avec module, ajouter

OAuthModule.forRoot() dans la partie **imports[]** de **app.module.ts**

avec **import { OAuthModule } from 'angular-oauth2-oidc';**

En version "standalone" , ajouter **provideOAuthClient()** dans providers:[] de **app.config.ts** avec `import { provideOAuthClient } from 'angular-oauth2-oidc';`

src/app/common/service/session.service.ts

```
import { Injectable } from '@angular/core';
import { Router } from '@angular/router';
import { AuthConfig, OAuthErrorEvent, OAuthInfoEvent,
  OAuthService, OAuthSuccessEvent } from 'angular-oauth2-oidc';
import { UserInSession } from '../data/user_in_session';
import { Location } from '@angular/common';

@Injectable({
  providedIn: 'root'
})
export class SessionService {

  private _userInSession = new UserInSession();

  public get userInSession() { return this._userInSession; }

  public set userInSession(u: UserInSession) {
    this._userInSession = u;
    sessionStorage.setItem("session.userInSession", JSON.stringify(this._userInSession));
  }

  constructor(private oauthService: OAuthService, private router: Router, private location: Location) {
    this.initOAuthServiceForCodeFlow();
    let sUser = sessionStorage.getItem("session.userInSession");
    if(sUser) {
      this._userInSession = JSON.parse(sUser);
    }
  }

  initOAuthServiceForCodeFlow() {
    const authCodeFlowConfig: AuthConfig = {
      // Url of the Identity Provider
      issuer: 'https://www.d-defrance.fr/keycloak/realms/sandboxrealm',

      // URL of the SPA to redirect the user to after login
      // redirectUri: window.location.origin + "/ngr-loggedIn",

      silentRefreshRedirectUri: window.location.origin + this.location.prepareExternalUrl("/silent-refresh.html"),
      useSilentRefresh: true,

      postLogoutRedirectUri: window.location.origin + this.location.prepareExternalUrl("/ngr-logInOut"),
      // ou /ngr-welcome ou ...

      // The SPA's id. The SPA is registered with this id at the auth-server
      clientId: 'anywebappclient',
      // clientSecret if necessary (not very useful for web SPA)
      // dummyClientSecret is required if client not public (client authentication: on + credential in keycloak)
      // dummyClientSecret: 'DMzPzIV2OQAphSbR84D7ebwxjrUNBgq5',
      responseType: 'code',

      // set the scope for the permissions the client should request
      // The first four are defined by OIDC.
      // Important: Request offline_access to get a refresh token
      // The api scope is a usecase specific one
      scope: 'openid profile resource.read resource.write resource.delete',

      showDebugInformation: true,
    };
    this.oauthService.configure(authCodeFlowConfig);
```

```

this.oauthService.oidc = true; // ID_Token

this.oauthService.setStorage(sessionStorage);

this.oauthService.loadDiscoveryDocumentAndTryLogin();

this.oauthService.events.subscribe(
  event => {
    if (event instanceof OAuthSuccessEvent) {
      //console.log("OAuthSuccessEvent: "+JSON.stringify(event));
      console.log("OAuthSuccessEvent: "+event.type);
      this.manageSuccessEvent(event);
    }
    if (event instanceof OAuthInfoEvent) {
      // console.log("OAuthInfoEvent: "+JSON.stringify(event));
      console.log("OAuthInfoEvent: "+event.type);
    }
    if (event instanceof OAuthErrorEvent) {
      // console.error("OAuthErrorEvent: "+JSON.stringify(event));
      console.log("OAuthErrorEvent: "+event.type);
    } else {
      console.warn(event.type);
    }
  });

//end of initOAuthServiceForCodeFlow

manageSuccessEvent(event : OAuthSuccessEvent){
if(event.type=="token_received"){
  console.log("***** token_received *****")
  this._userInSession.authenticated = true;
  let grantedScopesObj : object = this.oauthService.getGrantedScopes();
  this._userInSession.grantedScopes =<any> grantedScopesObj;
  console.log("grantedScopes="+JSON.stringify(this._userInSession.grantedScopes));

  var claims : any = this.oauthService.getIdentityClaims();
  console.log("claims="+JSON.stringify(claims))
  if (claims) this._userInSession.username= claims.preferred_username + "("+ claims.name + ")";

  if(this.oauthService.silentRefreshRedirectUri != null){
    //this.router.navigateByUrl("/ngr-loggedIn");
    this.router.navigateByUrl("/ngr-welcome"); //with message form this SessionService if successuf login
  }
}
}

delegateOidcLogin(){
  this.oauthService.initLoginFlowInPopup();
}

oidcLogout(){
  this.oauthService.logOut(); //clear tokens in storage and redirect to logOutEndpoint
  //this.oauthService.revokeTokenAndLogout(); //warning : problems if no CORS settings !!!!

  //delete old cookies (oauth2/oidc/keycloak):
  document.cookie = "AUTH_SESSION_ID=; expires=Thu, 01 Jan 1970 00:00:00 UTC; path=/;";
  document.cookie = "AUTH_SESSION_ID_LEGACY=; expires=Thu, 01 Jan 1970 00:00:00 UTC; path=/;";
}

public get accessTokenString() : string {
  return this.oauthService.getAccessToken();
}
}

```

src/app/common/interceptor/my-auth.interceptor.ts

```
import { Injectable } from '@angular/core';
import { HttpRequest, HttpHandler, HttpEvent, HttpInterceptor } from '@angular/common/http';
import { Observable } from 'rxjs';

@Injectable()
export class MyAuthInterceptor implements HttpInterceptor {

  constructor() {}

  intercept(request: HttpRequest<unknown>, next: HttpHandler): Observable<HttpEvent<unknown>> {
    let access_token = sessionStorage.getItem('access_token'); //NB: access_token for angular-oauth2-oidc or ...
    if(access_token && access_token!="" ){
      const authReq = request.clone({headers:
        request.headers.set('Authorization', 'Bearer ' + access_token)});
      return next.handle(authReq);
    }
    else
      return next.handle(request);
    }
  }
}
```

+ configuration classique de l'intercepteur (voir chapitre HTTP/Appel REST)

log-in-out.component.ts

```
import { Component, OnInit } from '@angular/core';
import { SessionService } from '../common/service/session.service';

@Component({
  selector: 'app-log-in-out',
  templateUrl: './log-in-out.component.html',
  styleUrls: ['./log-in-out.component.scss']
})
export class LogInOutComponent {

  constructor(public sessionService : SessionService) {
  }

  public login() {   this.sessionService.delegateOidcLogin(); }

  public logout() {   this.sessionService.oidcLogout(); }
}
```

log-in-out.component.html

```
<a href="https://www.d-defrance.fr/keycloak/realms/sandboxrealm/.well-known/openid-configuration"
target="_new">openid-configuration</a> &nbsp;  <span class="redClass">(delete old cookies if necessary (login
error) !!!)</span> <br/>
LogInOut COMPONENT (login with OAuth2/OIDC keycloak server)<br/>
<p>
  <button (click)="login()" class="btn btn-primary">Login</button> &nbsp;  &nbsp;  
  <button (click)="logout()" class="btn btn-primary">Logout</button> &nbsp;  <span
class="redClass">(if logout error, go back and try login)</span>
</p>
```

footer.component.html

```
...
username=<b>{{sessionService.userInSession.username}}</b> &nbsp;
authenticated=<b>{{sessionService.userInSession.authenticated}}</b><br/>
grantedScopes=<b>{{sessionService.userInSession.grantedScopes}}</b>
```

my-app [welcome](#) [basic](#) [userAccount](#) [conversion](#) [oauth2/oidc](#) [loginOut](#) [devise \(crud\)](#)

[openid-configuration](#) (delete old cookies if necessary (login error) !!!)

LoginOut COMPONENT (login with OAuth2/OIDC keycloak server)

Login

Logout

(if logout error, go back and try login)

examples of good Username/Password:

- mgr1/pwd1 (MANAGE_RW : resource.read , resource.write)
- admin1/pwd1 (ADMIN_CRUD : resource.read , resource.write , resource.delete)
- user1/pwd1 (USER : resource.read)

[direct/standalone login \(without oauth2/oidc server\)](#)

footer - couleurFondPreferee: username=? authenticated=**false**
roles=? grantedScopes=

my-app [welcome](#) [basic](#) [userAccount](#) [conversion](#) [oauth2/oidc](#) [loginOut](#) [devise \(crud\)](#)

[openid-configuration](#) (delete old cookies if necessary (login error) !!!)

LoginOut COMPONENT (login with OAuth2/OIDC keycloak server)

Login

examples of good Username/Password:

- mgr1/pwd1 (MANAGE_RW : resource.read , resource.write)
- admin1/pwd1 (ADMIN_CRUD : resource.read , resource.write , resource.delete)
- user1/pwd1 (USER : resource.read)

[direct/standalone login \(without oauth2/oidc server\)](#)

footer - couleurFondPreferee:

roles=? grantedScopes=

footer - couleurFondPreferee: username=**admin1(jean Bon)** authenticated=**true**
roles=? grantedScopes=**openid,profile,resource.delete,resource.read,email,resource.write**

XVII - Annexe – PWA (Progressive Web App)

1. PWA (Progressive Web App) – aperçu général

1.1. Présentation des "progressive web apps"

Les "**Progressive Web Apps**" (PWA) sont des **applications web modernes (html/css/js)** , **pouvant fonctionner** en mode "**hors connexion**" et axées sur les **mobiles** .

Ces applications "PWA" sont censées rivaliser avec des applications mobiles (natives ou hybrides) .

Quelques comparaisons :

	Développement	Exécution
Application mobile native (ex : java pour Android)	Langage et api spécifique à une plateforme mobile (ex : java et android-sdk ou bien ios/iphone/swift)	application mobile (à télécharger depuis un "store") et à exécuter sur un type précis de smartphone (android ou iphone)
Application mobile hybride (ex : <i>cordova</i> et/ou <i>ionic</i>)	Code source en html/js relativement portable (android , iphone, ...) mais nécessitant une étape de construction qui est moyennement simple avec android et délicate avec ios/iphone	même environnement d'exécution et comportement qu'une application native (avec néanmoins des performances et fonctionnalités quelquefois un peu plus limitées)
PWA (Progressive Web App)	Code source en html/css/js très portable ne nécessitant pas de phase de construction complexe et dépendante d'une plateforme mobile	Une "pwa" s'exécute au sein d'un navigateur de smartphone mais avec le "plein écran" possible et d'autres spécificités. Les fonctionnalités techniques d'une "pwa" seront donc liées aux versions (idéalement récentes) des navigateurs.

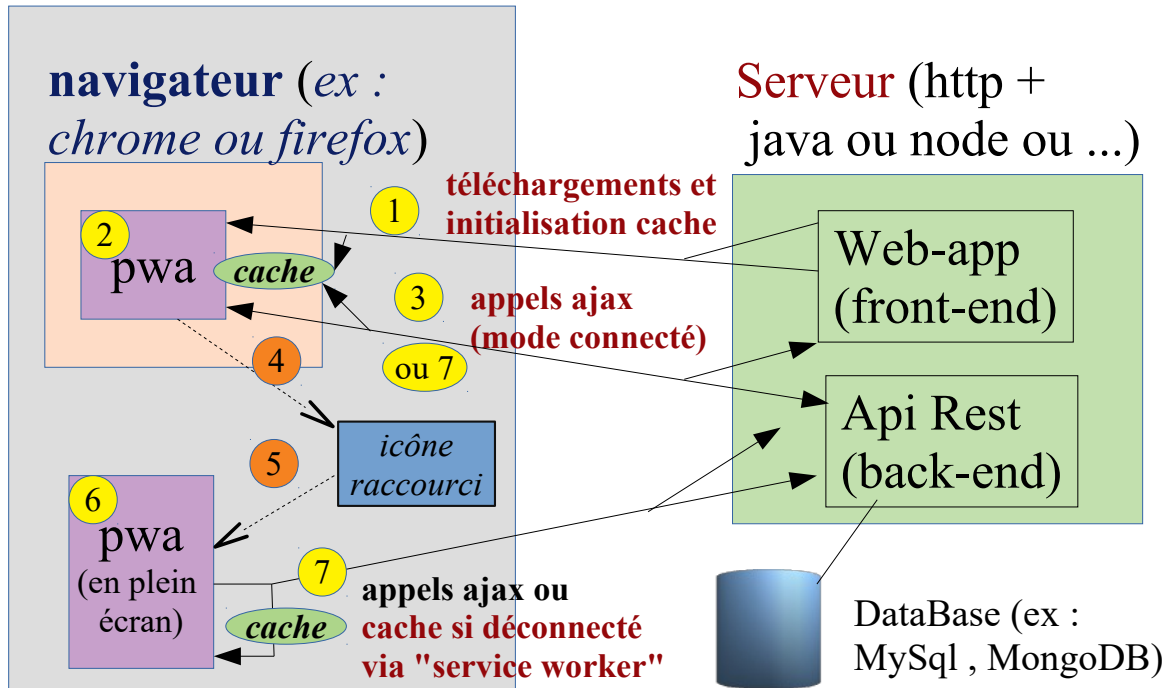
Le concept de "pwa" est très moderne et pour l'instant assez peu répandu en 2018/2019 . L'essor prévu des "pwa" dépendra essentiellement du support offert par les navigateurs des smartphones . Pour l'instant "android , chrome , firefox" avancent clairement dans cette direction . Apple (ios/iphone) comporte un navigateur supportant partiellement les "pwa" . On verra dans l'avenir les choix stratégiques d'Apple (ouverture ou fermeture) .

Dans "Progressive Web App" le terme "progressif" signifie que l'utilisateur d'un smartphone peut de manière très progressive :

- **navigation sur internet et utiliser l'application web comme un site ordinaire**
- utiliser potentiellement l'application en mode "**plein écran**" (comme une appli mobile)
- **générer un "icône/raccourci" de lancement sur le fond du bureau** (pour relancer ultérieurement cette appli en mode "navigateur pas encore ouvert")
- **utiliser certaines parties de l'application en mode "déconnecté"** grâce à des "*service-workers*" (tâches de fond pouvant faire office de "*cache*" ou "*proxy*")
- **finalement utiliser l'application web aussi facilement que si elle était native** et *disposer en prime d'une réactualisation (mise à jour) automatique du code* (vers la dernière version en date).

PWA (Progressive Web App)

smartphone (ex : android)



1.2. Fonctionnalités d'une "progressive web app"

Une "progressive web app" est avant tout une bonne "web app" s'utilisant bien sur divers navigateurs (PC/desktop, smartphone, tablettes) et qui a en plus certaines fonctionnalités spécifiques au contexte mobile (en mode quelquefois déconnecté).

Fonctionnalités d'une bonne "web app" :

- **Responsive**
- Lightweight
- Géolocalisation (via navigator.geolocation HTML5)
- etc (intuitive, ergonomique, efficace ,)

Fonctionnalités *supplémentaires* d'une "progressive web app" :

- **Web App Manifest** (Add to Home Screen)
- **Service Worker**
 - *cache*
 - *notifications*
- ...

1.3. Paramétrage fondamental pour "responsive" sur smartphone

index.html

```
<html>
<head>
<title>my pwa</title>
<link rel="stylesheet" href="/css/bootstrap.min.css">
<link rel="stylesheet" type="text/css" href="css/styles.css" />
<link rel="icon" href="/img/pwa-icon_48_48.png" /> <!-- favicon.ico by default -->
<link rel="manifest" href="manifest.json" />
<base href="/" />
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <!-- for responsive bootstrap on mobile device -->
</head>
...
</html>
```

La ligne

```
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
```

est indispensable pour obtenir un bon comportement "responsive" sur un appareil mobile (ex : smartphone ou tablette) .

Sans celle-ci , ce qui fonctionne bien au sein d'un navigateur sur PC/desktop , ne fonctionne malheureusement pas bien sur un téléphone "android" (c'est tout petit ou bien ça déborde) .

1.4. Quelques exemples d'applications web en mode "PWA"

La plupart des sites web d'actualités fonctionnent aujourd'hui en mode "pwa" (avec cependant plus ou moins de soins apportés au mode "off-line" quelquefois inopérant) :

- le figaro
- le monde
- ...

1.5. Emulateur android permettant de visualiser les effets "pwa" :

- ~~blueStack 4 ne fonctionne pas bien en mode "pwa"~~
- l'émulateur android proposé par défaut avec "android studio" fonctionne bien
-

2. Web App Manifest / add to home screen

2.1. Génération d'un icône "raccourci" en fond d'écran

De façon à ce qu'un navigateur récent (fonctionnant sur un mobile) puisse proposer la génération d'un icône raccourci sur le fond du bureau, il faut référencer un fichier manifest.json (décrivant l'image de l'icône dans différentes tailles).

Ce fichier appelé "web app manifest" contiendra quelques informations complémentaires (nom de l'application,).

2.2. Fichier "web app manifest"

Le manifest JSON offre un moyen de décrire un point d'entrée de l'application, afin de proposer à l'utilisateur une expérience d'application native lors du prochain lancement de la PWA :

- Icône dédiée sur la Home (lien URL)
- Lancement en plein écran,

Ce fichier est à déposer à la racine du serveur (souvent placé à coté du Service Worker).

Exemple :

manifest.json

```
{ "name": "My Progressive Web App",
  "short_name": "MyPWA",
  "start_url": "/index.html",
  "icons": [
    {
      "src": "img/pwa-icon_48_48.png",
      "sizes": "48x48",
      "type": "image/png"
    },
    {
      "src": "img/pwa_72_72.png",
      "sizes": "72x72",
      "type": "image/png"
    },
    {
      "src": "img/pwa-cube_96_96.png",
      "sizes": "96x96",
      "type": "image/png"
    },
    {
      "src": "img/pwa-cube_144_144.png",
      "sizes": "144x144",
      "type": "image/png"
    },
    {
      "src": "img/pwa-icon_192_192.png",
      "sizes": "192x192",
      "type": "image/png"
    }
  ],
  "display": "standalone",
  "background_color": "#4e8ef7",
  "theme_color": "#4e8ef7"
}
```

Ce fichier doit être référencé au niveau de **index.html**

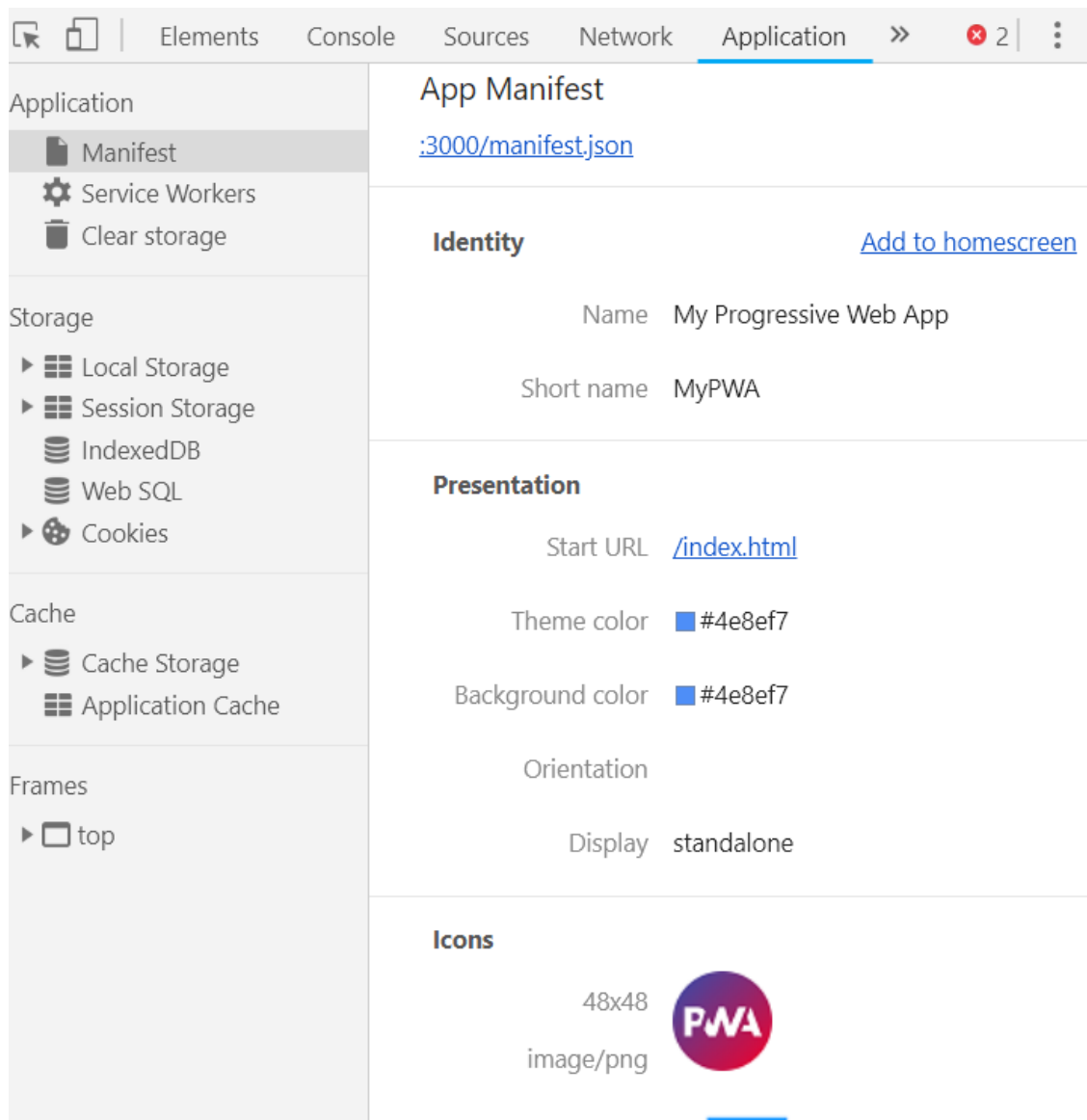
```
<!DOCTYPE html>
```

```

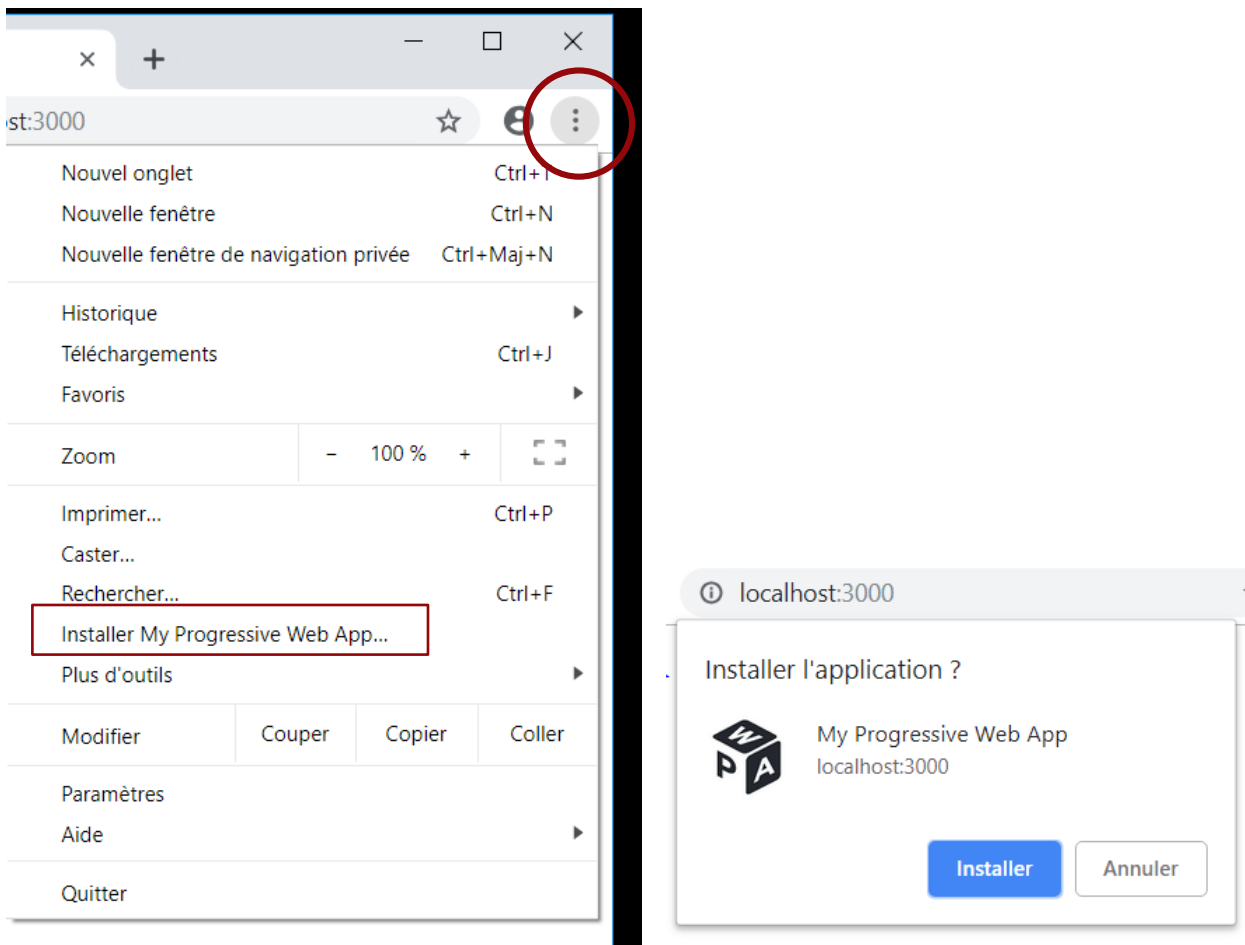
<html> <head>
  <meta charset="UTF-8">
  <title>My progressive Web App</title>
  <link rel="manifest" href="manifest.json">
</head>
<body>    ... </body> </html>

```

Reconnaissance du fichier "*web app manifest*" au sein de la partie "*Application*" de "*outils de développement*" du navigateur "Chrome Desktop" :



2.3. Effets au sein du navigateur "Chrome Desktop" (tests)



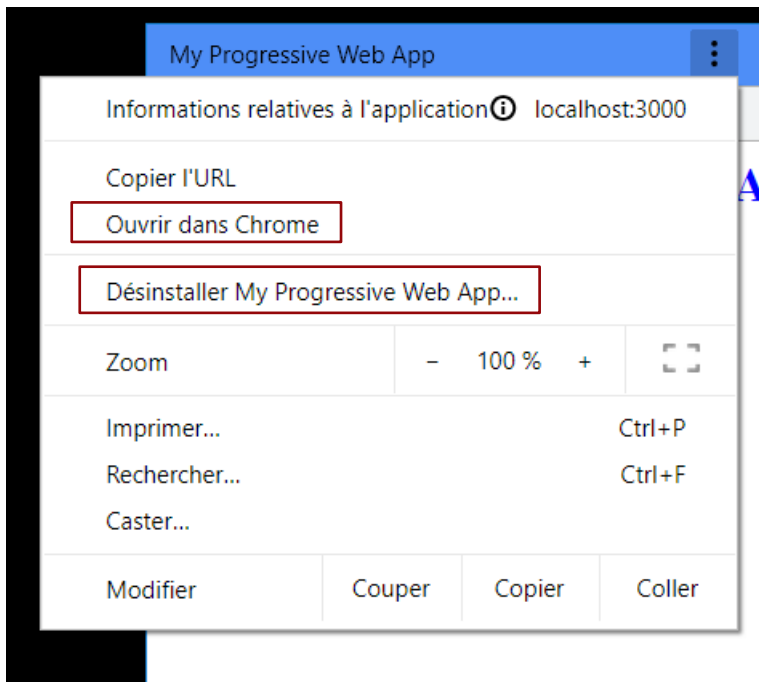
==> cette action génère un icône raccourci sur le fond du bureau :



En cliquant sur ce raccourci, l'application se lance dans une fenêtre spéciale (proche plein-écran) et avec les mêmes fonctionnalités que le navigateur "Chrome" habituel.



Cette application "pwa" comporte un menu contextuel permettant éventuellement la désinstallation de l'application ou bien le lancement de celle ci au sein du navigateur "Chrome" en mode habituel "multi-sites" .



2.4. Attention , pas de "add to home screen" sans service-worker enregistré et gérant les événements "install" et "fetch"

En début de développement (ou de "tp") , il faudra au minimum prévoir le code suivant pour que le navigateur propose le menu "add to home screen" ou "installer la pwa" ou "page/créer un raccourci"

dans *index.html* (ou *client.js*) :

```
if ('serviceWorker' in navigator) {
  window.addEventListener('load', function() {
    navigator.serviceWorker.register('service-worker.js')
      .then(() => navigator.serviceWorker.ready)
      .then(function(registration) {
        // Registration was successful
        console.log('ServiceWorker registration successful with scope: ', registration.scope);
      }, function(err) {
        // registration failed :
        console.log('ServiceWorker registration failed: ', err);
      });
  });
}
```

dans *service-worker.js*:

```
self.addEventListener('install', function(event) {
  console.log('[ServiceWorker] install ***');
```

```
});  
  
self.addEventListener('activate', function(event) {  
    console.log('[ServiceWorker] activate ***');  
});  
  
self.addEventListener('fetch', function(event) {  
    console.log('[ServiceWorker] fetch ***');  
});
```

2.5. Effets au sein du navigateur "Chrome pour android"

Lorsqu'une application web est reconnue comme progressive (webmanifest + service-worker + ...), l'entrée "add to home screen" (ou bien "installer l'application ...") apparaît dans le menu principal du navigateur.

2.6. Effets au sein du navigateur "Firefox pour android"

Lorsqu'une application web est reconnue comme progressive (webmanifest + service-worker + ...), l'entrée "page / ajouter un raccourcis" (ou bien "...") apparaît dans le menu principal du navigateur. En outre, un icône "maison +" apparaît quelquefois pour signifier un "add to home screen" possible en cliquant dessus.

2.7. Lien ou bouton facilitant l'installation (A2HS)

Pour inviter l'utilisateur à installer l'application en mode A2HS (Add To Home Screen), il est possible d'ajouter un lien ou bouton qui sera pris en compte par différents navigateurs mobiles (ex : "Chrome pour android", "Firefox pour android").

Ce bouton servira simplement à inviter l'utilisateur (via javascript) à gérer l'événement **"beforeinstallprompt"** prévu pour demander l'installation en fond de bureau.

Ce bouton soit idéalement être invisible avant la réception de l'événement.

Exemple de code :

```
<button class="add-button">Add to home screen</button>
```

```
.add-button {
  position: absolute;
  top: 1px;
  left: 1px;
}
```

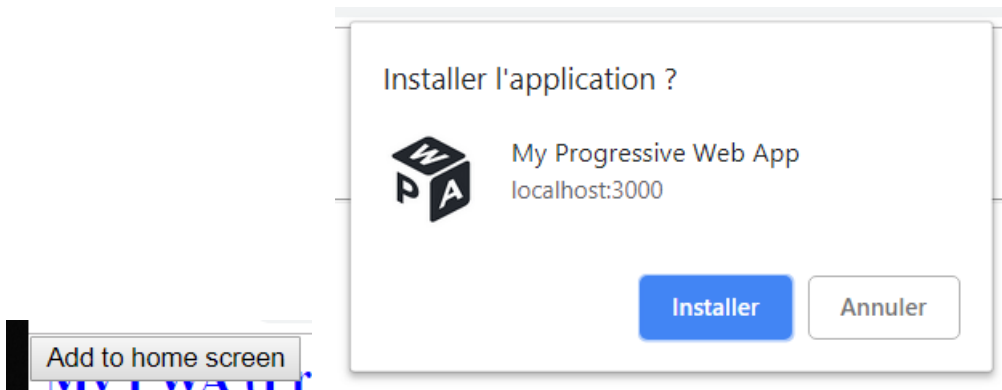
dans un bloc de script en fin de partie "body" :

```
let deferredPrompt;
const addBtn = document.querySelector('.add-button');
addBtn.style.display = 'none';

window.addEventListener('beforeinstallprompt', (e) => {
  // Prevent Chrome 67 and earlier from automatically showing the prompt
  e.preventDefault();
  // Stash the event so it can be triggered later.
  deferredPrompt = e;
  // Update UI to notify the user they can add to home screen
  addBtn.style.display = 'block';

  addBtn.addEventListener('click', (e) => {
    // hide our user interface that shows our A2HS button
    addBtn.style.display = 'none';
    // Show the prompt
    deferredPrompt.prompt();
    // Wait for the user to respond to the prompt
    deferredPrompt.userChoice.then((choiceResult) => {
      if (choiceResult.outcome === 'accepted') {
        console.log('User accepted the A2HS prompt');
      } else {
        console.log('User dismissed the A2HS prompt');
      }
      deferredPrompt = null;
    });
  });
});
```

Effets (en mode "pré-test") avec "Chrome-Desktop" sous windows :



NB :

L'événement "*beforeinstallprompt*" ne sera déclenché par un *navigateur mobile récent* que si les conditions suivantes sont réunies :

- L'appli (PWA) n'est pas déjà installée
- "user engagement heuristic" (l'utilisateur a activement utilisé l'appli au moins 30s)
- L'appli (pwa) comporte le fichier "web app manifest".
- L'appli (pwa) est servie par un serveur sécurisé (https).
- Au moins un "service worker" est enregistré avec un gestionnaire pour l'événement fetch .

Nb : A des fin de "pré-test" avec "Chrome-desktop" on pourra éventuellement (en http et pas https) , rendre visible le bouton dès le début (pas de `addBtn.style.display = 'none'`) .

3. Service-worker et pwa pour Angular

Dès les versions 6, 7 du framework "Angular" de Google , l'intégration des fonctionnalités "service-worker" et "pwa" est bien prise en charge via l'extension "NGSW" / "*@angular/pwa*" .

Pour tester les aspects "pwa" et "service-worker" du framework Angular , il faudra idéalement:

- effectuer les bonnes configuration (génération via `ng add @.../pwa` , édition `ngsw-config.json`,)
- construire une version de production via "`ng build`"
- installer l'application construite sur un serveur http (ex : nginx ou http-server ou ...)
- effectuer des tests via "Chrome Desktop" ou depuis un navigateur mobile (fonctionnant par exemple sur un système Android réel ou simulé/émulé) .

3.1. initialisation "pwa / sw" appli angular

```
ng add @angular/pwa
```

Cette commande (de angular-cli) permet d'ajouter tout un tas d'éléments "pwa" et "service-worker" à une application angular :

- ajout du package *@angular/service-worker* dans *package.json* et configurations associées

- ajout de **manifest.webmanifest** à la page *index.html* (+ icônes associés dans *public/icons*)
- création du fichier de configuration **ngsw-config.json** (paramétrage des caches)
- ajout de **"serviceWorker": "ngsw-config.json"** dans la partie "configurations/production" de **angular.json** permet de préciser qu'il faudra utiliser un fichier *ngsw-worker.js* ou autre comme worker en mode production avec le fichier de paramétrage *ngsw-config.json*.

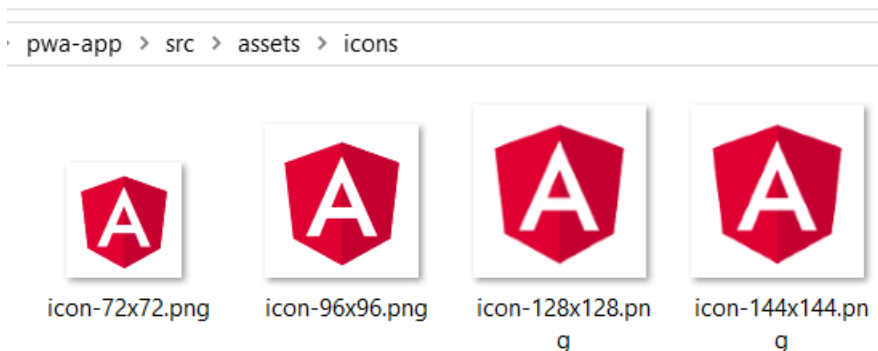
Exemple de dépendance "npm" ajoutée dans **package.json** :

```
"dependencies": {  
  ...  
  "@angular/service-worker": "^21.1.0"  
}
```

exemple de fichier **manifest.webmanifest** généré :

```
{  
  "name": "myapp",  
  "short_name": "myapp",  
  "theme_color": "#1976d2",  
  "background_color": "#fafafa",  
  "display": "standalone",  
  "scope": "/",  
  "start_url": "/",  
  "icons": [  
    {  
      "src": "assets/icons/icon-72x72.png",  
      "sizes": "72x72",  
      "type": "image/png"  
    },  
    ...  
    {  
      "src": "assets/icons/icon-512x512.png",  
      "sizes": "512x512",  
      "type": "image/png"  
    }  
  ]  
}
```

maintenant dans "public" :



exemple de lien ajouté dans index.html :

```
<link rel="manifest" href="manifest.webmanifest">
...
```

Éléments quelquefois ajoutés (à l'ancienne) dans *app.module.ts*:

```
...
import { ServiceWorkerModule } from '@angular/service-worker';

@NgModule({...
imports: [
...
ServiceWorkerModule.register('/ngsw-worker.js',
                             { enabled: environment.production })
],
```

Configuration équivalente plus moderne en mode "standalone" :

app.config.ts

```
import { provideServiceWorker } from '@angular/service-worker';
...
provideServiceWorker('ngsw-worker.js', {
  enabled: !isDevMode(), registrationStrategy: 'registerWhenStable:30000' })
...
```

3.2. Configuration du cache (ngsw-config.json)

Le fichier *src/ngsw-config.json* (pris en compte lors de la construction d'une application via **ng build**) permet de préciser quels sont les éléments à placer en cache et les stratégies de mise à jour.

Dans ce fichier les chemins doivent commencer par "/" et seront interprétés en relatif vis à vis de la racine des fichiers sources (placés dans src puis déplacés dans les bundles de production) .

Sauf indication contraire, les expressions des chemins sont basés sur le format "**glob**" (pas de regexp):

- ****** matches 0 or more path segments.
- ***** matches 0 or more characters excluding /.
- **?** matches exactly one character excluding /.
- The **!** prefix marks the pattern as being negative, meaning that only files that don't match the pattern will be included.

Example patterns:

- **/**/* .html** specifies all HTML files.
- **/*.html** specifies only HTML files in the root.
- **!/**/* .map** exclude all sourcemaps.

Groupes d'éléments en cache :

assetGroups	fichiers statiques (pour une version précise de l'application) , ex : icônes , images , ... , avec stratégies " <i>prefetch</i> " ou " <i>lazy</i> "
dataGroups	fichiers (souvent "json") de données fabriqués dynamiquement (souvent via des web services "rest") . paramétrages de type " <i>maxAge</i> " , ...

Modes d'installation en cache (pour un des assetGroups):

prefetch : dès le début (au premier démarrage de l'application) , les ressources sont téléchargées et stockées en cache .

lazy : au fur et à mesure des requêtes déclenchées . Les ressources ne sont placées en cache que si elles ont été téléchargées via le fonctionnement normal de l'application piloté par l'utilisateur .

La valeur par défaut de "*installMode*" est "*prefetch*" .

Modes de mise à jour du cache (pour un des assetGroups):

prefetch : dès le début (au premier rechargement de l'application modifiée) , les ressources modifiées sont téléchargées et stockées en cache .

lazy : au fur et à mesure des requêtes déclenchées . Les ressources ne sont remplacées en cache que si elles ont été téléchargées via le fonctionnement normal de l'application piloté par l'utilisateur .

La valeur par défaut de "*updateMode*" est la valeur de "*installMode*" .

Expression des chemins :

files=... pour les éléments internes à l'application (ex : icônes et petites images dans assets)

urls=... pour les éléments externes à l'application (ex : CDN pour css, fonts, ... et images téléchargées via http)

Paramétrages pour un des dataGroups :

maxSize	nombre maxi d'éléments (réponses) stockés en cache . éviction si taille dépassée
maxAge	durée maxi de validité en cache (ex : 3d12h)
timeout (paramètre facultatif)	durée d'attente d'une réponse réseau (en mode "online" dégradé) avant d'utiliser le contenu du cache en plan B (ex : 5s30u)
strategy	"performance" (pour données évoluant peu) ou "freshness" (pour données importantes à rafraîchir souvent)

unités :

- d: days
- h: hours
- m: minutes
- s: seconds
- u: milliseconds

Exemple de fichier src/ngsw-config.json :

```
{
  "index": "/index.html",
  "assetGroups": [
    {
      "name": "app",
      "installMode": "prefetch",
      "resources": {
        "files": [
          "/favicon.ico",
          "/index.csr.html", "/index.html", "/manifest.webmanifest",
          "/*.css",
          "/*.js"
        ]
      }
    }, {
      "name": "assets",
      "installMode": "lazy",
      "updateMode": "prefetch",
      "resources": {
        "files": [
          "**/*.(svg|cur|jpg|jpeg|png|apng|webp|avif|gif|otf|ttf|woff|woff2)"
        ]
      }
    }
  ]
}
```

ajouts possibles mais complexes à tester/optimiser/... :

```
{
  ...
  "dataGroups": [
    {
      "name": "xy-api",
      "urls": [
        "https://www.mycompany.com/xy"
      ],
      "cacheConfig": {
        "maxSize" : 10000,
        "maxAge" : "7d" ,

```

```

    "strategy" : "freshness" ,
    "timeout" : "5s"
  }, {
    .... (avec "strategy" : "performance" et autre(s) url(s) )
  }
]
}

```

3.3. Mise en "pseudo-production" pour effectuer un test

installation (en mode global) via npm du serveur http-server :

```
npm install -g http-server
```

Génération de l'appli avec ses bundles de production dans le répertoire "dist" :

```
ng build
```

Lancement du serveur http avec la prise en charge de l'application angular :

```
http-server -c-1 dist/myapp/
```

ou bien

```
http-server -c-1 dist/myapp/browser
```

NB : l'option **-c-1** signifie "désactiver les caches coté serveur http"

l'url par défaut est <http://localhost:8080> ou <http://localhost:8080/index.csr.html>

l'option `--proxy http://localhost:8282 ./api-rest-xy` permet (si besoin) de configurer une redirection vers une api rest

Attention : en réelle production, le service worker ne fonctionne normalement bien qu'avec HTTPS (et pas http) .

En phase de test , cette limitation est quelquefois levée sur localhost .

3.4. Test partiel direct avec angular cli :

```
ng serve --configuration=production
```

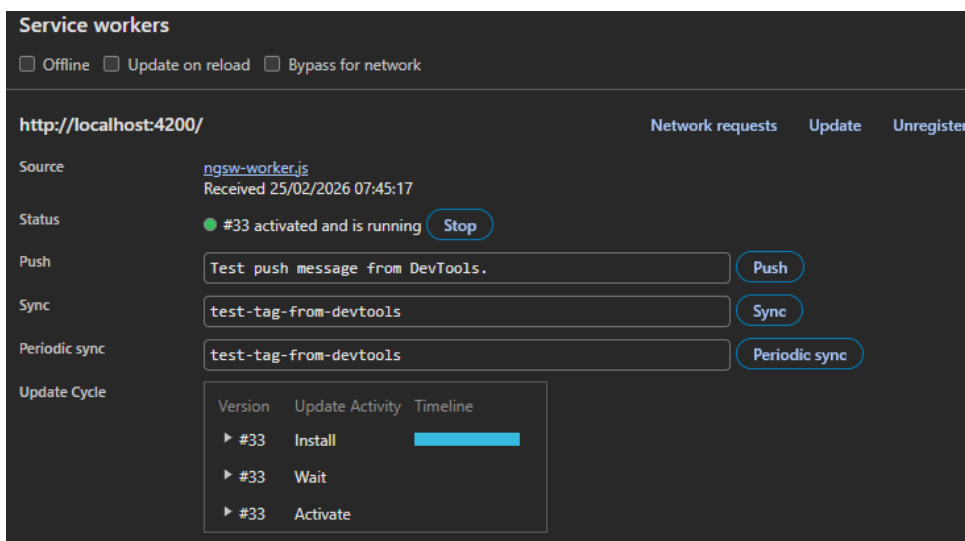
Attention : quelquefois , d'autres technologies vont mettre la pagaille dans les fonctionnalités de **@angular/pwa** (ex : ssr ou postcss ou autre).

Par exemple, le fichier **ngsw.json** (généré à partir de **ngsw-config.json**) comporte une partie **"hashTable"** avec des valeurs qui doivent restées stables . Autrement dit , les fichiers générés lors du build ne doivent plus être re-modifiés par la suite .

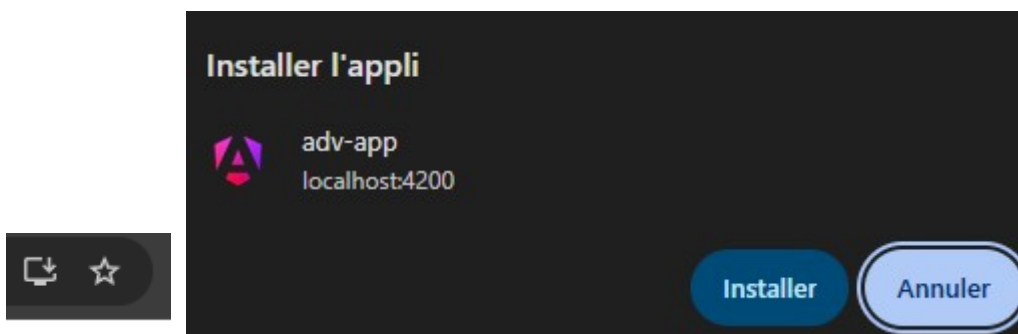
3.5. Test du comportement "off-line" pwa

1. **Première visite** : Ouvrez l'application dans votre navigateur. Le service worker va mettre en cache les fichiers fondamentaux (HTML, JS, CSS, etc.).
2. **Vérifier l'enregistrement** : Ouvrez les outils de développement, allez dans l'onglet "Application", puis "Service Workers". Vous devriez voir votre service worker enregistré.
3. **Tester offline** : Dans les outils de développement, activez l'option "Offline" dans l'onglet "Network". Rechargez la page. L'application doit toujours afficher son contenu — elle fonctionne "offline", grâce au cache.
4. **Tester l'installation** : Sur mobile ou dans Chrome desktop, vous devriez voir une icône d'installation dans la barre d'adresse. Cliquez dessus pour installer l'application sur l'écran d'accueil.

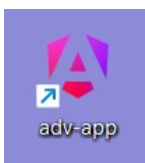
Vue des services-workers depuis la console du navigateur chrome :



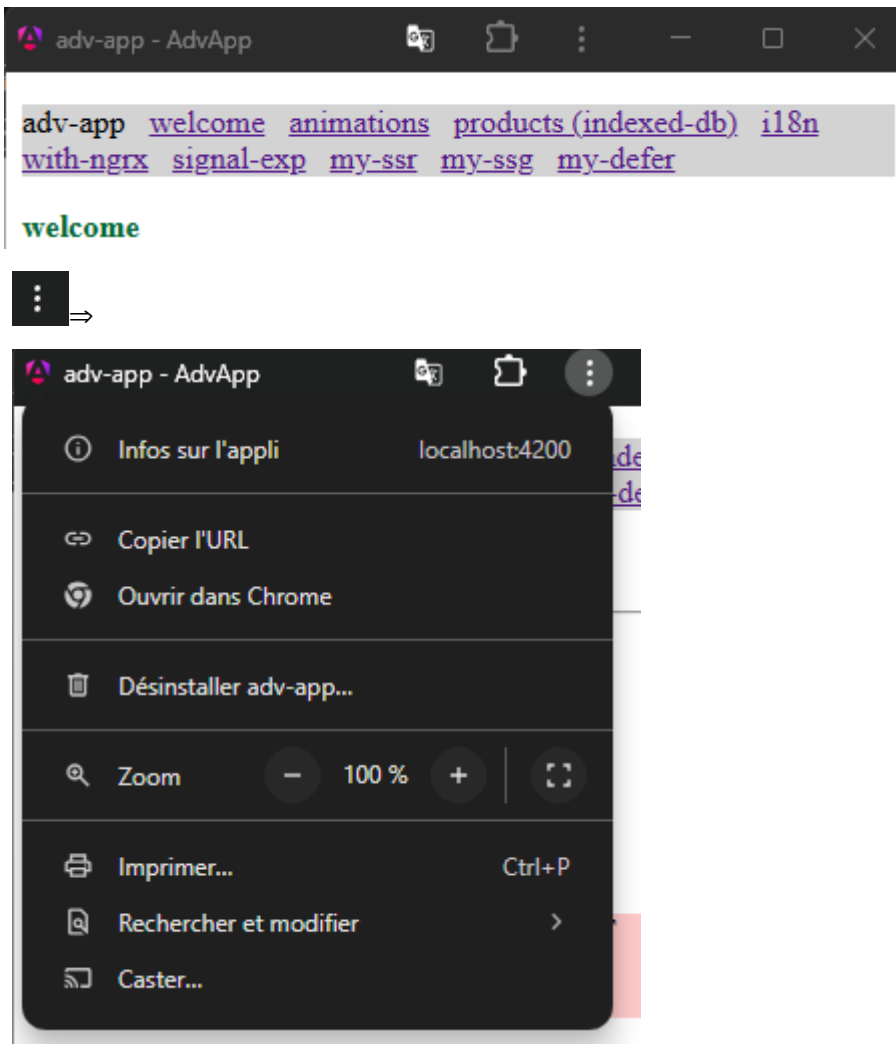
Proposition d'installation de l'application (depuis navigateur chrome) :



⇒ nouvel icône sur le fond du bureau du PC ou du smartphone



et le navigateur bascule en mode "appli"



Remarque : l'installation "icône, ..." fonctionne plutôt mieux avec Chrome qu'avec firefox

3.6. Notifications sur mises à jour disponibles et activées

```
import { inject, Injectable } from '@angular/core';
import { SwUpdate } from '@angular/service-worker';

@Injectable({
  providedIn: 'root',
})
export class SWUpdateLogService {

  private swUpdate = inject(SwUpdate);

  constructor() {

    this.swUpdate.versionUpdates.subscribe((evt) => {

      switch (evt.type) {
        case 'VERSION_DETECTED':
          console.log(`Downloading new app version: ${evt.version.hash}`);
          break;
        case 'VERSION_READY':
          console.log(`Current app version: ${evt.currentVersion.hash}`);
          console.log(`New app version ready for use: ${evt.latestVersion.hash}`);
          break;
        case 'VERSION_INSTALLATION_FAILED':
          console.log(`Failed to install app version '${evt.version.hash}': ${evt.error}`);
          break;
      }

    });

  }

}
```

Ce service est à injecter dans au moins un composant (ex : welcome).

Cela permet surtout de comprendre ce qui se passe quelquefois pas bien :

```
Failed to install app version '2f1ae2ac3ea307d3f73973df0ef529f82ef5b079': Hash mismatch (cacheBustedF
http://localhost:4200/main-UFUXCWUK.js: expected 04cc6acdc29f1eb481798e4564d3ccc25cfdeb36, got 1b7085
busting)
Error: Hash mismatch (cacheBustedFetchFromNetwork): http://localhost:4200/main-UFUXCWUK.js: expected
1b7085aaaaad1f00cf0ec8c45f8cbcab7280942e3 (after cache busting)
```

Autres fonctionnalités de ce genre à étudier sur le site de référence d'angular

<https://angular.io/guide/service-worker-communications>

XVIII - Annexe – extension @angular/material

1. Angular-Material (librairie de composants)

Angular-Material est une librairie de composants graphiques qui sont

- destinés à être intégrés au framework **angular**
- basés sur le look "**material**" (projet transversal mis en avant par "google" entre autres)

Angular-Material offre des composants intéressants tels que les onglets , les boîtes de dialogue , ...

Ces composants sont pour certains agrémentés d'un redimensionnement automatique (comportement "responsive").

Angular-material est un concurrent direct de "*ngx-bootstrap*" et "*primeNg*".

NB : La plupart des composants de "angular-material" ne gèrent que très peu l'aspect "disposition / placement" . On a souvent besoin d'une technologie complémentaire pour cela .

Bien qu'étant facultatif , le complément angular "**flex-layout**" est souvent utilisé en accompagnement de "angular-material" .

Remarque : Bien que pas très conseillée pour éviter des juxtapositions de looks différents et hétérogènes , une utilisation conjointe/complémentaire de "angular-material" et d'une autre librairie de composants (telle que ngx-bootstrap) est techniquement possible . Cette idée a d'ailleurs été mise en oeuvre au sein d'un projet (existant mais peu utilisé) baptisé "....." .

1.1. intégration de "angular-material" au sein d'un projet angular

```
ng add @angular/material
```

et facultativement :

```
npm install -s @angular/flex-layout
```

Effet dans package.json (exemple):

```
...
"dependencies": {
  "@angular/animations": "^8.2.14",
  "@angular/cdk": "^8.2.3",
  ... ,
  "@angular/flex-layout": "^8.0.0-beta.27",
  "@angular/material": "^8.2.3",
}
...
```

Effet ou paramétrages dans **angular.json** :

```
....
"styles": [
  "./node_modules/@angular/material/prebuilt-themes/indigo-pink.css",
  "src/styles.scss"
],
....
```

Type d'importations techniques à ajouter directement ou indirectement dans **app.module.ts** :

```
import { ImportMaterialModule } from './common/imports/import-material.module';
import { FlexLayoutModule } from "@angular/flex-layout";
...
@NgModule({
...
imports: [
  BrowserModule,  AppRoutingModule,  BrowserAnimationsModule,
  FlexLayoutModule,  ImportMaterialModule ,
  FormsModule,  ReactiveFormsModule
],
...
...

```

src/app/common/imports/**import-material.module.ts**

```
import { NgModule } from '@angular/core';
import { MatTabsModule } from '@angular/material/tabs';
import { MatInputModule } from '@angular/material/input';
import { MatSelectModule } from '@angular/material/select';
import { MatIconModule } from '@angular/material/icon';
import { MatMenuModule } from '@angular/material/menu';
import { MatButtonModule } from '@angular/material/button';
import { MatCheckboxModule } from '@angular/material/checkbox';
import { MatRadioModule } from '@angular/material/radio';
import { MatCardModule } from '@angular/material/card';
import { MatToolbarModule } from '@angular/material/toolbar';
import { MatFormFieldModule } from '@angular/material/form-field';
import { MatSidenavModule } from '@angular/material/sidenav';
import { MatListModule } from '@angular/material/list';
import { MatDatepickerModule } from '@angular/material/datepicker';
import { MatNativeDateModule } from '@angular/material/core';
import { MatAutocompleteModule } from '@angular/material/autocomplete';
import { MatSlideToggleModule } from '@angular/material/slide-toggle';
import { MatExpansionModule } from '@angular/material/expansion';
import { MatBadgeModule } from '@angular/material/badge';
import { MatProgressSpinnerModule } from '@angular/material/progress-spinner';
import { MatTooltipModule } from '@angular/material/tooltip';
import { MatDialogModule } from '@angular/material/dialog';
import { MatTableModule } from '@angular/material/table';
import { MatTreeModule } from '@angular/material/tree';
import { MatStepperModule } from '@angular/material/stepper';
```

```
import {MatSortModule} from '@angular/material/sort';
import {MatPaginatorModule} from '@angular/material/paginator';

@NgModule({
  imports: [
    MatTabsModule,      MatCardModule, MatExpansionModule,
    MatIconModule,      MatFormFieldModule, MatInputModule, MatSelectModule,
    MatButtonModule,    MatListModule, MatCheckboxModule, MatSlideToggleModule,
    MatRadioModule,     MatMenuModule, MatToolbarModule, MatSidenavModule,
    MatDatepickerModule, MatNativeDateModule, MatAutocompleteModule,
    MatBadgeModule, MatProgressSpinnerModule, MatTooltipModule, MatDialogModule,
    MatTableModule,    MatTreeModule, MatStepperModule, MatSortModule, MatPaginatorModule
  ],
  exports: [
    MatTabsModule,      MatCardModule, MatExpansionModule, MatIconModule, MatFormFieldModule,
    MatInputModule,     MatSelectModule, MatButtonModule, MatListModule,
    MatCheckboxModule, MatSlideToggleModule, MatRadioModule, MatMenuModule,
    MatToolbarModule,   MatSidenavModule, MatDatepickerModule, MatNativeDateModule,
    MatAutocompleteModule, MatBadgeModule, MatProgressSpinnerModule, MatTooltipModule, MatDialogModule,
    MatTableModule,    MatTreeModule, MatStepperModule, MatSortModule, MatPaginatorModule
  ]
})
export class ImportMaterialModule { }
```

2. Essentiel de "flex-layout" (en intégration angular)

npm install -s @angular/flex-layout

...html

empilement (si moins de 960px):

```
<div class="container" fxLayout.lt-md="column"
fxLayoutAlign="center" fxLayoutGap="10px" fxLayoutGap.lt-md="2px">
  <div class="a" fxFlex="25%">divA (25%)</div>
  <div class="b" fxFlex="50%">divB (50%)</div>
  <div class="c">divC</div>
</div>
```

empilement (si moins de 960px):

divA (25%) divB (50%) divC

empilement (si moins de 960px):

divA (25%)
divB (50%)
divC

breakpoints	size(px)	breakpoints	size(px)	breakpoints	size(px)
xs (extra small)	599 ou moins	lt-sm (less than small)	moins que 600		
sm (small or medium)	600 à 959	lt-md (less than md)	moins que 960	gt-xs (greater than xs)	600 ou plus
md (medium)	960 à 1279	lt-lg (less than large)	moins que 1280	gt-sm (greater than sm)	960 ou plus
lg (large)	1280 à 1919	lt-xl (less than xl)	moins que 1920	gt-md (greater than md)	1280 ou plus
xl (extra large)	1920 à 5000			gt-lg (greater than large)	1920 ou plus

3. Quelques composants "angular-material"

Card (panneau d'encadrement) :



Basic

```
<mat-card class="my-card">
  <mat-card-header class="my-card-header">
    <mat-card-title>Basic</mat-card-title>
    <!-- <mat-card-subtitle>angular material</mat-card-subtitle> -->
  </mat-card-header>

  <mat-card-content class="my-card-content">
    ...
  </mat-card-content>
</mat-card>
```

```
.basic { background: white; }
.my-card { border: 1px; border-style:solid; border-color:blue ; padding : 0 }

.my-card-header {
  background-color: rgb(23, 23, 112); color : white;
  padding-left: 1em; padding-top: 0.5em;
}

.my-card-content { padding: 1em; }
```

Composants "onglets" (tab , tab-group , ...)

```
<mat-tab-group>
  <mat-tab label="tva">
    <app-tva></app-tva>
  </mat-tab>
  <mat-tab label="titre onglet2">
    ...contenu onglet 2...
  </mat-tab>
</mat-tab-group>
```

tva

demo angular flexLayout

Composants élémentaires pour formulaires

```

<div>
<form role="form" class="form-container" >
<mat-form-field [appearance]="settingService.my_mat_appearance">
  <mat-label>ht:</mat-label>
  <input matInput placeholder="ht" name="ht" [(ngModel)]="ht" (input)="onCompute()" />
  <!-- <mat-icon matSuffix>favorite</mat-icon> -->
  <mat-hint>montant hors taxe</mat-hint>
</mat-form-field>

<mat-form-field [appearance]="settingService.my_mat_appearance">
  <mat-label>taux (en%):</mat-label>
  <mat-select placeholder="taux" name="taux" [(ngModel)]="taux"
    (selectionChange)="onCompute()">
    <mat-option *ngFor="let t of listeTaux" [value]="t">{{t}}</mat-option>
  </mat-select>
</mat-form-field>
</form>
tva: <span>{{tva | currency:'EUR':symbol:'1.0-2'}}</span> <br/>
ttc: <span>{{ttc | currency:'EUR':symbol:'1.0-2'}}</span>
</div>

```

....CSS

```

.form-container { display: flex; flex-direction: column; }
.form-container > * { width: 30%; }

```

Look avec le paramètre [appearance]=" 'standard' " :

ht:

200

montant hors taxe

taux (en%):

20

tva: €40

ttc: €240

Look avec le paramètre [appearance]=" 'outline' "

ht:

200

montant hors taxe

taux (en%):

20

NB : les valeurs possibles de *appearance* sont "standard" , "outline" , "fill" et "legacy" .

Autres composants de base (exemples):

```
<div>
  <form role="form" class="form-container" >
    <mat-form-field [appearance]=" 'standard' ">
      <!-- [hideRequiredMarker]="false" [floatLabel]="auto" by default on mat-form-field -->
      <mat-label>full name:</mat-label>
      <input matInput placeholder="name" name="name"
        minLength="6" maxLength="20" required
        [(ngModel)]= "person.name" />
      <mat-hint align="end">full name</mat-hint>
    </mat-form-field>
  </div>
  <div>
    <label>kind: </label>
    <mat-radio-group placeholder="kind" name="kind" [(ngModel)]= "person.kind" >
      <mat-radio-button matInput *ngFor="let k of listeKind" [value]="k">{{k}}</mat-radio-button>
    </mat-radio-group>
    <!-- mat-radio-group cannot be put inside mat-form-field , ??? -->
  </div>

  <mat-form-field [appearance]=" 'standard' ">
    <mat-label>email:</mat-label>
    <input matInput name="email" type="email" placeholder="email" [(ngModel)]= "person.email" />
  </mat-form-field>

  <mat-checkbox name="crazy" [(ngModel)]= "person.crazy" >crazy (as checkbox)</mat-checkbox>
  <mat-slide-toggle name="crazy" [(ngModel)]= "person.crazy" >crazy (as slide-toggle)</mat-slide-toggle>
  <!-- mat-checkbox cannot be put inside mat-form-field , already has a label -->

  <mat-form-field [appearance]=" 'standard' ">
    <mat-label>birthday:</mat-label>
    <input matInput name="birthday" [matDatepicker]= "myDatePicker"
      placeholder="birthday" [(ngModel)]= "person.birthday" /> <!-- type="date" if no picker -->
    <mat-datepicker-toggle matSuffix [for]= "myDatePicker"></mat-datepicker-toggle>
    <mat-datepicker #myDatePicker></mat-datepicker>
  </mat-form-field>

  <mat-form-field [appearance]=" 'standard' ">
    <mat-label>childNumber:</mat-label>
    <input matInput name="childNumber" type="number" placeholder="childNumber" [(ngModel)]= "person.childNumber" />
  </mat-form-field>
  <mat-form-field [appearance]=" 'standard' ">
    <mat-label>preferredColor:</mat-label>
    <input matInput name="preferredColor" type="color" placeholder="preferredColor" [(ngModel)]= "person.preferredColor" />
  </mat-form-field>
  <mat-form-field [appearance]=" 'standard' ">
    <mat-label>password:</mat-label>
    <input matInput name="password" type="password" placeholder="password" [(ngModel)]= "person.password" />
  </mat-form-field>
  <mat-form-field [appearance]=" 'standard' ">
    <mat-label>country:</mat-label>
    <input matInput name="country" placeholder="country"
      (ngModelChange)= "adjustFilteredCountries()"
      [(ngModel)]= "person.country" [matAutocomplete]= "autoCountry" />
    <!-- for reactiveForm , [formControl]= "myControl" and no ngModel -->
    <mat-autocomplete #autoCountry= "matAutocomplete">

```

```

        <mat-option *ngFor="let c of filteredCountries | async" [value]="c"> {{c}} </mat-option>
    </mat-autocomplete>
</mat-form-field>
<mat-form-field [appearance]=" 'standard' ">
    <mat-label>comment:</mat-label>
    <textarea matInput name="comment" placeholder="comment" [(ngModel)]="person.comment"></textarea>
</mat-form-field>
</form>
person: <span>{{person | json}}</span> <br/>
</div>

```

full name: *

didier defrance

kind: ☒ Male ☐ Female

email:

didier@d-defrance.fr

☒ crazy (as checkbox)

☒ crazy (as slide-toggle)

birthday:

7/11/1969

childNumber:

2

preferredColor:

password:

country:

F

France

Finlande

birthday:

7/11/1969

JUL 1969

S M T W T F S

JUL

1 2 3 4 5

6 7 8 9 10 11 12

13 14 15 16 17 18 19

20 21 22 23 24 25 26

27 28 29 30 31

preferredColor:

Couleurs

Couleurs de base :

Couleurs personnalisées :

Teinte : 160 Rouge : 0

Satur : 240 Vert : 0

Lum : 120 Bleu : 255

OK Annuler

Ajouter aux couleurs personnalisées

....ts

```

....
import { Observable, of } from 'rxjs';
import { map, startWith } from 'rxjs/operators';

@Component({ ... })
export class VariousFormComponent implements OnInit {
    listeKind = [ "Male", "Female" ];
    //myControl = new FormControl(); //if reactiveForm and autocomplete
    countries = [ "France", "Finlande", "Allemagne", "Autriche", "Italie", "Espagne", "..." ];
    filteredCountries: Observable<string[]>;

    person : Person = new Person();
    constructor() {}
    ngOnInit(){}
}

```

```

adjustFilteredCountries() {
  this.filteredCountries = of(this.countries)
    .pipe(
      map(listCountries => this._filterCountriesIgnoreCase(listCountries, this.person.country))
    );
}

private _filterCountriesIgnoreCase(listCountries: string[], value: string): string[] {
  const filterValue = value.toLowerCase();
  return listCountries.filter(c => c.toLowerCase().includes(filterValue));
}
}

```

Composants "boîte de dialogue" (exemple , autres variantes possibles)

example-dialog.component.ts

```

import { Component, OnInit, Inject } from '@angular/core';
import { MatDialogRef, MAT_DIALOG_DATA } from '@angular/material/dialog';
import { MyDialogData } from './myDialogData';

@Component({
  selector: 'app-example-dialog',
  templateUrl: './example-dialog.component.html',
  styleUrls: ['./example-dialog.component.scss']
})
export class ExampleDialogComponent implements OnInit {

  /*
  entryComponents: [ExampleDialogComponent],
  must be added in @NgModule()
  */

  constructor(
    public dialogRef: MatDialogRef<ExampleDialogComponent>,
    @Inject(MAT_DIALOG_DATA) public data: MyDialogData) {}

  onNoClick(): void { this.dialogRef.close(); }

  ngOnInit() { }
}

```

```

export class MyDialogData {
  name: string;
  animal: string;
}

```

example-dialog.component.html

```

<h1 mat-dialog-title>Hi {{data.name}}</h1>
<div mat-dialog-content>
  <p>What's your favorite animal?</p>
  <mat-form-field>

```

```

<mat-label>Favorite Animal</mat-label>
<input matInput [(ngModel)]="data.animal">
</mat-form-field>
</div>
<div mat-dialog-actions>
  <button mat-button (click)="onNoClick()">No Thanks</button>
  <button mat-button [mat-dialog-close]="data.animal" cdkFocusInitial>Ok</button>
</div>

```

Exemple d'utilisation :

.....ts

```

import { Component, OnInit } from '@angular/core';
import { MatDialog } from '@angular/material/dialog';
import { ExampleDialogComponent } from './example-dialog/example-dialog.component';

@Component({ selector: 'app-divers', templateUrl: './divers.component.html',
  styleUrls: ['./divers.component.scss']
})
export class DiversComponent implements OnInit {
  animal: string;
  name: string;

  constructor(public dialog: MatDialog) {}

  openDialog(): void {
    /*
    entryComponents: [ExampleDialogComponent],
    must be added in @NgModule()
    */
    const dialogRef = this.dialog.open(ExampleDialogComponent, {
      width: '250px',
      data: {name: this.name, animal: this.animal}
    });

    dialogRef.afterClosed().subscribe(result => {
      console.log('The dialog was closed');
      this.animal = result;
    });
  }
  ngOnInit() {}
}

```

```

<ol>
  <li>

```

```

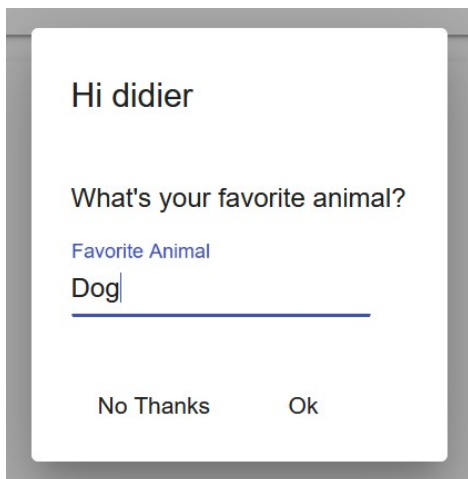
    <mat-form-field>
      <mat-label>What's your name?</mat-label>
      <input matInput [(ngModel)]="name">
    </mat-form-field>
  </li>
  <li>
    <button mat-raised-button (click)="openDialog()">open dialog</button>
  </li>
  <li *ngIf="animal">
    You chose: <i>{{animal}}</i>
  </li>
</ol>

```

What's your name?

1. didier

2. open dialog



What's your name?

1. didier

2. open dialog

3. You chose: Dog

Composants "step",

isLinear (step by step, cannot skip step)

1 Fill out your name

2 Fill out your address

3 Done

Name *

didier

Next

isLinear (step by step, cannot skip step)

1 Fill out your name

2 Fill out your address

3 Done

You are now done. (data: { "firstStep": { "name": "didier" }, "secondStep": { "address": "123 rue Elle 75000 Par ici" } })

Back Reset

Composants "mat-table" avec dataSource

table of countries (click on column header to sort)

Filter

☐	code	name ↑	capital city	area	population
☒	de	Allemagne	Berlin	357386	83073100
☒	es	Espagne	Madrid	505911	46934632
☐	fr	France	Paris	632734	67795000
☐	it	Italie	Rome	301336	60359546
☐	en	Royaume-Uni	Londres	246690	65761117
Total				2044057	323923395
Items per page: 50 1 – 5 of 5 < < > >					

Attention: la selection globale et calcul du total ne tient pas compte de la pagination (par defaut)

selected countries : [{ "code": "de", "name": "Allemagne", "capital_city": "Berlin", "population": 83073100, "area": 357386, "regions": null }, { "code": "es", "name": "Espagne", "capital_city": "Madrid", "population": 46934632, "area": 505911, "regions": null }]

NB : bien que l'exemple suivant combine "selection, filtrage , tri , totaux, pagination, ..." , chacun de ces aspects est facultatif et l'on peut dans beaucoup de cas effectuer une mise en oeuvre bien plus simple .

Dans la logique complexe/élaborée de construction/fonctionnement des tableaux "**mat-table**" , les **lignes d'entête , de données et de "totaux/pied de tableau"** seront automatiquement générés à partir des éléments complémentaires suivants :

- **liste ordonnées des noms de colonnes à afficher** (ex : *displayedColumns* coté .ts)
- **définition abstraite de chaque colonnes** (<ng-container **matColumnDef**="colNamexy">)
- **source de données** (intermédiaire "**dataSource**" entre données et vue , prenant en compte les **tris** et éventuels filtrages,)

with-table.component.ts

```
import { Component, OnInit, ViewChild } from '@angular/core';
import { GeoService } from '../common/service/geo.service';
import { MatTableDataSource } from '@angular/material/table';
import { MatSort } from '@angular/material/sort';
import { Country } from '../common/data/country';
import { SelectionModel } from '@angular/cdk/collections';
import { MatPaginator } from '@angular/material/paginator';

@Component({
  selector: 'app-with-table',
  templateUrl: './with-table.component.html',
  styleUrls: ['./with-table.component.scss']
})
export class WithTableComponent implements OnInit {

  displayedColumns: string[] = ['select','code', 'name', 'capital_city', 'area', 'population'];
  countries : Country[] = [];
  dataSource = new MatTableDataSource(this.countries); //this.countries; if no sort

  selection = new SelectionModel<Country>(true /*allowMultiSelect*/, [] /*initialSelection*/);

  constructor(private geoService: GeoService) { }

  //sort refer to table with matSort directive in .html
  //useful for get order choice (by name , by population, ...)
  @ViewChild(MatSort, {static: true}) sort: MatSort;

  @ViewChild(MatPaginator, {static: true}) paginator: MatPaginator;

  ngOnInit() {
    this.geoService.getCountries().subscribe(
      (countries)=> { /*this.dataSource=countries; if no sort*/
        this.countries=countries;
        this.dataSource= new MatTableDataSource(countries);
        this.dataSource.sort=this.sort; //by name or by ...
        this.dataSource.paginator = this.paginator;
      }
    );
  }
}
```

```

    this.dataSource.filterPredicate =
      (data: Country, filter: string) => !filter || (data.name.toLowerCase()).includes(filter);
  }
);
}

applyFilter(event: Event) {
  const filterValue = (event.target as HTMLInputElement).value;
  this.dataSource.filter = filterValue.toLowerCase();
}

/** Gets the total population and total area of all countries. */
getTotalPopulation() {
  //this.countries.map(..) or this.dataSource.filteredData.map(...)
  return this.dataSource.filteredData.map(c => c.population)
    .reduce((acc, value) => acc + value, 0);
}

getTotalArea() {
  return this.dataSource.filteredData.map(c => c.area)
    .reduce((acc, value) => acc + value, 0);
}

/** Whether the number of selected elements matches the total number of rows. */
isAllSelected() {
  const numSelected = this.selection.selected.length;
  const numRows = this.dataSource.data.length;
  return numSelected === numRows;
}

/** Selects all rows if they are not all selected; otherwise clear selection. */
masterToggle() {
  this.isAllSelected() ?
    this.selection.clear() :
    this.dataSource.data.forEach(row => this.selection.select(row));
}

/** The label for the checkbox on the passed row */
checkboxLabel(row?: Country): string {
  if (!row) {
    return `${this.isAllSelected() ? 'select' : 'deselect'} all`;
  }
  return `${this.selection.isSelected(row) ? 'deselect' : 'select'} row ${row.code}`;
}
}

```

with-table.component.css

```

.selected {
  background-color: red;
}

.mat-row.highlighted {

```

```

background: lightblue;
}

table {
  width: 100%;
  overflow-x: auto;
  overflow-y: hidden;
  min-width: 500px;
}

.mat-header-cell, .mat-sort-header {
  background-color: lightblue;
  font-weight: bold;
}

.my-table-container-for-scroll{
  height: 400px;
  overflow: auto;
}

tr.mat-footer-row {
  font-weight: bold;
}

```

with-table.component.html

```

<h4>table of countries (click on column header to sort)</h4>

<mat-form-field>
  <mat-label>Filter</mat-label>
  <input matInput (keyup)="applyFilter($event)"
    placeholder="Ex. i">
</mat-form-field>
<div class="my-table-container-for-scroll">
<table mat-table [dataSource]="dataSource" matSort>

<!-- Note that these columns can be defined in any order.
  The actual rendered columns are set as a property on the row definition -->

  <!-- code Column -->
  <ng-container matColumnDef="code">
    <th mat-header-cell *matHeaderCellDef> code </th>
    <td mat-cell *matCellDef="let c"> {{c.code}} </td>
    <td mat-footer-cell *matFooterCellDef>Total</td>
  </ng-container>

  <!-- Name Column -->
  <ng-container matColumnDef="name">
    <th mat-header-cell *matHeaderCellDef mat-sort-header> name </th>
    <td mat-cell *matCellDef="let c"> {{c.name}} </td>
    <td mat-footer-cell *matFooterCellDef></td>
  </ng-container>

  <!-- Capital-city Column -->
  <ng-container matColumnDef="capital_city">

```

```

    <th mat-header-cell *matHeaderCellDef> capital city </th>
    <td mat-cell *matCellDef="let c"> {{c.capital_city}} </td>
    <td mat-footer-cell *matFooterCellDef></td>
</ng-container>

<!-- Population Column -->
<ng-container matColumnDef="population">
  <!-- mat-sort-header only ok if matSort in table -->
  <th mat-header-cell *matHeaderCellDef mat-sort-header> population </th>
  <td mat-cell *matCellDef="let c"> {{c.population}} </td>
  <td mat-footer-cell *matFooterCellDef> {{getTotalPopulation()}}</td>
</ng-container>

<!-- Area Column -->
<ng-container matColumnDef="area">
  <th mat-header-cell *matHeaderCellDef mat-sort-header> area </th>
  <td mat-cell *matCellDef="let c"> {{c.area}} </td>
  <td mat-footer-cell *matFooterCellDef> {{getTotalArea()}}</td>
</ng-container>

<!-- Checkbox selection Column -->
<ng-container matColumnDef="select">
  <th mat-header-cell *matHeaderCellDef>
    <mat-checkbox (change)="$event ? masterToggle() : null"
      [checked]="selection.hasValue() && isAllSelected()"
      [indeterminate]="selection.hasValue() && !isAllSelected()"
      [aria-label]="checkboxLabel()">
    </mat-checkbox>
  </th>
  <td mat-cell *matCellDef="let row">
    <mat-checkbox (click)="$event.stopPropagation()"
      (change)="$event ? selection.toggle(row) : null"
      [checked]="selection.isSelected(row)"
      [aria-label]="checkboxLabel(row)">
    </mat-checkbox>
  </td>
  <td mat-footer-cell *matFooterCellDef></td>
</ng-container>

<tr mat-header-row *matHeaderRowDef="displayedColumns; sticky: true"></tr>
<tr mat-row *matRowDef="let row; columns: displayedColumns;"
  > <!-- (click)="selection.toggle(row)" --> </tr>
<!-- if mat-footer-row , mat-footer-cell must be defined for each displayedColumns -->
<tr mat-footer-row *matFooterRowDef="displayedColumns;"></tr>
</table>
<mat-paginator [pageSizeOptions]="[50, 20, 12, 6, 3]" showFirstLastButtons></mat-paginator>
</div>

```

Attention: la selection globale et calcul du total ne tient pas compte de la pagination (par default)

selected countries : {{selection._selected | json}}

XIX - Annexe – internationalisation angular (i18n)

1. internationalisation (i18n)

$i18n = i + 18\text{caractères} + n$

1.1. extension nécessaire

Depuis la version 9 de angular, il faut ajouter l'extension @angular/localize

```
ng add @angular/localize
```

1.2. Mode opératoire

1. Par ajout de `i18n="..."` au sein des **templates html** , *marquer les zones ayant besoins d'être traduites (le texte original doit idéalement être en anglais)*
2. générer le fichier de traduction `src/locale/messages.xlf` via la commande
`ng extract-i18n --output-path src/locale`

1.3. paramétrage au niveau d'un template html

```
<h1 i18n="main header|Friendly welcoming message">Welcome!</h1>
```

```
<h1 i18n="main header|Friendly welcoming message@@welcome">Welcome!</h1>
```

i18n (attribut sans valeur , sans indication) demande simplement à effectuer une traduction du texte de la balise.

i18n="texte_description" demande une traduction en fonction d'une description

i18n="meaning | description" demande une traduction en fonction d'une signification et d'une description .

i18n="@@custom_translation_id" ou

i18n="description@@custom_translation_id" ou

i18n="meaning | description@@custom_translation_id" permet en plus de contrôler la valeur de l'id qui sera généré dans le fichier de traduction en lui affectant une valeur parlante/significative et non changeante (pas partiellement aléatoire).

Selon les paramétrages des fichiers de traduction, toutes des traductions associées à la même signification auront par défaut la même valeur .

Souvent utilisé au niveau d'une véritable balise html (ex : <p> , <h3> , ...) l'attribut i18n peut également être utilisé sur <ng-container> ce qui aura pour effet de ne générer que le texte traduit (sans balise html englobante)

Exemple: <ng-container i18n>I don't output any element</ng-container>

Application d'une traduction au niveau d'un attribut :

```
<img [src]="logo" i18n-title title="Angular logo" />
```

Via *i18n-attributeName*, la valeur de l'attribut est alors d'abord traduite avant d'être utilisée comme une propriété dans l'arbre DOM.

1.4. Fichiers de traduction

Format par défaut : [XML Localization Interchange File Format \(XLIFF\)](#), version 1.2)

Autres formats supportés : XLIFF 2 et XMB (Xml Message Bundle).

Un fichier xliif a généralement l'extension .xlf

NB : l'ancienne commande "~~ng xi18n --out-file src/locale/messages.xlf~~" (époque angular 8) a été remplacée par "**ng extract-i18n --output-path** src/locale".

```
ng extract-i18n --output-path src/locale
```

options de la commande : --format=xlf ou --format=xlf2 ou --format=xmb

→ fichier généré : **src/locale/messages.xlf**

Début de variante avec traductions :

```
src/locale/messages.fr.xlf (que l'on peut créer par copier/coller)
```

//attention, le contenu du fichier **messages.fr.xlf** reste à traduire manuellement ou automatiquement !!!

messages.de.xlf (généré par copier/coller ou autrement)

```
<?xml version="1.0" encoding="UTF-8" ?>
<xliif version="1.2" xmlns="urn:oasis:names:tc:xliif:document:1.2">
  <file source-language="de" datatype="plaintext" original="ng2.template">
    <body>
      <trans-unit id="e27c4e1996f81811e5f18c03bfc30af4db0650cf" datatype="html">
        <source>welcome</source>
        <context-group purpose="location">
          <context context-type="sourcefile">
            src/app/advanced/with-traduction/with-traduction.component.html</context>
          <context context-type="linenumber">2</context>
        </context-group>
      </trans-unit>
    </body>
  </file>
</xliif>
```

--> baser des paramétrage de traduction uniquement sur des numéros de ligne qui peuvent changer, ce n'est pas très viable.

```
<h3 i18n="generic.welcome|my simple welcome message">welcome</h3>
```

```
<button i18n="ok">ok</button> <br/>
```

==>

...

```

<trans-unit id="50c1a51a1c7bd808f40a497ca24543b5b6612169" datatype="html">
  <source>welcome</source>
  <target>bienvenu</target> <!-- a ajouter (pas trop tot) -->
  <context-group purpose="location">... </context-group>
  <note priority="1" from="description">my simple welcome message</note>
  <note priority="1" from="meaning">generic.welcome</note>
</trans-unit>
<trans-unit id="e644f6231bef1a54e64391b9c6856d3b92822a8f" datatype="html">
  <source>ok</source>
  <context-group purpose="location">....</context-group>
  <note priority="1" from="description">ok</note>
</trans-unit>
...

```

Attention, un ajout manuel de **<target>traduction</target>** peut être potentiellement écrasé en régénérant le fichier via certaines commandes

1.5. Eventuelles traductions via xlf-translate

```
npm install -g xlf-translate
```

Si au sein des templates html , les "*meaning*" ont été précisés via `i18n="generic.welcome|welcome message"` on peut alors se faire aider par l'utilitaire *xlf-translate* de manière à remplir les parties manquantes `<target> ... </target>` d'un fichier *messages.fr.xlf* ou autre .

Après `ng extract-i18n --output-path src/locale` et après avoir créer `src/locale/messages.fr.xlf` comme une copie de `src/locale/messages.xlf` on peut :

créer un fichier de traduction tel que

translate.fr.yml

```

generic:
  welcome : bienvenu
  ok : ok
  cancel : annuler

```

lancer_traduction.bat

```

REM npm install -g xlf-translate
xlf-translate --lang-file translate.fr.yml messages.fr.xlf
pause

```

Ceci permet de générer toutes les parties manquantes `<target> ... </target>` dans le fichier **messages.fr.xlf** qui est modifié sur place.

Par défaut, seules les parties manquantes sont ajoutées sans écrasement des anciennes valeurs (sauf si option **--force**)

1.6. Configuration des traductions au niveau du projet angular

Dans le haut de **angular.json**

```
{
  "$schema": "./node_modules/@angular/cli/lib/config/schema.json",
  "version": 1,
  "newProjectRoot": "projects",
  "projects": {
    "myapp": {
      "projectType": "application",
      "schematics": {
        "@schematics/angular:component": {
          "style": "css"
        }
      },
      "root": "",
      "prefix": "app",
      "i18n": {
        "sourceLocale": "en",
        "locales": {
          "fr": {
            "translation": "src/locale/messages.fr.xlf", "subPath": "fr"
          },
          "de": {
            "translation": "src/locale/messages.de.xlf", "subPath": "de"
          }
        }
      },
      "architect": {
        ...
      }
    }
  }
}
```

Dans le milieu de **angular.json**

```
{
  "projects": {
    "myapp": {
      "architect": {
        "build": {
          ...
        },
        "configurations": {
          "fr": {
            "localize": ["fr"],
            "outputPath": "dist/exp-adv-app-fr/",
            "i18nMissingTranslation": "warning"
          },
          "de": {
            "localize": ["de"],
            "outputPath": "dist/exp-adv-app-de/",
            "i18nMissingTranslation": "warning"
          },
          ...
        },
        "production": {
          ...
        }
      }
    }
  }
}
```

- Sans "i18nMissingTranslation": **"error"** ou **"ignore"**, des **"warning"** par défaut.
- Pour la production, un éventuel paramétrage supplémentaire de type "baseHref": "/fr/" peut quelquefois être utile (par exemple pour switcher de langue via des routes commençant par

"/en/..." ou "/fr/..." ou autre) .

et encore plus bas dans **angular.json** :

```
{
  "projects": {
    "myapp": {
      "architect": {
        "serve": {
          "builder": "@angular-devkit/build-angular:dev-server",
          "options": {
            "browserTarget": "myapp:build"
          },
          "configurations": {
            "production": {
              "browserTarget": "myapp:build:production"
            },
            "fr": {
              "buildTarget": "myapp:build:fr"
            },
            "de": {
              "buildTarget": "myapp:build:de"
            }
          }
        }
      }
    }
  }
}
```

NB : au sein d'une ancienne version de angular.json, "buildTarget" était "browserTarget".

dans **package.json**

```
...
"scripts": {
  "ng": "ng",
  "start": "ng serve",
  "start:fr": "ng serve --configuration=fr",
  "start:de": "ng serve --configuration=de",
  "build": "ng build",
  "test": "ng test",
  "lint": "ng lint",
  "build:fr": "ng build --configuration=fr",
  "build:de": "ng build --configuration=de",
  "e2e": "ng e2e"
},
...
```

Ceci permet de lancer des commandes de ce type

```
npm run start:fr -- --port=4201
npm run start:de -- --port=4202
```

http://localhost:4200 --> sans traduction

http://localhost:4201 --> traductions françaises

http://localhost:4202 --> traductions allemandes

1.7. aspects avancés / internationalisation

Avec ou sans "s" (ou bien afficher "un" ou "plusieurs")

--> pluralization :

```
<span i18n>Updated {minutes, plural, =0 {just now} =1 {one minute ago} other
{{{minutes}}} minutes ago}</span>
```

avec une interprétation spéciale dite "ICU" sur la pluralité de la variable minutes (sorte de switch/case sur la valeur de minutes)

avec dans le fichier de traduction :

```
<trans-unit id="5a134dee893586d02bffc9611056b9cadf9abfad" datatype="html">
  <source>{VAR_PLURAL, plural, =0 {just now} =1 {one minute ago} other {<x
id="INTERPOLATION" equiv-text="{minutes}"> minutes ago} }</source>
  <target>{VAR_PLURAL, plural, =0 {à l'instant} =1 {il y a une minute} other {il
y a <x id="INTERPOLATION" equiv-text="{minutes}"> minutes} }</target>
</trans-unit>
```

Alternatives :

```
<span i18n>The author is {gender, select, male {male} female {female} other
{other}}</span>
```

avec dans le fichier de traduction :

```
<trans-unit id="eff74b75ab7364b6fa888f1cbfae901aaaf02295" datatype="html">
  <source>{VAR_SELECT, select, male {male} female {female} other {other}
}</source>
  <target>{VAR_SELECT, select, male {un homme} female {une femme} other
{autre} }</target>
</trans-unit>
```

1.8. Eventuel choix dynamique d'une langue

Principe :

- générer plusieurs versions "fr", "en", "de" de l'application angular avec "baseHref": "/fr/" ou "baseHref": "/en/" , ...
- dans un menu principal de l'application (à base de liste déroulante par exemple) , faire en sorte que l'on puisse déclencher des liens hypertextes en "/en/" ou "/fr/" ,

Exemple :

```
<ul>
  <li *ngFor="let language of languageList">
    <a href="/{{language.code}}/">
      {{language.label}}
    </a>
  </li>
</ul>
```

Pour que ça fonctionne, il faut bien paramétrer une configuration cohérente au niveau du serveur HTTP (ex : nginx) vis à vis des répertoires et des "baseHref" .

Configuration importante dans **angular.json** :

...

```

"prefix": "app",
  "i18n": {
    "sourceLocale": "en",
    "locales": {
      "fr": {
        "translation": "src/locale/messages.fr.xlf",
        "subPath": "fr"
      },
      "de": {
        "translation": "src/locale/messages.de.xlf",
        "subPath": "de"
      }
    }
  },
  "architect": { ...

```

Résultat par défaut de ng build	ng build --localize
▼ dist\adv-app ▼ browser ★ favicon.ico <> index.html JS main-5BLPT5II.js JS polyfills-7RKI4BGK.js # styles-D3QTZVDM.css	▼ dist\adv-app ▼ browser ▼ de ★ favicon.ico <> index.html JS main-MAG77BXF.js JS polyfills-Z656XY6O.js # styles-D3QTZVDM.css > en > fr

XX - Annexe – Enoncés de TP (angular avancé)

1. Préparations et installations

Eventuelle installation de nodeJs et npm (si nécessaire):

Si **node -v** et **npm -v** se sont pas des commandes reconnues (dans un terminal CMD ou autre) alors télécharger et installer **nodeJs** (pour windows 64 bits ou autre en version LTS). Relancer ensuite un nouveau terminal et lancer **npm -v** pour vérifier.

Eventuelle installation de typescript (si nécessaire):

Si **tsc -v** est une commande inconnue (dans un terminal), alors lancer la commande suivante :

```
npm install -g typescript
```

Installation de angular-cli :

Si **ng -help** est une commande non reconnue , alors lancer la commande suivante :

```
npm install -g @angular/cli
```

Installation du cli nx :

Si **nx --version** est une commande non reconnue , alors lancer la commande suivante :

```
npm add --global nx
```

Installer si nécessaire l'IDE **Visual Studio code**.

2. TD "création d'un workspace Nx"

Créer un workspace nx en mode "angular-monorepo"

Voici quelques choix possibles :

- ✓ Application name · **my-nx-workspace**
- ✓ Which bundler would you like to use? · **esbuild**
- ✓ Default stylesheet format · **css**
- ✓ Do you want to enable Server-Side Rendering (SSR) and Static Site Generation (SSG/Prerendering)? · **Yes**
- ✓ Which unit test runner would you like to use? · **vitest-angular**
- ✓ Test runner to use for end to end (E2E) tests · **playwright** ou **cypress**
- ✓ Speed up GitHub Actions, GitLab CI, and more with Nx Cloud? · **skip**
- ✓ Help improve Nx by sharing your usage data? · **No**

NB :

- *Sans être compliquée, cette phase de préparation est assez longue ...*
- *Lorsque le workspace nx aura été créé , on pourra le charger/ouvrir (via "openFolder") dans VisualStudioCode ou bien un IDE équivalent.*

3. TP "début application principale"

Au sein du workspace nx créé précédemment , créer un nouveau projet de type "**angular:app**" (ex : **my-app**) via une commande adéquate de nx.

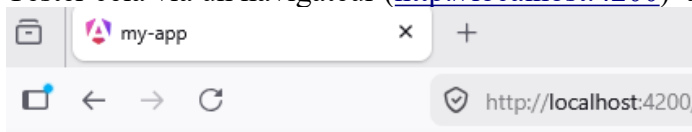
Au départ , esbuild et pas encore besoin de SSR .

Attention : Si la commande "nx show projects" n'arrive pas à trouver/afficher le projet my-app il faudra peut être supprimer ou renommer le fichier .gitignore du répertoire my-nx-workspace .

Structurer le début de cette application avec :

- un sous composant "header" (qui comportera quelques liens de navigation : vers welcome et autres), via le .css on appliquera une couleur de fond "lightcyan" ou autre sur header .
- un composant "welcome" que l'on atteindra via la route par défaut

Tester cela via un navigateur (<http://localhost:4200>) après avoir lancer **nx serve my-app**



header - [welcome](#)

welcome works!

4. TP "Service HTTP avec RxJs"

Phase1 :

- Coder un service angular "ZipService" qui permettra d'invoquer l'api rest http://api.zippopotam.us/fr/75001_ou_autre
- Injecter ce service dans un nouveau composant angular (ex : ZipCurrencyComponent) qui affichera au minimum les données d'un lieu selon la valeur de son code postal
- Ajouter de quoi naviguer vers le "ZipCurrencyComponent" et tester le bon fonctionnement

header - [welcome](#) [zip](#)

zippopotam api call

zip :

data : { "country": "France", "country abbreviation": "FR", "post code": "75001", "places": [{ "place name": "Paris 01 Louvre", "longitude": "2.3417", "latitude": "48.8592", "state": "Île-de-France", "state abbreviation": "A8" }] }

éléments techniques à mettre en place :

- classes ou interfaces ZippopotamResponse et ZippopotamPlace dans common/data/Zippopotam.ts selon structure de la réponse à <http://api.zippopotam.us/fr/75001>
- provideHttpClient() dans app.config.ts
- inject(HttpClient) et méthode getZippopotamData\$() dans ZipService
- importation de FormsModule et JsonPipe dans ZipCurrencyComponent

- inject(ChangeDetectorRef); et this.changeDetector.markForCheck(); dans ZipCurrencyComponent

Phase2 (facultative mais conseillée):

- Ajouter de nouvelles fonctionnalités dans "ZipCurrencyComponent" et des services en arrière plan qui exploiteront certains opérateurs RxJs (filtrage ou tri ou autres combinaisons)

Quelques Idées/Suggestions :

currencies

**** base : EUR / Euro ****

USD : US Dollar , JPY : Japan Yen ,

GBP : Great Britain Pound (Livre sterling) ,

DKK : Danemark Couronne/Krone ,

CNY: ChiNa Yuan, KRW : KoRea Won , ...

order:

filteringCurrencyCodes:

```
currencies : [ { "code": "CNY", "value": 7.902704 }, { "code": "DKK", "value": 7.472058 }, {
"code": "EUR", "value": 1 }, { "code": "GBP", "value": 0.862938 }, { "code": "JPY", "value":
184.929496 }, { "code": "KRW", "value": 1751.743686 }, { "code": "USD", "value": 1.16306 }
]
```

Le web service public fixer.io retourne les taux de change des principales monnaies.
Son url est :

let apiKey = "26ca93ee7fc19cbe0a423aaa27cab235"; //ici avec apiKey de didier

let wsUrl = `http://data.fixer.io/api/latest?access\_key=\${apiKey}`

La réponse de ce web service est dans ce format :

```
{ "success": true, "timestamp": 1779774247, "base": "EUR", "date": "2026-05-26", "rates":
{ "AED": 4.271343, ..., "EUR": 1, ..., "GBP": 0.862938, ..., "USD": 1.16306, ..., "ZWL": 374.504938 } }
```

Via une combinaison adéquate d'opérateurs RxJs (dans partie .pipe()) , on pourra (par exemple) :

- afficher la réponse (brute, d'origine) dans la console du navigateur
- transformer les taux de change du format d'origine vers un format liste de "Currency"
- trier soit par code de monnaie croissant , soit par valeur du taux de change
- filtrer les codes de monnaies intéressantes (ex : EUR,USD,GBP)

```
{ "EUR": 1, ..., "GBP": 0.862938, ..., " } { "code": "EUR", "value": 1 }, { "code": "GBP", "value":
```

USD":1.16306 }	0.862938 }, { "code": "USD", "value": 1.16306 }
export interface CurrencyMap{ [index: string] : number; }	export class Currency { constructor(public code : string, public value : number){} }

Pour tester (facultativement) , un observable d'ordre 2 , on pourra éventuellement enchaîner/combiner via mergeMap() ou autre deux appels de Web Services (dans la partie Zip):

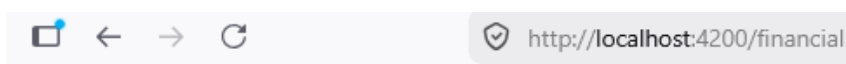
- 1) via un appel au WS "zippopotam" on récupère les coordonnées GPS (longitude , latitude) à partir du codePostal (zip)
- 2) via un appel au WS "openweathermap" on récupère des informations météorologiques à partir des coordonnées géographiques

Exemple d'appel au WS "openweathermap":

```
let apiKey = "27a68278aebec75af9d4fc23d7a68f75"; //ou autre après inscription
//let lat = 48.856614;//latitude (ici de Paris)
//let lon = 2.3522219;//longitude (ici de Paris)
let wsUrl = `https://api.openweathermap.org/data/2.5/weather?lat=${geoCoord.lat}&lon=${geoCoord.lon}&appid=${apiKey}`
return this.http.get<any>(wsUrl);
```

5. TP "composant avec signaux"

- Programmer un nouveau composant "FinancialComponent" qui servira à saisir certains paramètres tels que "nbMonth" , "amount" , "annualRate" et qui calculera et affichera la somme à rembourser chaque mois pour rembourser un emprunt à mensualité constante .
- On placera la formule de calcul dans un service angular "FinancialService" à injecter.
- On utilisera au maximum des signaux angular (ex : signal() , computed() , ...)



header - [welcome](#) [zip](#) [financial](#)

financial computation

nbMonth:
amount :
annualRate (%):

monthlyPayment: 942.6990170396044

Formule de calcul :

```
monthlyPaymentValue(nbMonth : number, amount : number , annualRate : number):number {
    let monthRate = (annualRate / 12)/100;
    return (amount * monthRate) / ( 1 - Math.pow(1+monthRate, -nbMonth));
}
```

6. TP avec Observable et signal

Au sein du composant "**ZipCurrencyComponent**", modifier au moins une partie du code de façon à ce qu'une valeur observable issue d'un appel HTTP soit transformée en signal prêt à être affiché.

7. TP "composants réutilisables" dans projet my-lib

- Au sein du workspace Nx, créer un nouveau sous projet "my-lib" de type angular:lib (au sens qui peut plus peut moins, on pourra placer les options **--publishable** **--importPath my-lib** lors de la création de la librairie même si on ne s'en sert pas en TP).
- Au sein de ce projet "my-lib" créer au moins un composant réutilisable
- Au sein du projet "application principale", ajouter quelque part une utilisation d'un des composants réutilisables de "my-lib"

Idee de composants réutilisables :

- selector (un choix de type string en output() à partir d'une liste d'éléments en input())
- togglePanel avec titre et sous composant paramétrable et panelOpenState de type model()
- confirmDialog : boîte de dialogue de confirmation (message, "yes", "no", ...)

configuration et éléments à mettre en place :

- src/index.ts de my-lib
- fichier package.json dans my-lib et my-app (avec partie "dependencies")
- imports nécessaires (ex : import {XyzComponent} from '@my-nx-workspace/my-lib');

Exemple de résultats :

financial computation

☐ loan computing

nbMonth:
 amount :
 annualRate (%):

monthlyPayment: 942.6990170396044

☒ loan computing

<lib-selector> appliqué dans Welcome:

couleurChoisie = green

welcome to my-app simple-selector choix couleur • red • green • blue couleurChoisie = green	reset couleurChoisie to blue ? ✖ No/Cancel ✔ Yes/Ok/Confirm
--	---

8. TP "lazy loading de composant et sous-routes"

1. Mettre en place un "lazy loading de composant" sur "ZipCurrencyComponent"
2. On testera/visualisera si possible le comportement "lazy"
3. Décomposer "ZipCurrencyComponent" en "partie menu avec sous routes" + nouveaux sous composants tels que "ZipComponent" + "CurrencyComponent"
4. Réadapter le LazyLoading via un "loadChildren" basé sur m.zipCurrencyRoutes

header - [welcome](#) [zip_currency](#) [financial](#) [person](#) [catFact](#)

zippopotam , weather and currency call

[zip](#) [currency](#) ...

zip , geoCoord and weather

zip :

data :

geoCoord :

wheater :

9. TP facultatif/délicat "architecture micro-frontend"

- Au sein du workspace nx , créer un nouveau sous projet "mfe1" de type application angular
- Ajouter les fonctionnalités "federative micro-frontend" au niveau de l'application principale (rôle "host") et du projet "mfe1" (rôle "remote")
- **En cas de soucis de versions → npm install --legacy-peer-deps et si c'est nécessaire certains redémarrages peuvent quelquefois être utiles.**
- déplacer toute la partie "ZipCurrency" (service + composants + sous routes) dans le projet "mfe1"
- ajouter toutes les configurations nécessaires aux communications "micro-frontend"

- tester le bon fonctionnement :
nx serve mfe1 --port=4201
nx serve my-app
<http://localhost:4201> et <http://localhost:4200>
- Attention toSignal() n'est pas toujours bien compatible avec l'api "federative micro-frontend" (selon versions). Revenir si besoin à un code sans .toSignal().

NB: Si au moment du Tp , le client nx est un peu trop fragile (pas parfaitement synchronisé vis à vis de la dernière version d'angular et de "micro-frontend") , on pourra alors se rabattre sur plusieurs projets séparés (basés sur le client ordinaire "ng" et pas "nx") .

10. TP "formulaire avancé ou dynamique"

Phase 1 :

- Au sein l'application principale , créer un composant "PersonComponent" comportant un formulaire de saisie pour une données de type data.Person (avec "firstname" , "lastname" , email et age) . On pourra au choix s'appuyer sur l'api ReactiveForm/FormBuilder ou bien sur la nouvelle api SignalForm

Phase 2 (facultative) :

- Au sein du sous projet my-lib , créer un nouveau composant réutilisable "DynamicForm"
- Retoucher le composant "PersonComponent" au sein de l'application principale de façon à ce qu'une partie soit déléguée à "DynamicForm" .

En cas d'erreur de validation :

signalForm (person) in responsive tailwind css grid

firstname:

lastname:

email:

Please enter a valid email address

age:

You must be at least 18 years old

Lorsque tout est bien saisi :

message: valeurs

saisies={"firstname":"jean","lastname":"Bon","email":"jean.bon@xyz.com","age":"40"}

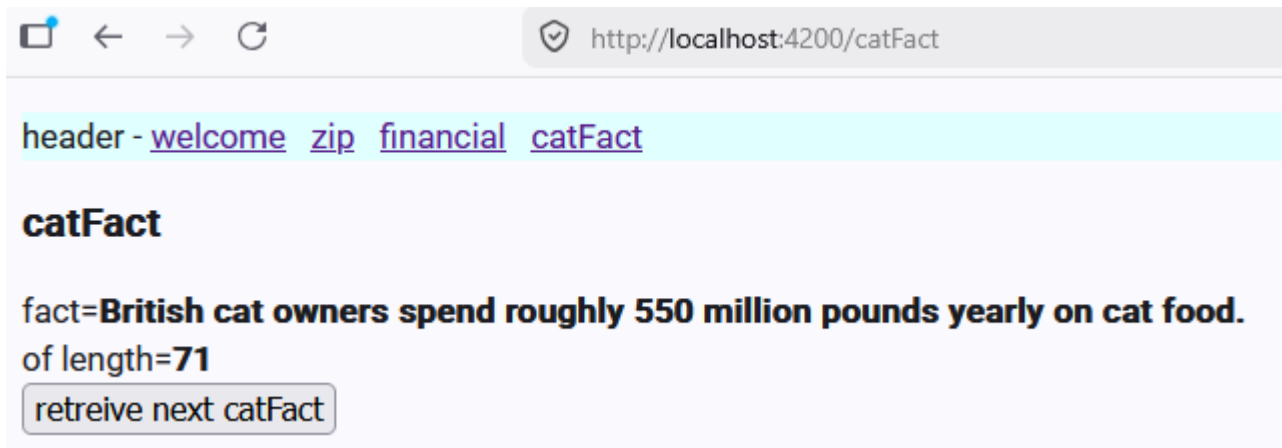
11. TP libre "gestion avancée des états (ngrx ou ...)"

- Ajouter un nouveau sous composant "AvisComponent" (éventuellement/facultativement décomposé en sous composants) qui s'appuiera par exemple sur **ngrx** de manière à bien gérer l'état d'une donnée "moyenne des avis" (note allant de 1 à 5 , exemple 3.7)

12. TP "resolver"

La web service public <https://catfact.ninja/fact> renvoi un des faits sur les chats au format suivant :
 {"fact":"Cats make about 100 different sounds. Dogs make only about 10.", "length":62}

- Programmer hyper rapidement/simplement "CatFactService" + "CatFactComponent" de manière à afficher le résultat du service rest <https://catfact.ninja/fact>
- Au sein du service basé sur HTTP+RxJs , ajouter dans la méthode d'appel en mode get un ajout de temporisation (pause) variable (par exemple 2000 ms) via **.pipe()** et l'opérateur **delay(nbMs)**
- Visualiser un effet "navigation vers vide + affichage différé après attente de récupération des données"
- Ajouter un resolver "catFactResolver" au niveau de la route permettant de naviguer vers "CatFactComponent"
- Visualiser navigation retardée + affichage en un seul coup des données pré-chargées .



13. TP "tests unitaires"

- Tester unitairement le service "FinancialService"
Pour un lancement via nx: `nx test my-app --include=**/financial.service.spec.ts`
- Tester unitairement le composant "FinancialComponent" sur la fonctionnalité calcul de mensualité à rembourser.
Pour éviter toute ambiguïté, on ajoutera si besoin des `type="number"` et `name="..."` sur les zones `<input ...>` du composant FinancialComponent et l'affichage du résultat sera encadré dans une balise `<span>` ayant un `id` connu.

14. TP "test e2e"

- Effectuer un test de calcul de mensualité en mode "e2e" soit avec cypress, soit avec playwright

Pour un lancement via nx et playwright :

`npx playwright install` (au préalable si besoin)

`nx e2e my-app-e2e`

et pas absolument besoin de lancer préalablement `nx serve my-app`.

15. TP "angular SSR"

- Ajouter la fonctionnalité "@angular/ssr"
- Ajuster si besoin certains points (ex : accès indirect à sessionStorage ou localStorage)
- effectuer quelques "build" et tests

16. TP libre "selon intérêts (PWA ou Oauth2 ou i18n)"

Ajouter et tester au moins une des fonctionnalités suivantes (à ajouter à l'application principale) :

- ProgressiveWebApp
- internationalisation
- authentication OAuth2 et gardien
- ...